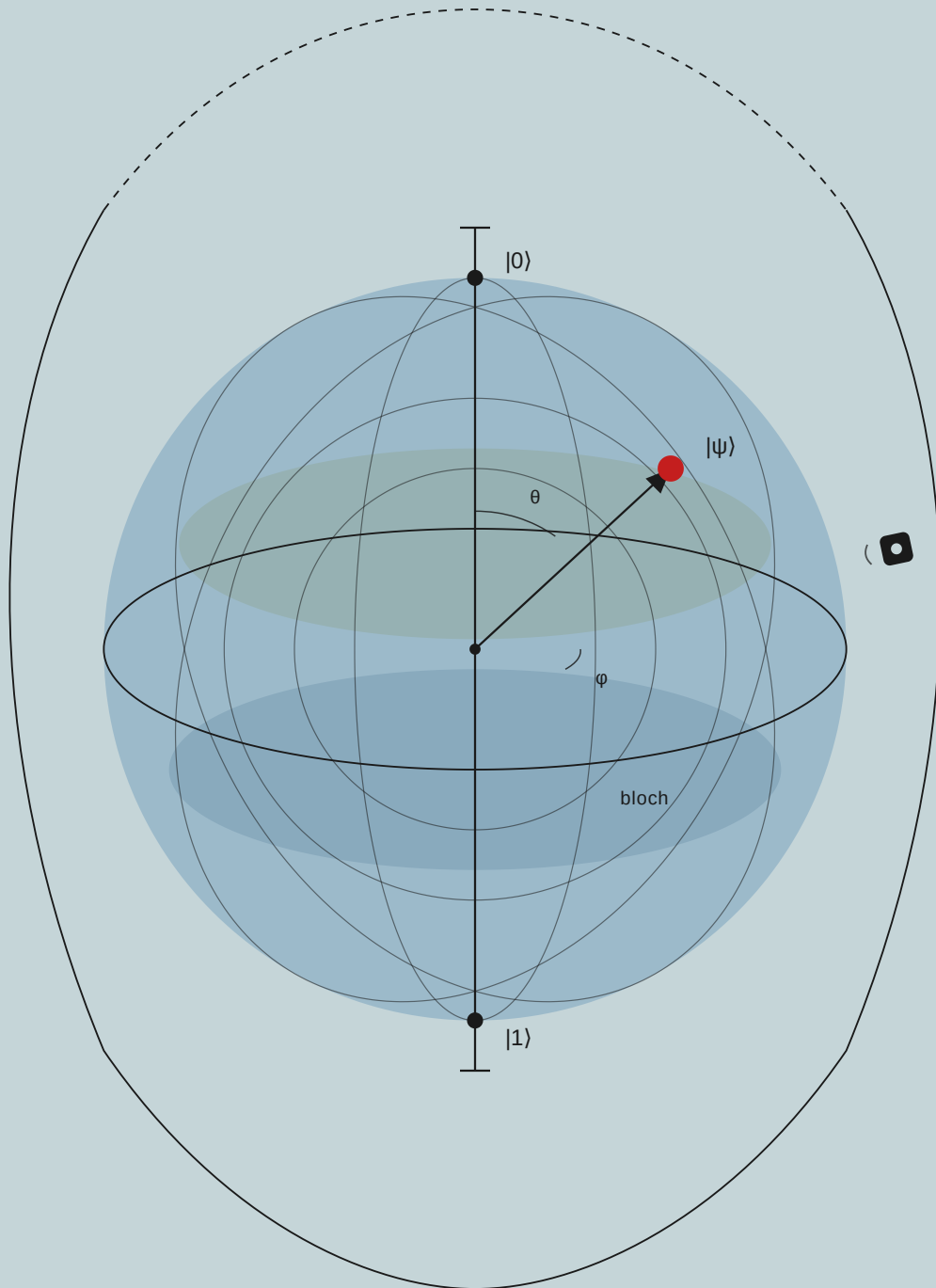


INTELLIGENCE AT THE QUANTUM SCALE



CODA 2

AI → QPU · TECHNICAL MANUAL

The instrument

Coda 2 is a **three-pane instrument** for a quantum processing unit. The first thing you see is not a chat void. It is the stage: **left drift** · **center device 3D** · **right cryostat**, with a bottom-third dock for goals. Visual, text, and math stay in view at once.

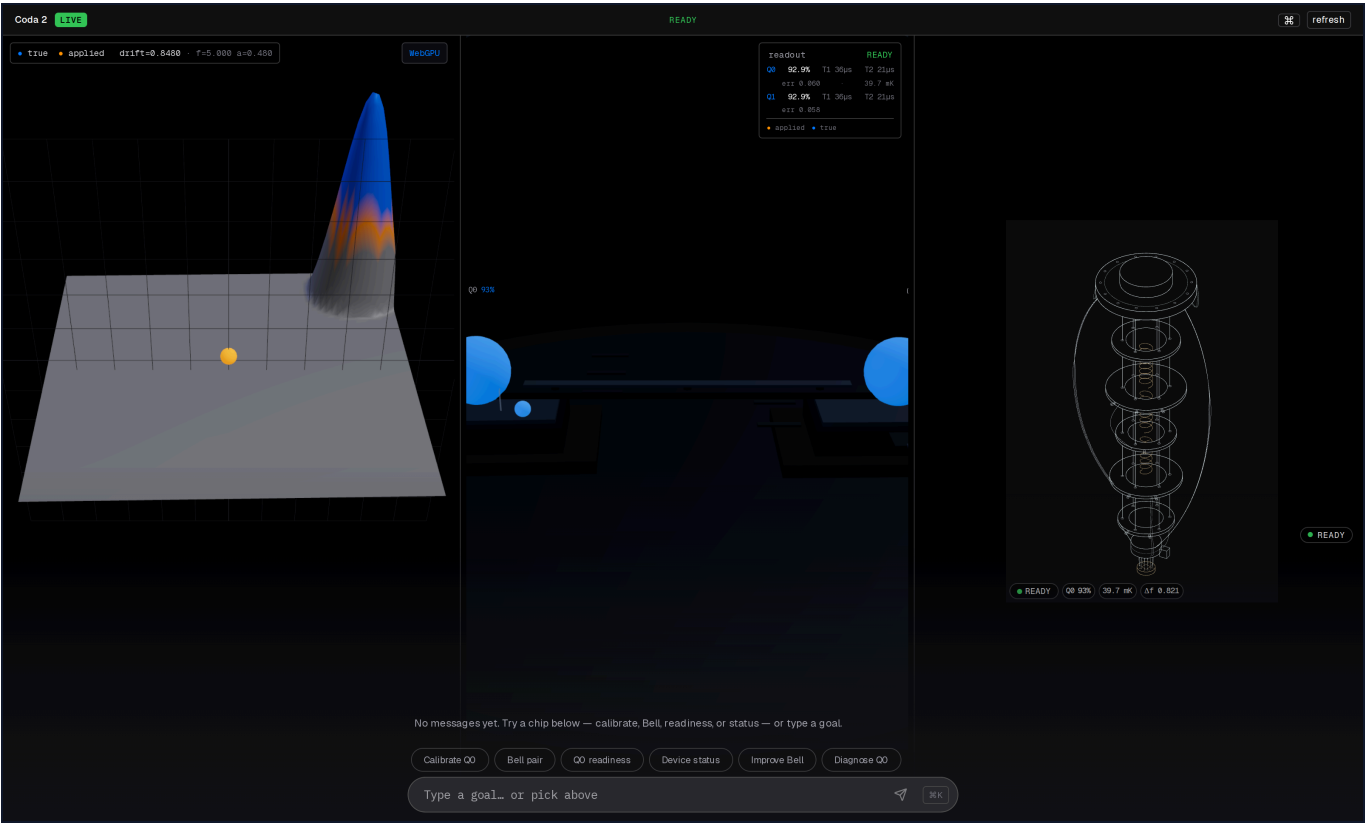


FIG. 0 The live instrument on main. Three panes at rest. Left: parameter-drift landscape (orange applied, blue true). Center: 3D device — qubit spheres Q0 and Q1, readout card. Right: isometric cryostat plate with a status strip. Dock: six workflow chips and the composer.

LEFT Param-drift landscape	CENTER Device 3D	RIGHT Cryostat plate
--------------------------------------	----------------------------	--------------------------------

Six workflow chips. Calibrate Q0 · Bell pair · Q0 readiness · Device status · Improve Bell · Diagnose Q0. Click one, or type a goal. The Calibrate Q0 path is unchanged: bring qubit 0 to ready, then the toolbar shows READY.

The product is **multimodal** on purpose: **visual** (the three panes), **text** (natural-language goals and the agent transcript), and **math** (fidelity, Δf , $T1 / T2$, mK) at the same time. A chat-only UI can take the same sentence and still hide the machine.

Still. Live captures under `docs/manual/assets/ui/manual-3pane-*.png`. Rebuild the PDF with `./docs/manual/build.sh` → `docs/manual/coda-2-manual.pdf`.

Who this is for

Coda 2 is a three-pane instrument for a quantum processing unit: parameter-drift landscape on the left, the 3D device in the center, the cryostat plate on the right, and a chat dock across the bottom. It is a control plane, and it is a stage. Three roles use one instrument. They do not do the same job.

Beyond a chat window

A normal chat window is a transcript. You type, a model replies, and any physics lives only in sentences that scroll away. Coda 2 is instrument-first. The first thing you learn is the stage in FIG. 0: drift on the left, the device in the center, the cryostat on the right. Language is one channel, not the only surface. Physics stays in view while you talk. The chips in the dock are one-click goals.

- **Visual** — the three panes: left drift landscape, center 3D device with the readout card, right cryostat plate.
- **Text** — natural-language goals and the agent transcript in the dock.
- **Math** — fidelity, Δf , $T1 / T2$, ToolTraces, and the metrics.

A chat-only UI can accept the same sentence and still hide the machine. Here the stage does not yield to the transcript. You drag the splitters, read the numbers, and send a goal without leaving the instrument. Use FIG. 0 as the picture of the product. The stills later in this book are the same three-pane chrome, not the older one-pane or two-pane layouts.

Three roles

Read the table. Then read the note for your role. The plate shows the same three stations. Each station uses one instrument.

ROLE	WHAT YOU DO	WHAT YOU USE
Quantum control engineer	Builds and maintains the path from natural language to the agent to the adapter.	The QPUAdapter seam. ToolTraces. The calibration loop. The three metrics.
QPU operator	Calibrates, checks readiness, runs circuits, and cancels jobs.	LIVE and READY. Chips and the composer. When to recalibrate. How to read Δf , fidelity, and the three panes.
Technical lead	Evaluates the product for their stack. Optional. The question is whether their driver can sit behind the adapter.	The operator path: calibrate, then READY, then a Bell pair. The three metrics. The adapter seam.

Table A.1. Who uses Coda 2. Pick your row. The later parts go deeper for that role.

Quantum control engineer

You build and maintain the control software. A goal enters as `POST /goals`. The agent maps the goal to a tool. Every tool call writes a ToolTrace.

You keep the QPUAdapter seam. The waist is six methods: `submit_job`, `poll_job`, `cancel_job`, `get_device_state`, `get_calibration`, `apply_calibration_update`. The calibration loop measures fidelity, decides, applies an update, and measures again. The three metrics are time-to-calibrated, calibration success rate, and interface latency.

The default path is offline. It does not need a language-model key. If an optional planner is on, it falls back to the same rules. You extend the tools, or you replace the adapter. You keep the seam.

QPU operator

You run the instrument. You calibrate, check readiness, run circuits, and cancel jobs. You land on the idle home. The toolbar shows LIVE when the UI is connected. It shows READY centered when the device is fit for a circuit. You send a goal with a chip, or you type in the composer. Recalibrate when the state chip falls back to DRIFT, or when Δf grows. High Δf means the controls are off-target.

Read fidelity as Q0 percent in the toolbar and on the readout card in the center pane. Read the left pane for drift (orange applied versus blue true). Read the right pane for the cryostat plate and the status strip under it. The state chip sits at the far right.

Technical lead

You evaluate Coda 2 for your stack. You are a technical lead or a buyer of the integration. You use the same product the operator uses. You do not run a separate trial.

You follow one path: calibrate qubit 0, see READY, then run a Bell pair. You read the three metrics. You read the adapter seam. A simulator stands in for the device until a hardware adapter is connected. Your driver replaces that simulator behind the same six methods.

NOT FOR

This manual is not a cryogenics handbook. The millikelvin figure is computed from fidelity on the simulator. It is not a fridge sensor. The right-hand plate is an isometric drawing, not a fridge readout. Coda 2 is not an open chat for science or finance. The planner accepts four goal families. If a sentence matches no keyword, the planner checks device state, then it runs a soft calibrate. That is the catalog.

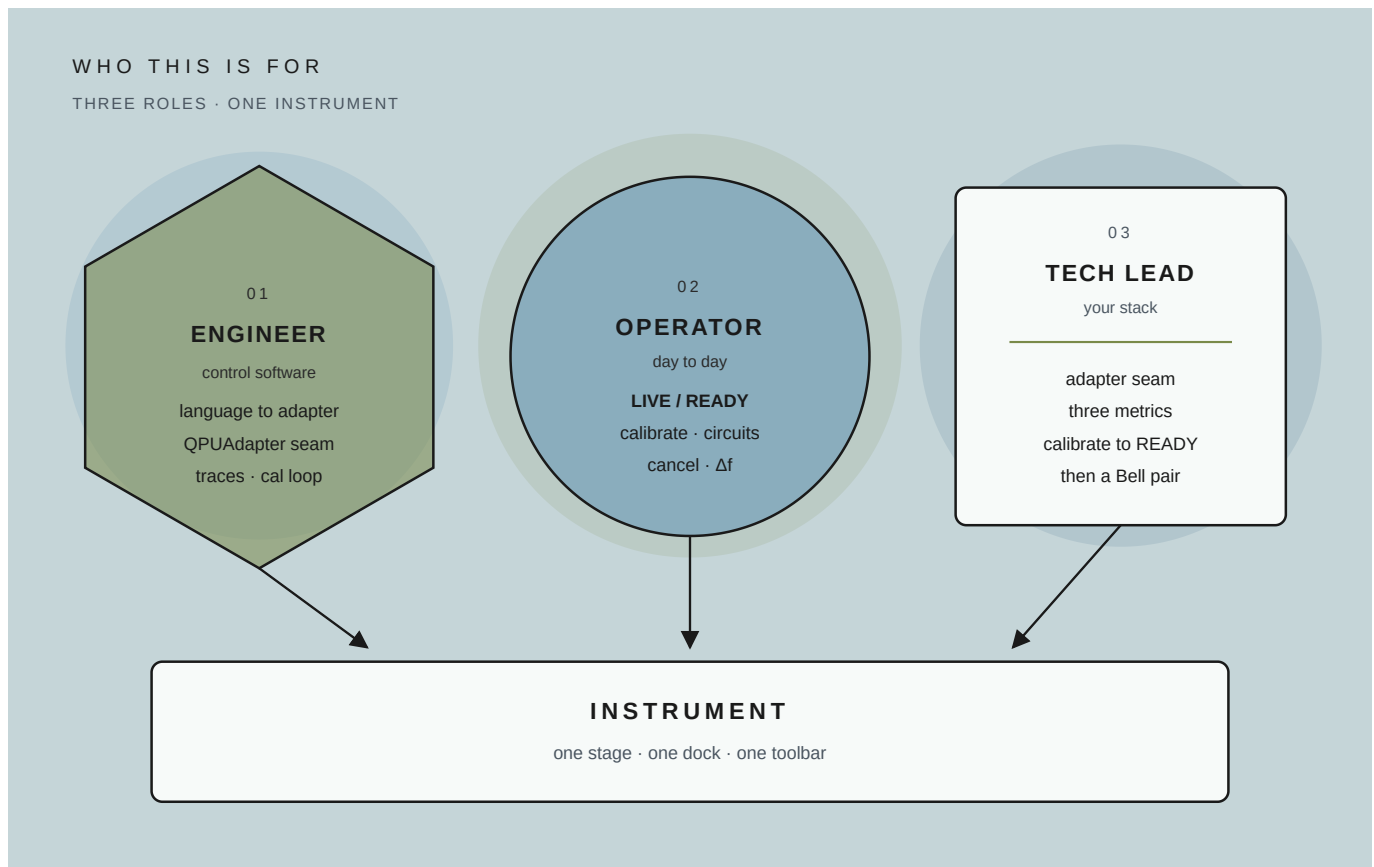


FIG. A.1 Who this is for. Three stations, one instrument. The engineer builds and maintains the path from language to the adapter. The operator calibrates, checks readiness, runs circuits, and cancels jobs. The technical lead evaluates the product for their stack.

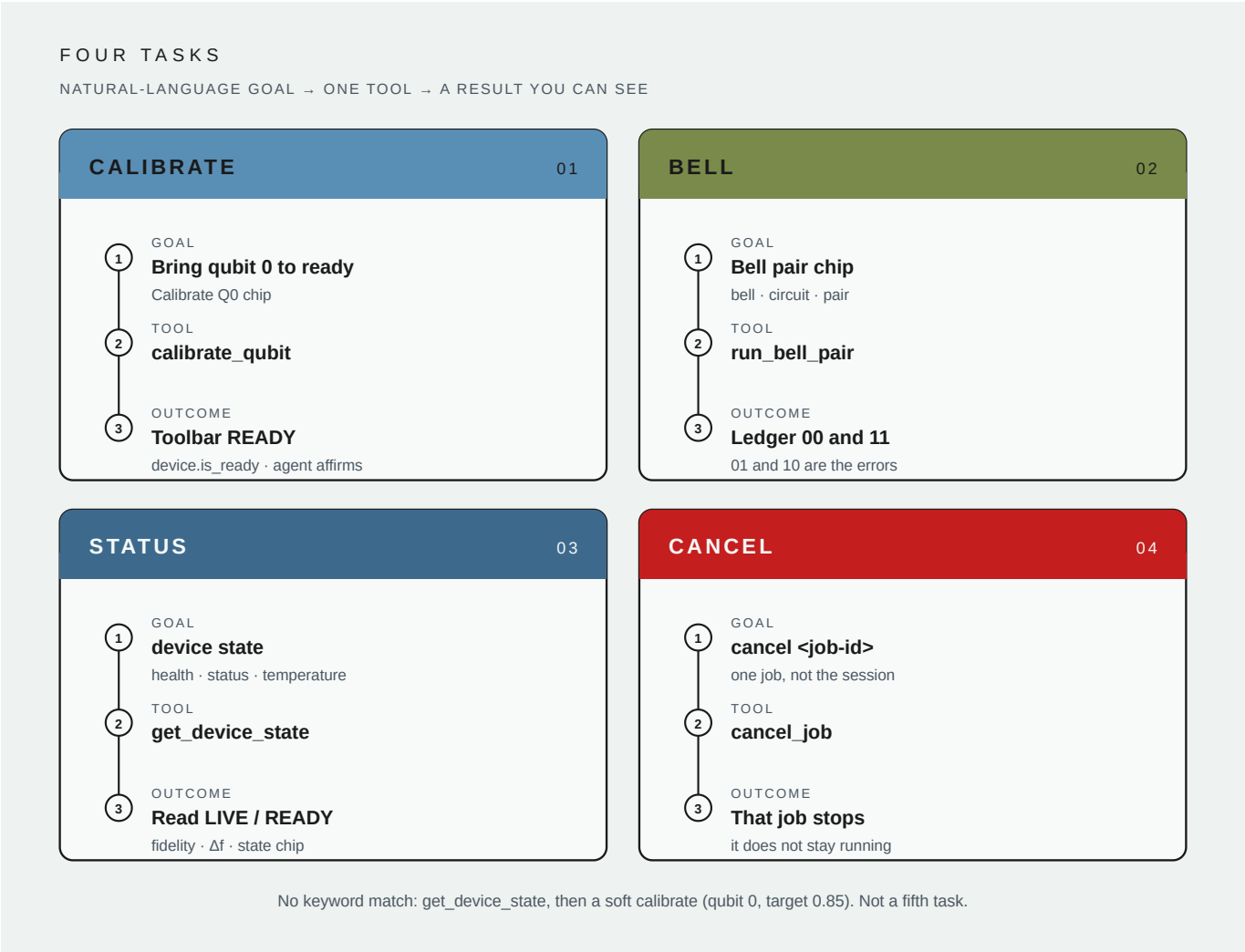
How to read this plate. Shape is the role. The hexagon is the engineer (the seam). The circle is the operator (a person at the screen). The rectangle is the technical lead (integration with their stack). Arrows run down into one instrument: stage, dock, and toolbar. You do not need a separate legend.

Four tasks

The product does four tasks. They match the deterministic planner (Table 7.2). Each task is a goal in ordinary language, one tool, and a result you can see. A chip is a short way to send the same goal.

TASK	EXAMPLE GOAL / CHIP	TOOL	SUCCESS
Calibrate / bring ready	“Bring qubit 0 to ready” / Calibrate Q0 chip	<code>calibrate_qubit</code>	Toolbar READY. <code>device.is_ready</code> is true. The agent affirms.
Run a circuit (Bell)	Bell pair chip	<code>run_bell_pair</code>	The agent reports an estimated Bell fidelity. Turn details hold the counts; most shots should be 00 and 11.
Check device state	“device state” / health, status, temperature	<code>get_device_state</code>	You read LIVE, READY, and the telemetry. You do not start a circuit.
Cancel a job	<code>cancel <job-id></code>	<code>cancel_job</code>	That job stops. It does not stay running.

Table A.2. Four tasks. Same mapping as Table 7.2 in the orchestrator. Keywords: bring / calibrat / ready / tune; bell / circuit / entangl / pair; state / health / status / temperature; cancel plus a job id.



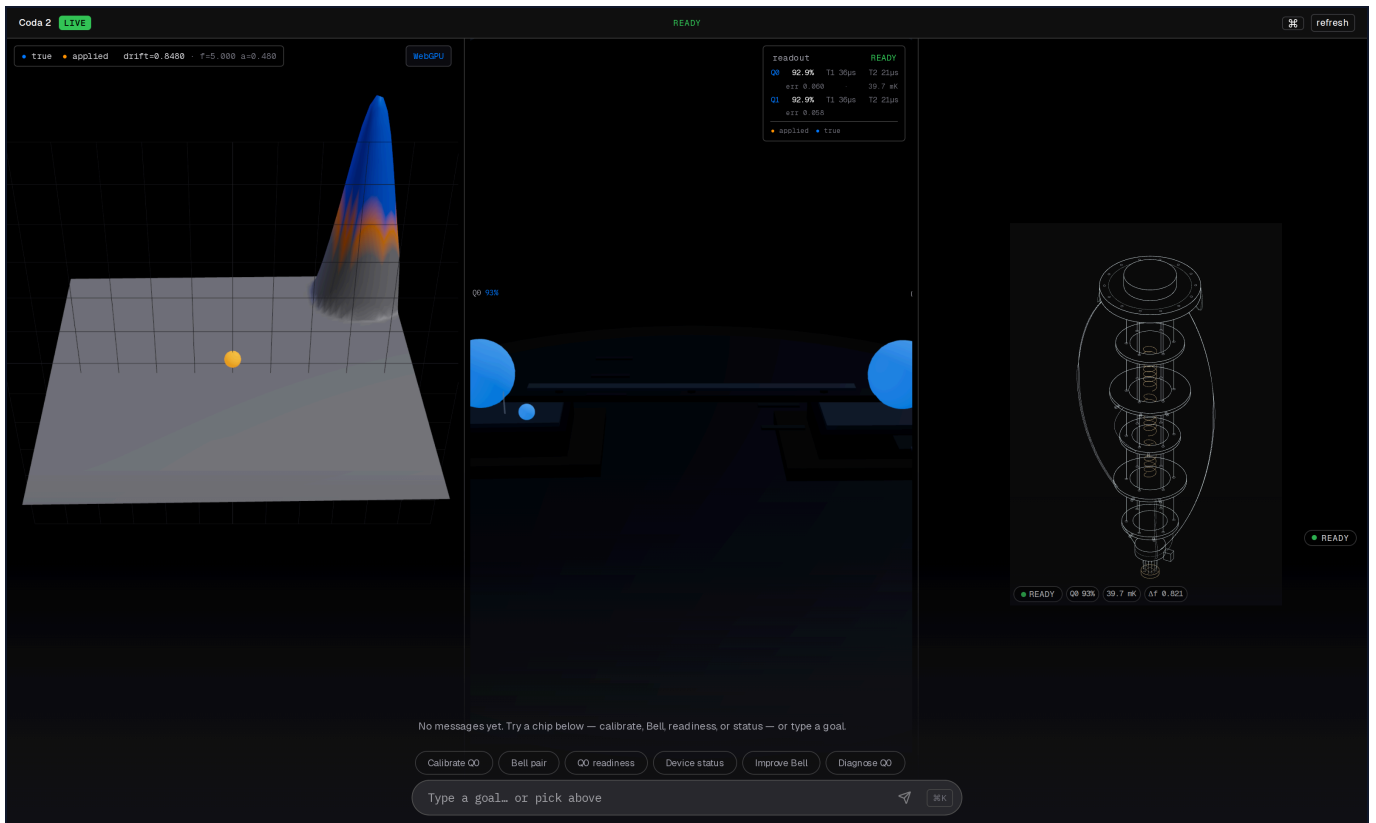


FIG. A.3 Idle home. The operator lands here. Left pane: drift grid (orange applied, blue true). Center pane: 3D device with a readout card at top right (Q0 / Q1 fidelity, T1, T2, err, mK). Right pane: cryostat plate with a status strip under it (READY · Q0 % · mK · Δf) and a state chip at the far right. Toolbar: Coda 2, LIVE, READY centered. Dock: empty transcript hint, six workflow chips, composer.

Task 1 — Calibrate, and bring the qubit to ready

Type `Bring qubit 0 to ready` in the composer, or click the Calibrate Q0 chip. Send. The planner calls `calibrate_qubit`. Success looks like this: the toolbar shows READY, `device.is_ready` is true, and the agent says the qubit is fit for a circuit.

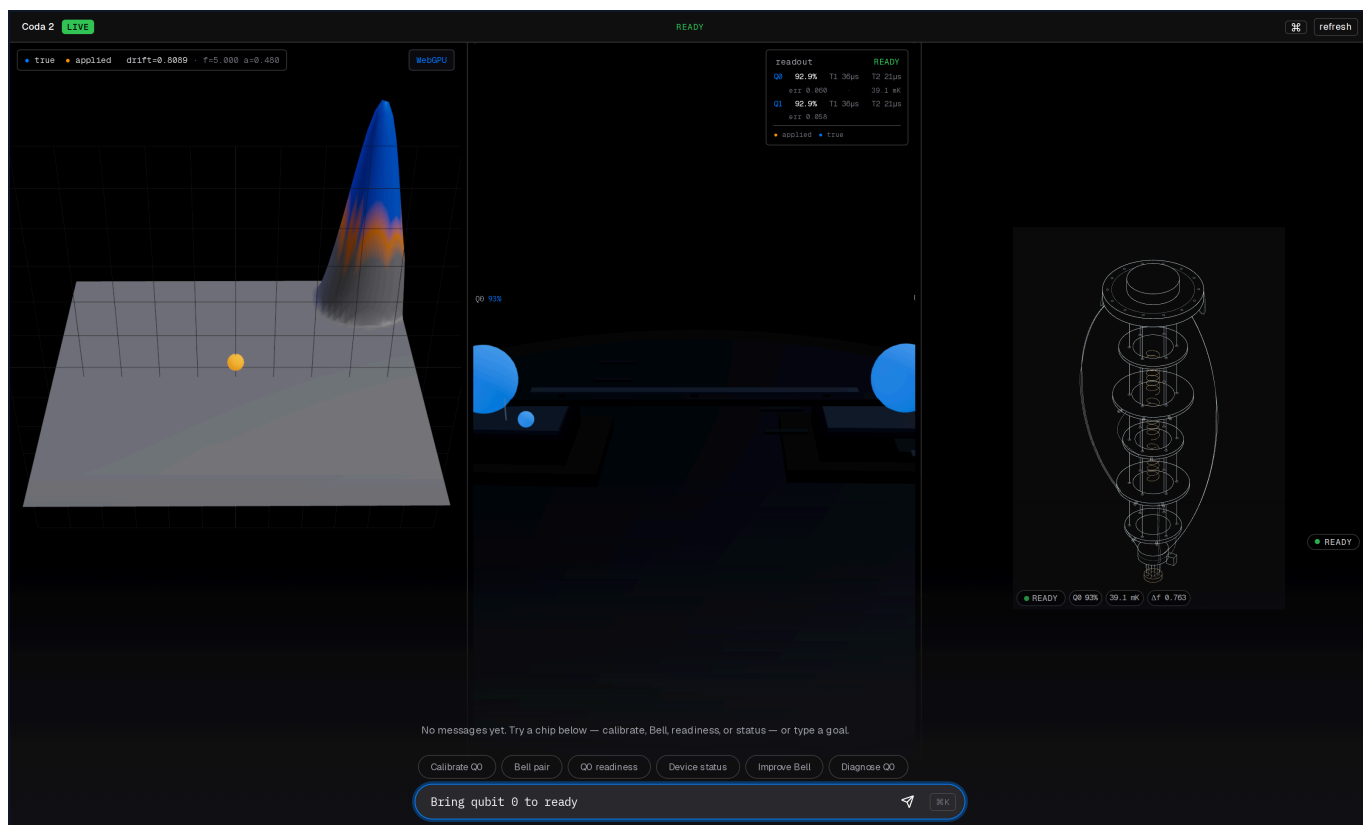


FIG. A.4 Goal typed. The words sit in the composer above the chips. Nothing on the stage moves until you send, or until you click Calibrate Q0. All three panes stay in view.

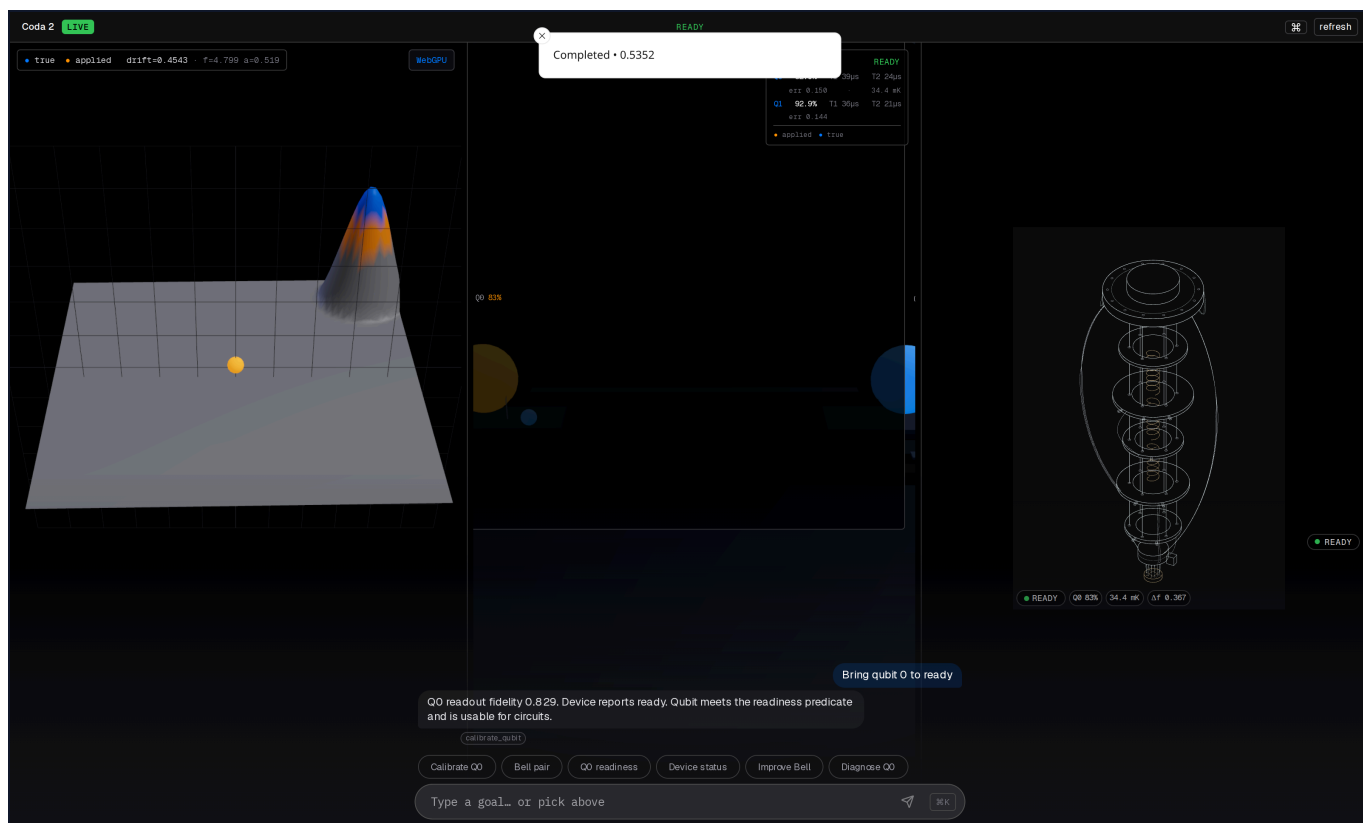


FIG. A.5 Calibrate done. Toolbar shows LIVE and READY. The agent affirms: Q0 climbed to about 0.69 fidelity and is READY for circuits. The cryostat status strip and the state chip at the far right read READY. This is the success still for task 1.

What success means here. READY on the toolbar matches `device.is_ready`. Fidelity is up. Δf is smaller than it was at idle. You can try a circuit next. The full step-by-step, with every number named, is the Instrument walkthrough (STE.3).

Task 2 — Run a circuit (Bell)

Prefer READY first. Click the Bell pair chip, or type a goal that says Bell, circuit, or pair. The planner calls `run_bell_pair`. Success looks like this: the agent reply reports an estimated Bell fidelity — about 0.42 on this seed. The transcript now carries two turns, and a small tool pill marks the run.

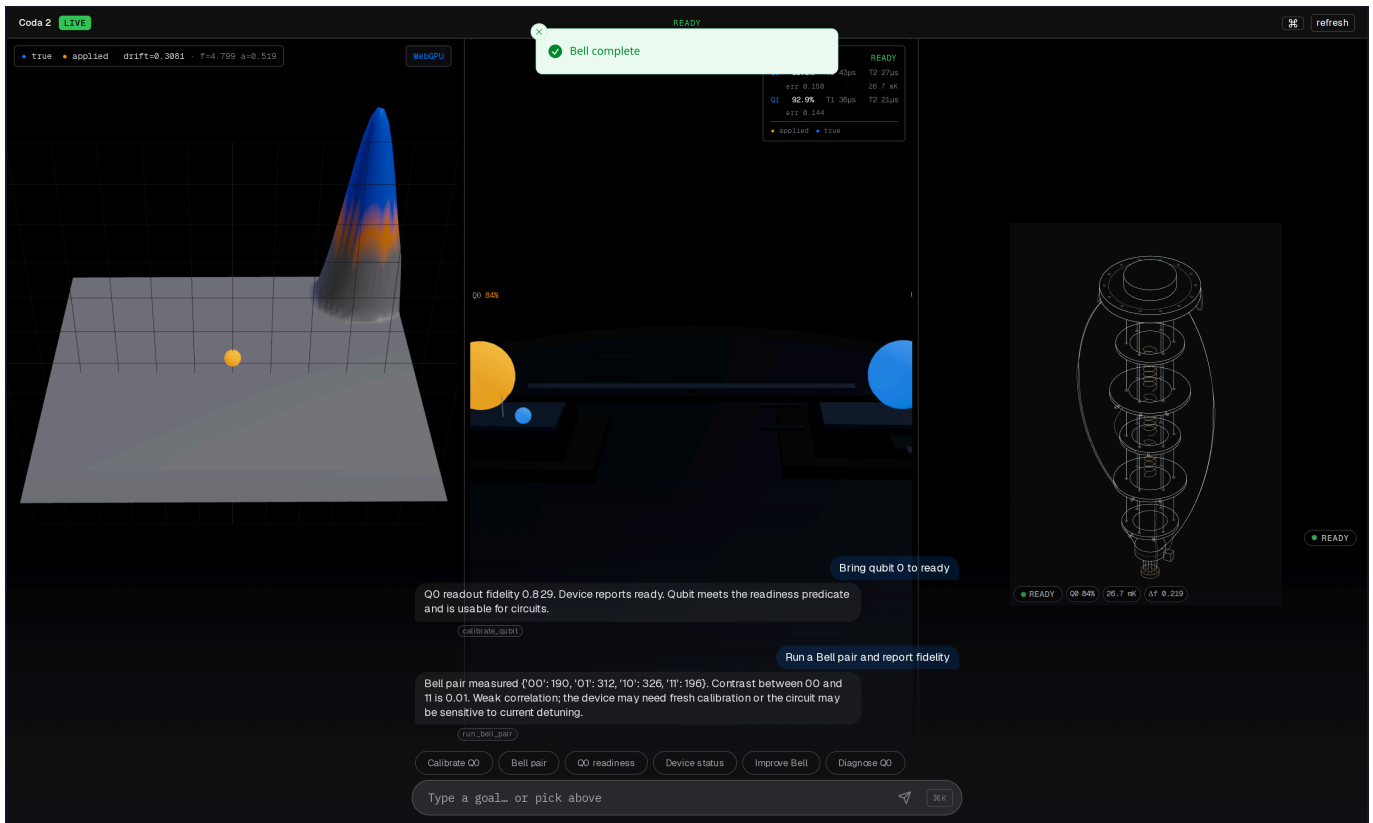


FIG. A.6 Circuit done. The transcript has a second turn: you ask for a Bell pair and report fidelity; the agent answers with the estimate and a `run_bell_pair` tool pill. All three panes remain visible: drift left, device center, cryostat right.

Task 3 — Check device state

Type `device state`, or a health, status, or temperature word. The planner calls `get_device_state`. Success looks like this: you can read LIVE, READY, fidelity, and Δf . You do not start a calibration, and you do not start a circuit. Use the idle home (FIG. A.3) before a goal. Use the map (FIG. A.7) after a session. There is no separate status still.

Task 4 — Cancel a job

Type `cancel` and the job id. The planner calls `cancel_job`. Success looks like this: that job stops. The session stays open. This book has no cancel still. FIG. A.2, card 04, is the picture for this task.

Map of the instrument

Use this still as a map. The labels are in the list. They are not drawn on the picture.

1. **Toolbar, top.** Brand and LIVE pill at left, READY centered, and refresh at right. LIVE means connected. READY means fit for a circuit. Q0 percent and fid are fidelity. Δf is detuning. High Δf means the controls are off-target.
2. **Left pane.** Drift grid. Orange is applied (what the control plane wrote). Blue is true (the simulator's hidden sweet spot). The grid is coplanar with the fidelity surface.
3. **Center pane.** 3D device: qubit spheres Q0 and Q1, chiplets, resonators, and a readout card at top right with Q0 / Q1 fidelity, T1, T2, err, and mK.

4. **Right pane.** Isometric dilution-refrigerator plate, labelled FIG 1 · DILUTION FRIDGE, with a status strip under it (READY · Q0 % · mK · Δf). Two splitters separate the three panes; drag one to resize. Each pane keeps a minimum width, roughly 22–28% of the stage.
5. **Dock, bottom.** Transcript, then workflow chips, then the composer. The composer is the lowest control (high-contrast iOS-dark field: solid #2c2c2e, bright typed text, readable placeholder). The stage continues under the bottom-third progressive frost. A state chip sits at the far right: DRIFT, CALIBRATING, READY, and the other lifecycle states.

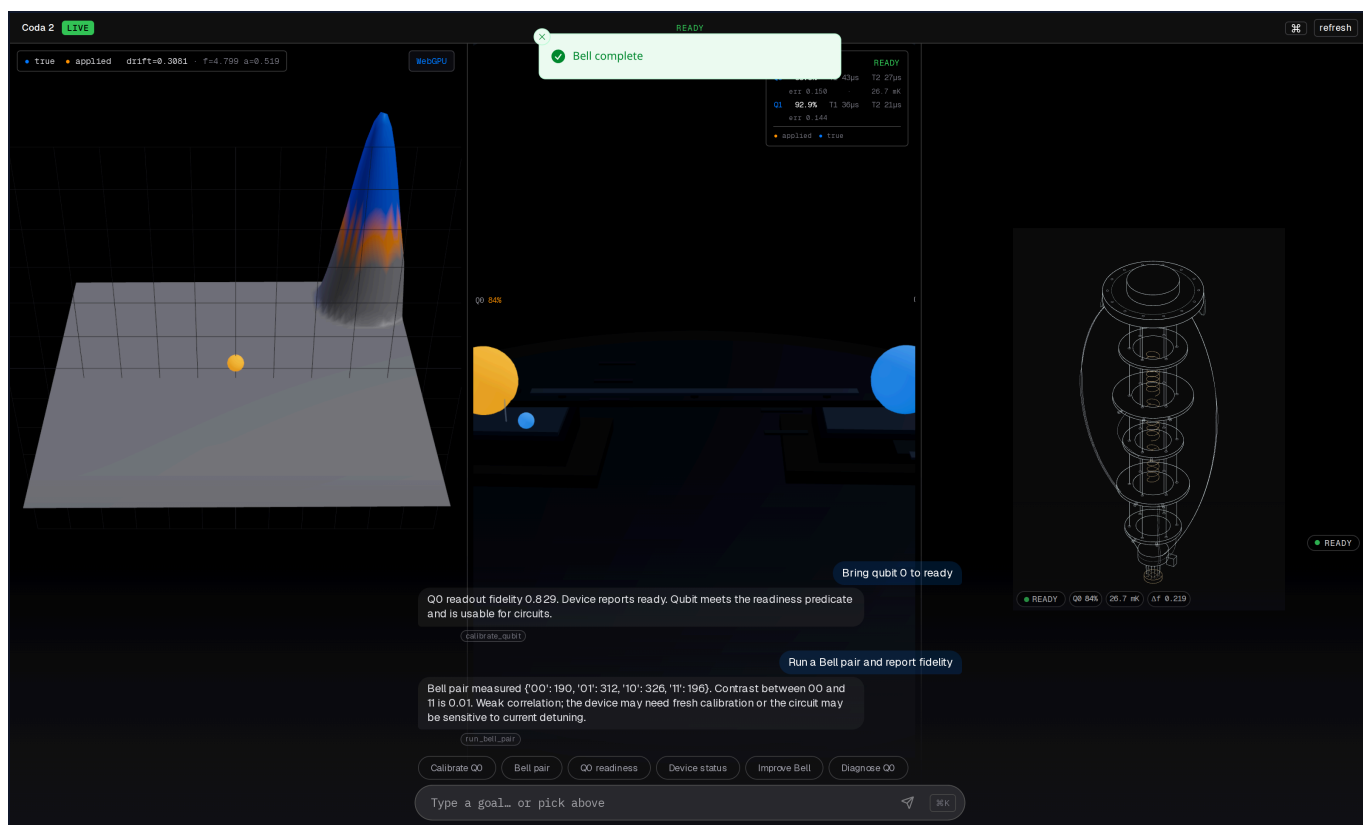


FIG. A.7 Map of the instrument. Top: toolbar (Coda 2, LIVE, READY, ⌘, refresh). Left: drift. Center: device 3D with the readout card. Right: cryostat plate and status strip. Bottom: dock (transcript, six workflow chips, composer). No callout arrows — use the five labels above. Older one-pane and two-pane stills are historical; they are not this map.

RULE OF THUMB

Four tasks only. Calibrate until READY. Then run Bell. Read state when you are not sure. Cancel when one job must stop. Any other sentence becomes a status check, then a soft calibrate.

Sources. Planner keywords: orchestrator/planner.py (Table 7.2). Adapter methods: adapter/base.py. Goals: POST /goals in api/server.py. Stills: assets/ui/manual-3pane-home.png, manual-3pane-typed.png, manual-3pane-ready.png, manual-3pane-bell.png. Plates: diagrams/plate-who-for.svg, diagrams/plate-four-tasks.svg. Tip: main (three-pane chrome).

Overview

Coda 2 takes a goal in ordinary language and runs it on a quantum processing unit through a typed tool catalog and a QPU adapter. The interaction is meant to go beyond a normal ChatGPT element.

You see and manipulate properties of a quantum computer in the format that fits the moment. Visual: a three-pane stage — drift landscape on the left, the 3D device in the center, the cryostat plate on the right. Text: natural-language goals and the agent transcript. Math: fidelity, Δf , T_1 / T_2 , ToolTraces, and metrics. The product is instrument-first. Physics stays in view. Language is one channel, not the only surface. A chat-only UI would bury those properties in prose and scroll them away.

An operator states a goal. An agent maps that goal onto a small catalog of typed tools. The tools call the adapter. The screen is an instrument: a live three-pane stage (drift · device · cryostat), and a dock that holds the conversation.

A simulator stands in for the device until a hardware adapter is connected.

Day to day the product does four jobs: bring a qubit to ready, run a Bell pair, check device state, and cancel a job. The control plane records time-to-calibrated, calibration success rate, and interface latency. The adapter seam is six methods. The default path is offline and does not need a language-model key. When the optional planner is on, it falls back to the same rules.

This edition of the product documents package `conductor-qpu 0.1.0`, `main` (three-pane chrome), through PR #25–#27. The stage is three panes behind two splitters. The left pane is the drift grid. The center pane is the device. The right pane is the cryostat plate.

Edition. Coda 2 · package `conductor-qpu 0.1.0` · `main` (three-pane chrome, PR #25–#27) · 2026-10-05 (PT)

Instrument walkthrough

Use this section when you operate the app. The screenshots below are a **complete user flow** captured on **main (three-pane chrome)** with the API on :8000, the UI on :3000, and the live narrator writing each agent reply (NVIDIA NIM when keyed, otherwise a template). Every shot shows all three panes: drift landscape on the left, the 3D device in the center, the cryostat plate on the right. The dock is a real chat transcript: your goal as a *you* bubble on the right, an *agent* reply on the left with a small tool pill under it, then the workflow chips, and the composer always the *lowest* control. Each STE step states what you typed or clicked and what the stage did next. Each figure also translates the on-screen numbers into plain English for that frame.

Dock layout (conversation shape)

The stage is three panes with two splitters between them. The dock sits at the bottom, over a bottom-third frost. Read the bottom of every screenshot top-to-bottom:

1. **Chat transcript** — scrollable turns. Each turn has a *you* line (the goal) and an *agent* line (plain-English narration from `agent_message`). Tiny tool pills may sit under the reply; they are secondary.
2. **Chips** — Calibrate Q0 · Bell pair · Q0 readiness · Device status · Improve Bell · Diagnose Q0 (one-click goals).
3. **Composer** — type a goal and send (or ⌘K). Lowest control in the dock.

The three-pane stage extends *under* this dock. Panes start roughly equal. Left: high-contrast drift grid, coplanar with the fidelity surface. Center: the 3D device — qubit spheres Q0 and Q1, chiplets, resonators — plus a readout card at top right (Q0 / Q1 fidelity, T1, T2, err, mK). Right: an isometric dilution-refrigerator plate labelled FIG 1 · DILUTION FRIDGE, with a status strip under it (READY · Q0 % · mK · Δf) and a `StageStateChip` at the far right (DRIFT · CALIBRATING · READY...). Two iPadOS Split View–style splitters separate the three panes. Hover shows a capsule drag handle. Drag to resize. Each pane keeps a minimum width, roughly 22–28% of the stage. There are no traffic lights, no glass inset, and no tabs. A Safari-safe bottom-third progressive frost (stacked solid translucent bands on a dedicated `.stage-blur` sibling — no `backdrop-filter` on the dock) softens only the lower third so the viz stays readable under the transcript without frosting the whole stage.

What every number means

Read this table once. Then the STE frames make sense.

LABEL	WHAT YOU SEE	WHAT IT MEANS FOR YOU
LIVE	Green pill in the toolbar	The UI is connected to the control plane. Fresh device data can arrive. If LIVE is missing, check that the API is running.
READY / CAL NEEDED	Status next to LIVE	READY means every qubit's readout fidelity is at least 0.82. Fit for a circuit attempt. CAL NEEDED means calibrate before you trust a circuit.
Q0 xx%	Toolbar percentage	Readout fidelity for qubit 0 as a percent. Higher is better — how often the qubit answers correctly when measured.
fid 0.xxxx	Blue number in the toolbar	Same fidelity as a fraction (0–1). Compare to the 0.88 calibration threshold.
mK	Temperature next to Q0	Reported temperature in millikelvin. Lower is colder. Theatrical here (tied to fidelity), not a fridge sensor.
Δf	Orange number	Frequency detuning. High Δf = controls are off-target (mistuned; fidelity will fall). Smaller is better.
applied (orange)	Marker / legend	Last parameters the control plane wrote — what the AWG would play.
true (cyan / blue)	Marker / legend	Hidden sweet spot the simulator owns. Calibration moves applied toward true. On real hardware you never see “true” directly.
blue sphere / ring	Drift stage cue	Blue = qubit is calibrated enough to trust for the next job (near the sweet spot).
orange sphere	Drift stage cue	Applied / off-target marker. When orange sits far from blue, you still need calibration.
three-pane stage	Full stage (no tabs)	Three panes at rest: left = high-contrast drift grid (orange applied, blue true); center = 3D device (qubit spheres Q0 / Q1, chiplets, resonators) with a readout card at top right; right = isometric cryostat plate with a status strip under it. Two iPadOS Split View–style splitters with hover capsule handles. Drag to resize (~22–28% minimum per pane). No glass inset. No tab flip. Bottom-third progressive frost under the chat dock; top ~2/3 stay sharp.
splitter	Two vertical separators	Drag a capsule handle to give one pane more width. Hover reveals the handle. You cannot collapse a pane below its minimum width (roughly 22–28%).
state chip	Bottom-right stage badge	Device lifecycle: OFFLINE → DRIFT → CALIBRATING → RUNNING → READY / FAILED. Grounded on live connection + goal status + <code>is_ready</code> .
0.88 line	Orange dashed line on climb charts	Calibration success line. Stay at or above it and the loop succeeded.
you / agent	Chat bubbles in the dock	<i>you</i> is the goal you sent. <i>agent</i> is the operator-facing sentence from the narrator (NVIDIA NIM when configured; otherwise a deterministic template). Always non-empty.

Table W.1. Operator glossary. Use it to read every STE frame below.

RULE OF THUMB

Fidelity is how often the qubit answers correctly — higher is better; ≥ 0.88 means calibrated for this product. Δf high = controls are off-target. Blue sphere = calibrated enough to trust. READY = fit to try a circuit. LIVE = UI talking to API. Chat bubbles (you right / agent left) above chips; high-contrast composer lowest; empty-state and chips lifted above the tint.

Complete user flow — STE sequence

The figures below are one continuous **successful** session: idle $\rightarrow$ type Calibrate Q0 $\rightarrow$ agent affirms READY on the three-pane stage $\rightarrow$ Bell $\rightarrow$ follow-up. All three panes (drift · device · cryostat) are in every wide shot; the dock close-ups isolate the chips and the transcript. Progressive blur is visible in every shot (grid / viz bleeding through the frosted dock). Captured with the live narrator; toolbar shows **LIVE** and **READY** after calibrate.

SUCCESS CRITERION FOR THIS WALKTHROUGH

Toolbar

READY

is on (`device.is_ready = true`). The agent bubble affirms that Q0 is READY for circuits. Transcript bubbles sit in the frosted dock directly above chips and composer — not mid-stage.

STE.1 — Idle home (three-pane stage), empty transcript

You do. Open `http://localhost:3000` with the API on `:8000`. Land on the three-pane stage. Do not send a goal yet.

What happens. Left pane = high-contrast drift grid, coplanar with the fidelity surface. Center pane = the 3D device (qubit spheres Q0 and Q1, chiplets, resonators) with a readout card at top right (Q0 / Q1 fidelity, T1, T2, err, mK). Right pane = the cryostat plate, labelled FIG 1 · DILUTION FRIDGE, with a status strip under it and a `StageStateChip` at the far right; at idle it typically reads DRIFT (or OFFLINE if the API is down). Each splitter shows a capsule handle on hover. Dock shows an empty transcript hint (“No messages yet. Try a chip below...”), six workflow chips, and an empty composer (lowest). Toolbar: brand and LIVE at left, READY centered,  and refresh at right. Viz extends under the progressive-blur dock. There is no glass inset and no tab control.

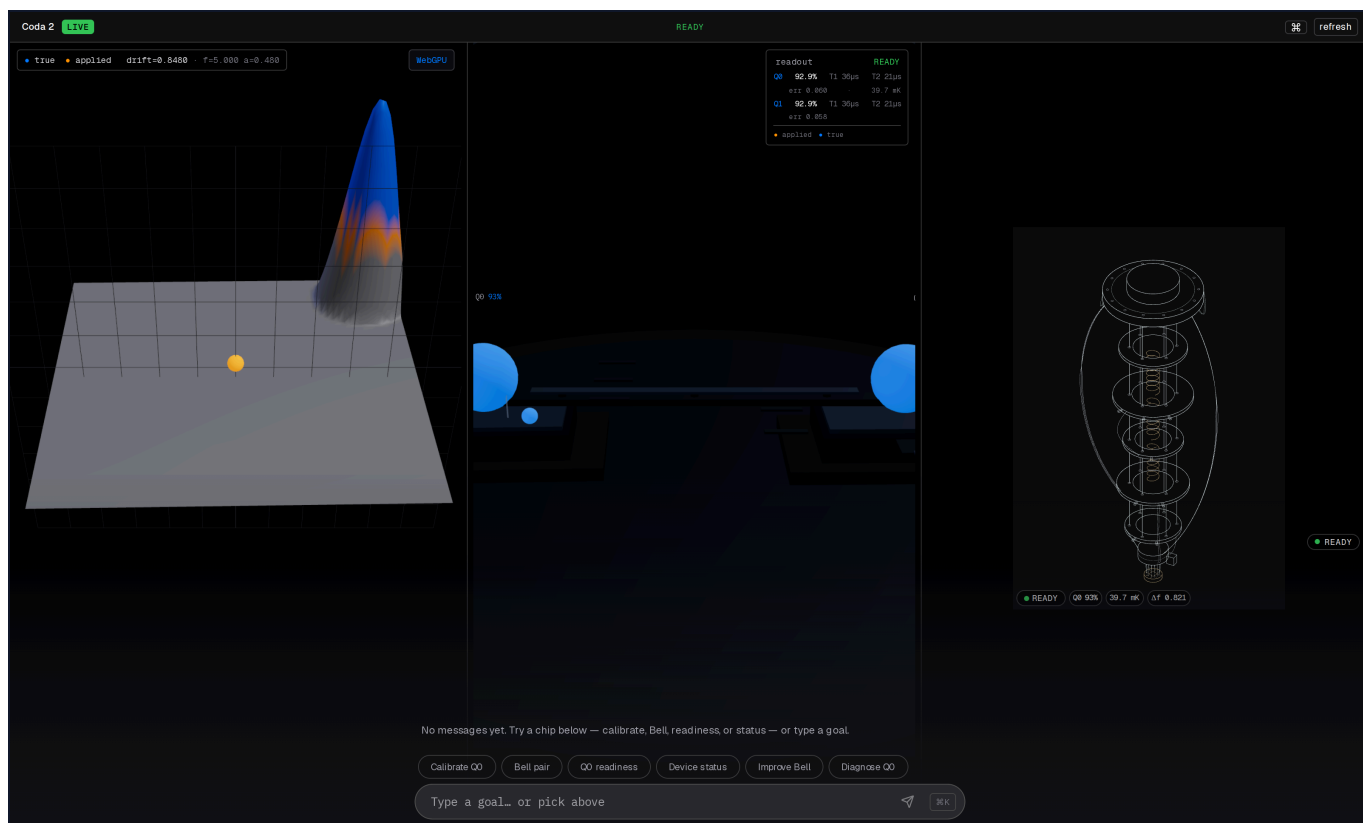


FIG. W.1 • STE.1 Idle three-pane stage. Drift grid coplanar on the left. 3D device with the readout card in the center. Cryostat plate with a status strip under it on the right. Two splitters with hover capsule handles. LIVE in the toolbar; READY centered. No traffic lights. No glass inset. No tabs. Empty chat transcript hint above six chips. Composer empty at the bottom of the frosted dock. `StageStateChip` at the far right. Grid lines bleed through the progressive blur.

Plain English for this frame. LIVE means the control plane is connected. The left grid is drawn in high contrast and lies in the same tilted plane as the fidelity surface. The tall “mountain” is that surface over frequency and amplitude error — the peak is where the qubit works best. Orange vs blue markers are applied vs true: if they sit apart, controls are still off-target (Δf will look high). The center pane is the device view: two qubit spheres plus chiplets and resonators, with a readout card at top right that repeats fidelity with T1 / T2, err, and mK labels. It is UI, not a fridge. The right pane is a static isometric plate with a live status strip; drag a splitter if you want one pane wider. No chat turn yet because you have not spoken. The frosted dock is translucent on purpose so you keep seeing the physics while you type.

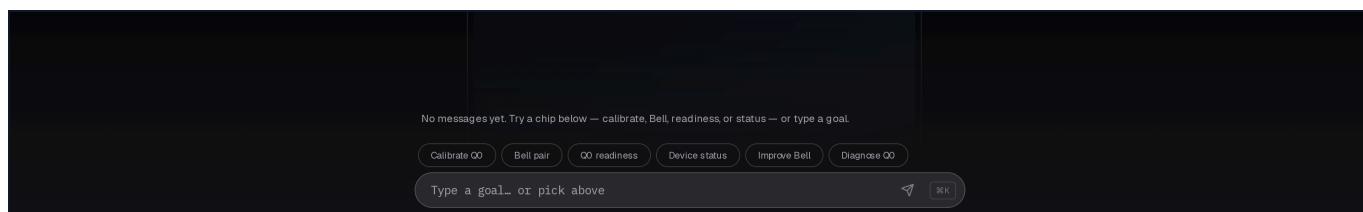


FIG. W.1B • STE.1 Dock close-up. Empty-state hint, the six workflow chips (Calibrate Q0 · Bell pair · Q0 readiness · Device status · Improve Bell · Diagnose Q0), and the composer. ⌘K opens the command palette; Enter sends the typed goal.

STE.2 — Type a goal in the composer

You do. Click the composer (lowest bar). Type `Bring qubit 0 to ready`. Do not send yet — or use the **Calibrate Q0** chip instead.

What happens. The composer holds your intent. Chips remain above it. Still no transcript turn until send.

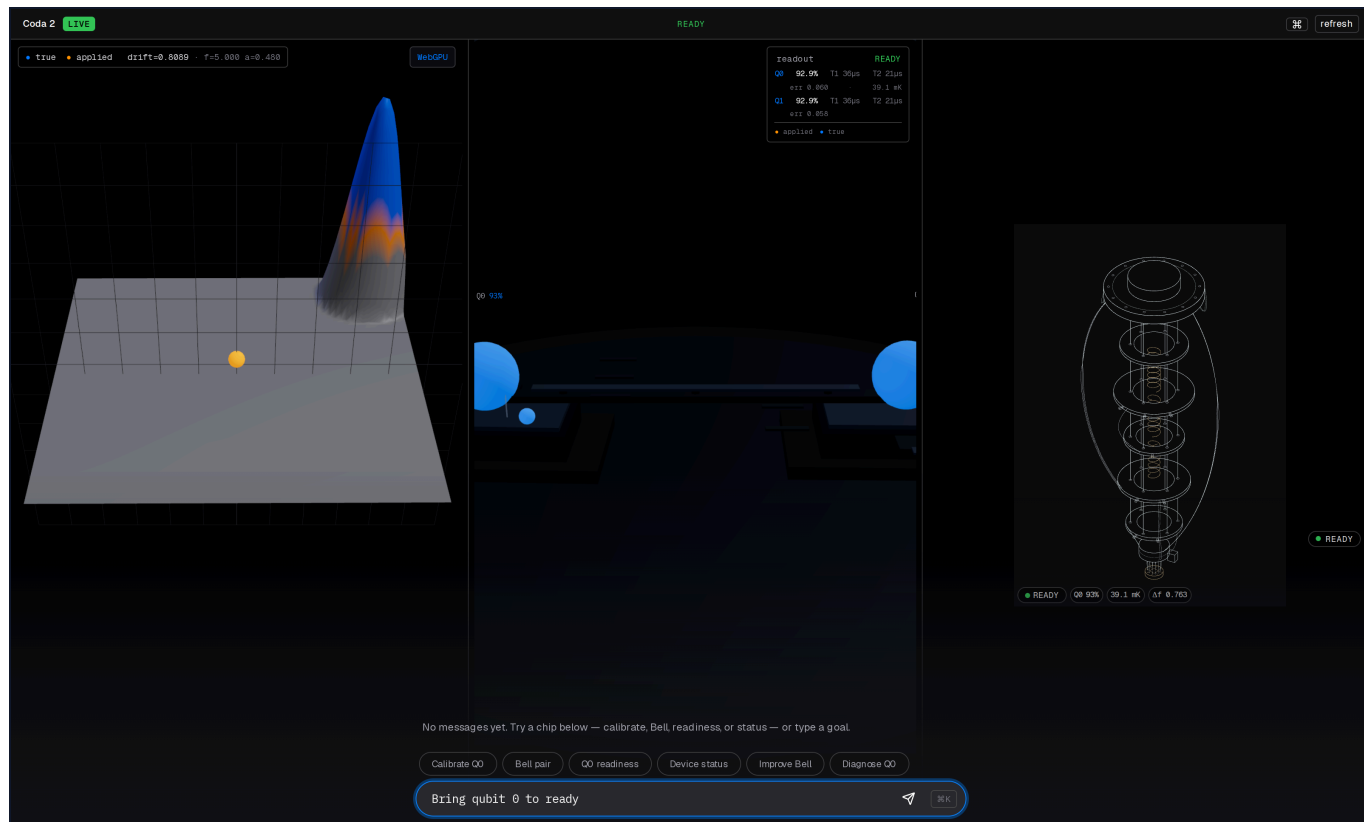


FIG. W.2 · STE.2 Composer typed. Goal text sits in the lowest dock control; the chips row stays above it. All three panes stay in view: drift left, device center, cryostat right. Progressive blur still shows stage content under the frosted bar.

Plain English for this frame. You are about to ask the agent to make qubit 0 trustworthy. Nothing has changed on the stage yet — typing alone does not move parameters. Send (or the Calibrate Q0 chip) is what starts the loop.

STE.3 — Send Calibrate Q0 → READY (success)

You do. Click chip **Calibrate Q0** (or press send on the typed goal).

What happens. A *you* bubble appears with the goal. While tools run, the agent line shows “running...”. The fidelity-climb HUD rises toward the 0.88 line. When the loop finishes successfully, the toolbar flips to **READY** and the agent line affirms that Q0 is READY for circuits. Toolbar Δf shrinks; fid / Q0 % rise. Toast may say “Reached threshold.”

Agent reply. The agent bubble gives a READY affirmation: Q0 is fit for circuits. Exact wording comes from the narrator (NVIDIA NIM when configured; otherwise a template) and can change from capture to capture. The affirmation must agree with `device.is_ready`.

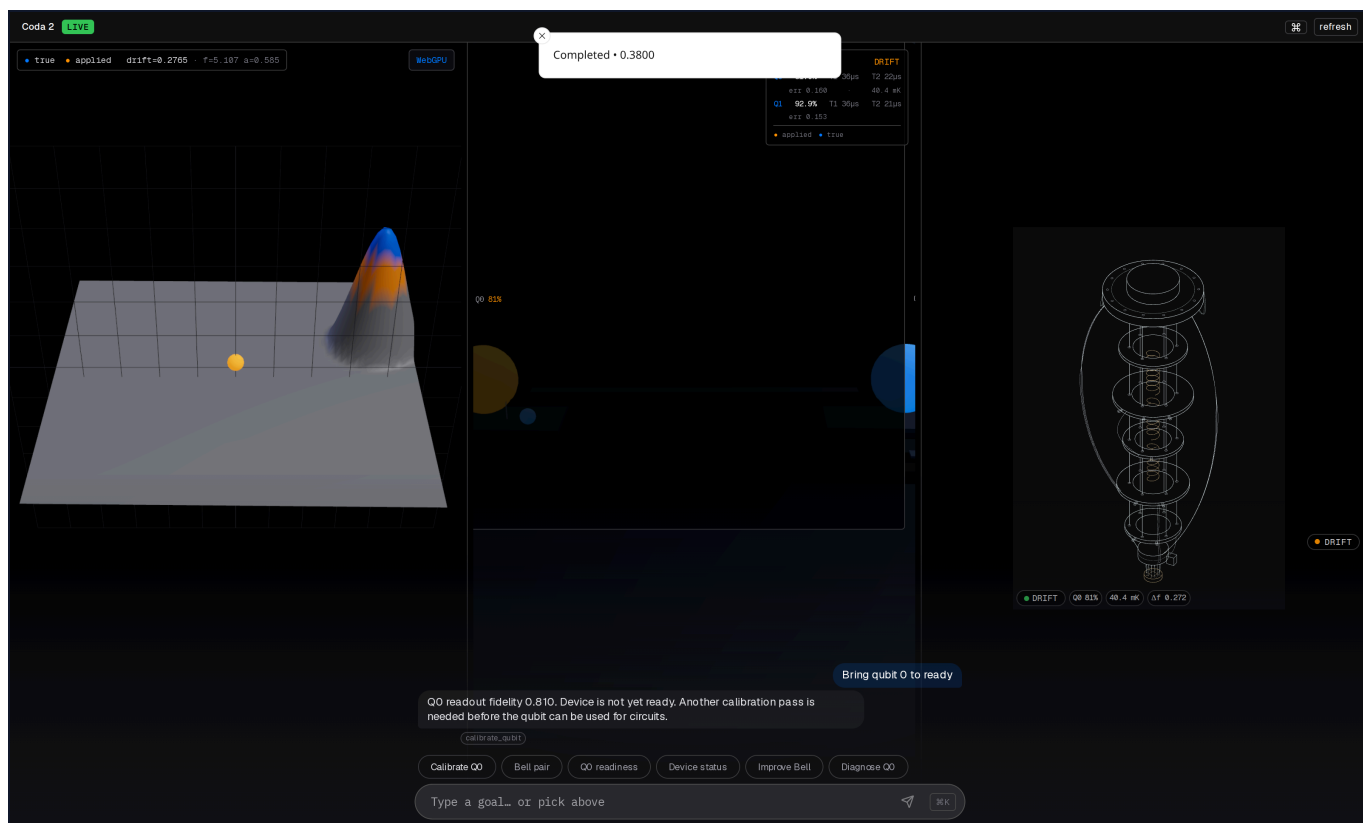


FIG. W.3 • STE.3 Calibrate turn. Your *you* bubble (“Bring qubit 0 to ready”) sits above the chips; the agent replies with the live fidelity and whether another pass is needed. All three panes stay visible: drift left, device center, cryostat right. The Calibrate Q0 path is unchanged.

Plain English for this frame. The loop is running: measure, decide, apply, measure again. When the fidelity-climb HUD is visible, its line is “how often the qubit answers correctly” across calibration steps and the orange dashed line is the pass bar (0.88). When the climb line sits above that bar, calibration succeeded for this product. The CALIBRATING badge is the same lifecycle signal in two places — the strip under the plate and the chip at the far right. If Δf is still non-tiny, controls are closing in on the sweet spot.

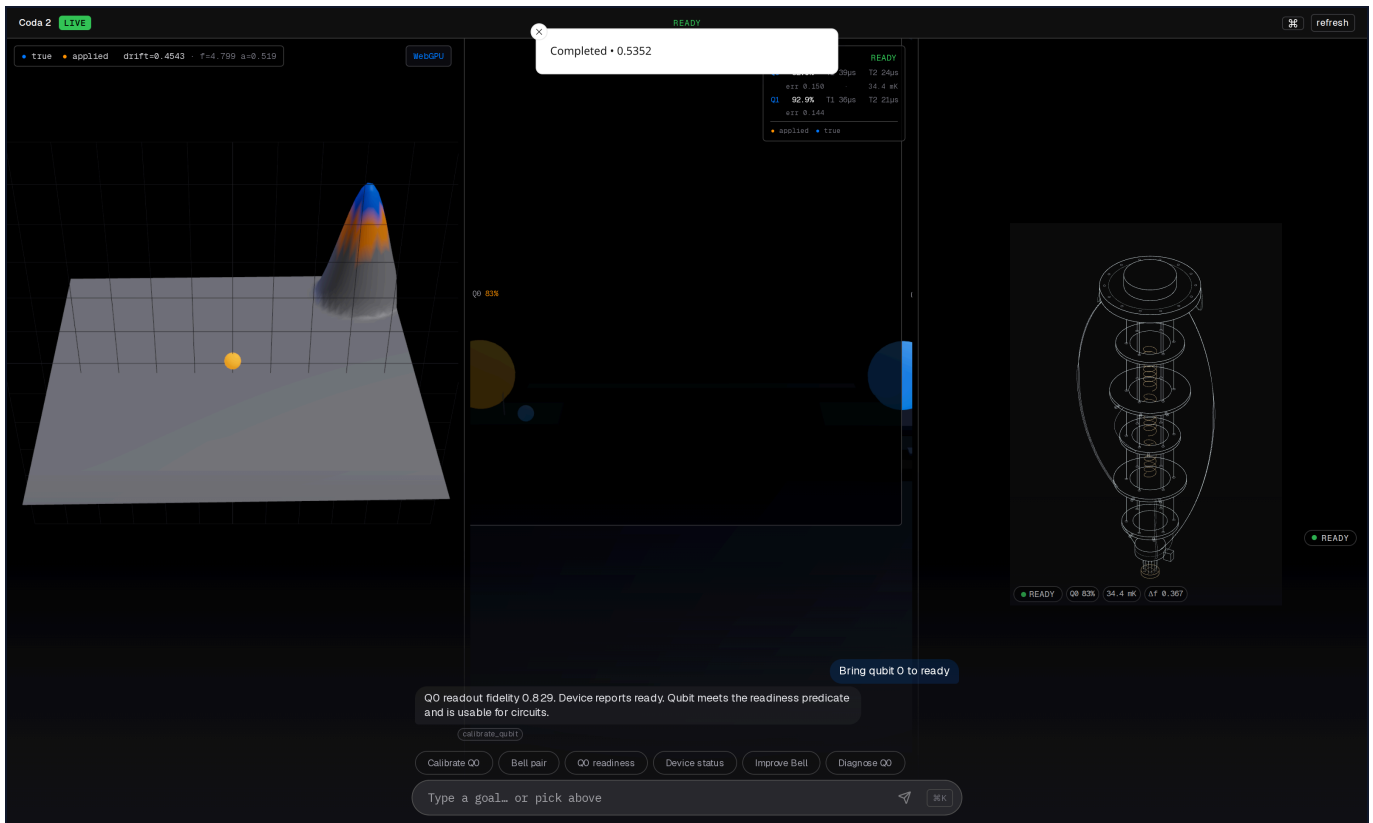


FIG. W.4 • STE.3 (SUCCESS) Successful outcome. Toolbar shows LIVE and READY. The agent reports that Q0 climbed to about 0.69 fidelity and is READY for circuits. The cryostat status strip and the `StageStateChip` at the far right read READY. All three panes stay in view: drift left, device with readout card center, cryostat plate right. Bubbles sit above chips; composer stays lowest.

Plain English for this frame. This is a successful calibrate. Read the toolbar left to right: LIVE = connected; **READY** = the device is fit for a circuit attempt; Q0 % and fid are the live fidelity. The readout card in the center pane repeats fidelity with T1 / T2 / err / mK for the device view. The strip under the cryostat plate repeats the same lifecycle state: **READY** · Q0 % · mK · Δf. The agent bubble is the narrator’s operator summary of `calibrate_qubit` — it must agree with **READY**, not contradict it. Tool pills under the bubble are the audit trail. Do not treat one screenshot’s sentence as a fixed script.

STE.4 — Three-pane stage after **READY** (same conversation)

You do. Nothing — stay on the stage. There is no device tab. Drift (left), device (center), and cryostat (right) are peer panes. Drag a splitter only if you want a different width.

What happens. See FIG. W.4. After calibrate, the `StageStateChip` at the far right reads **READY** and the cryostat status strip reads **READY** · Q0 % · mK · Δf. Toolbar shows LIVE with **READY** centered. All three panes stay visible with the same chat transcript. The left grid stays high-contrast and coplanar. The center device keeps its qubit spheres, chiplets, resonators, and the readout card at top right.

Plain English for this pane set. The glowing spheres in the center pane are qubit stand-ins. A bright / blue-leaning qubit with high % means “good enough to trust.” Orange emphasis or a low % means “still needs work.” The climb chart (if visible) still compares fidelity to the 0.88 pass bar. The state chip and the strip under the plate are the same lifecycle readout for this session — **READY** means fit for a circuit attempt. All three panes stay up; you do not flip a tab.

STE.5 — Bell pair adds a second turn

You do. Prefer READY first (toolbar green). Click chip **Bell pair** (goal: “Run a Bell pair and report fidelity”). Stay on the three-pane stage — all three panes remain visible while the transcript grows.

What happens. A second you/agent turn stacks in the transcript. Toast “Bell complete.” The agent sentence explains shot contrast or estimated fidelity in plain English.

Agent reply. The agent states the estimated fidelity — about 0.42 on this seed. Wording varies by capture. READY from calibrate still holds in the toolbar.

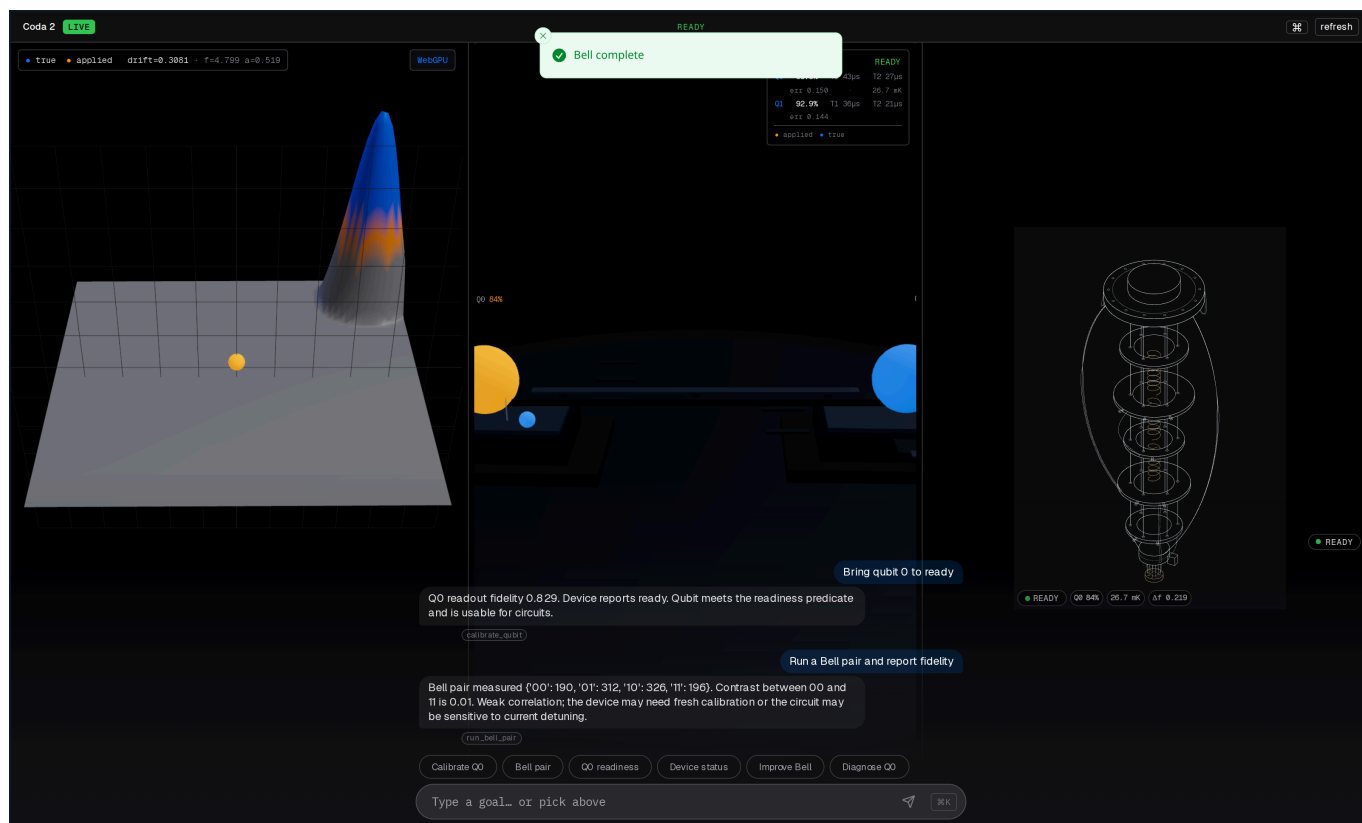


FIG. W.7 • STE.5 Multi-turn conversation on the three-pane stage. Transcript stacks calibrate then Bell above the chips; the agent reply carries a `run_bell_pair` tool pill; composer stays lowest. Drift left, device center, cryostat right, all still live. Progressive blur shows grid bleeding through the dock.

Plain English for this frame. Bell is a circuit job, not another calibration. The agent reports an estimated Bell fidelity (about 0.42 here) rather than raw shot counts: a toy noisy simulator, an estimate from few shots, and drift all push it down. The tool pill under the reply names the call that ran. High fid in the toolbar still means “answers are trustworthy enough.” Δf still means “are the controls on-target?” — if it climbs again later, re-calibrate before the next circuit. Two chat turns = two goals in one sitting: prepare the qubit, then run the entanglement check.

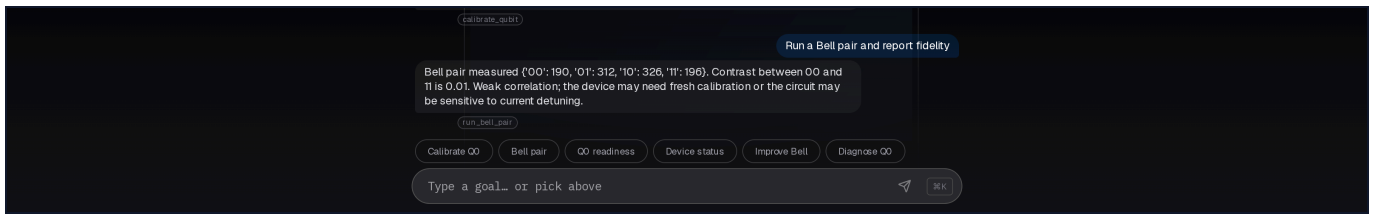


FIG. W.8 · STE.5 Dock close-up. Two turns: *you* on the right, *agent* on the left, a small tool pill under each agent reply (`calibrate_qubit` , `run_bell_pair`). The six workflow chips sit below; the composer is the lowest control.

STE.6 — Follow-up

You do. Type another goal in the composer (still lowest) — for example `device state` — and send. The transcript grows; the three panes keep updating.

What happens. A third turn appends. `get_device_state` answers from live device data: LIVE, READY, fidelity, T1 / T2, mK, Δf . The drift grid and the device readout card repaint on the next poll; the cryostat status strip keeps the same lifecycle chip.

Plain English for this step. Treat the stage as the operator’s home screen after a successful session: physics on three panes, agency in the dock. If LIVE is green and READY is on, you can try another circuit. If the state chip falls back to DRIFT or Δf grows, ask to Calibrate again — the same transcript will append a new turn. FIG. W.7 is the last wide shot; nothing here needs a new frame.

How chips and NL map to tools

CHIP / GOAL TEXT	TOOLS RUN	OPERATOR OUTCOME
Calibrate Q0 · “Bring qubit 0 to ready”	<code>calibrate_qubit</code>	Closed-loop measure → decide → apply until fid ≥ 0.88 or iterations end; on success toolbar READY and agent affirms “READY for circuits”
Bell pair · “Run a Bell pair and report fidelity”	<code>run_bell_pair</code>	Circuit job; agent explains shot contrast in plain English
Q0 readiness · “Report qubit 0 readiness and fidelity status”	<code>get_device_state</code>	Surfaces readout fidelity / readiness so Bell fidelity is not the only signal
Device status · “Report device health and temperature status”	<code>get_device_state</code>	Health / temperature / readiness snapshot
Improve Bell · “Run a precise Bell pair and report fidelity”	<code>run_bell_pair</code>	More shots (precise); better fidelity estimate after calibrate
Diagnose Q0 · “Check qubit 0 health and readout status”	<code>get_device_state</code>	Diagnostic-style status when fidelity alone is insufficient
Free text in the composer	Deterministic planner (optional LLM planner when enabled)	Same tool catalog; narrator always returns <code>agent_message</code> (NIM when keyed, else template)

Table W.2. Chips and natural-language goals map to the tool catalog. Planner stays offline-first; narrator uses NVIDIA NIM when `NVIDIA_NIM_API_KEY` is set.

HONESTY

These screenshots show a real control plane talking to a noisy simulator. There is no dilution refrigerator; the right-hand plate is an isometric illustration with live chips laid over it. Temperature in mK is computed from fidelity, not read from a fridge sensor. Detuning “true” is known only because the simulator owns it. Progressive frost is CSS: a dedicated `.stage-blur` sibling covering the bottom ~33vh with stacked solid translucent gradients (no `backdrop-filter` on `.dock` — Safari previously leaked full-stage frost). Transcript, empty state, chips, and the high-contrast composer paint above `.dock-tint` via `z-index:1`. The viz fills under the dock on purpose; only the bottom third is softened. Agent replies are generated live (NIM or template); wording varies run to run. On a real QPU the same toolbar labels still apply; the physics behind them changes behind the adapter.

Sources. Live UI at :3000 against API :8000 on main (three-pane chrome); captures under `docs/manual/assets/ui/`: `manual-3pane-home.png`, `manual-3pane-dock-chips.png`, `manual-3pane-typed.png`, `manual-3pane-calibrating.png`, `manual-3pane-ready.png`, `manual-3pane-bell.png`, `manual-3pane-dock-transcript.png` (STE.1–STE.6); `ui/app/page.tsx` (three-pane stage, two splitters, chat transcript + composer); `ui/app/globals.css` (`.dock` progressive layers); `ui/components/viz/calibration-surface.tsx`; `ui/components/viz/device-3d.tsx` (readout card); `ui/components/viz/cryostat-plate.tsx` (plate + status strip); `ui/components/stage/stage-state-chip.tsx`; `orchestrator/narrator.py`; `orchestrator/planner.py`; `calibration/service.py` (threshold 0.88); `models/types.py` (READINESS 0.82); `api/server.py` (`agent_message` on `POST /goals`).

FRONT MATTER

Release notes

One line per release, newest first. This book describes `main` at `3cc9bed` (three-pane chrome through PR #27, plus the stage-blur edge fix). Dates are America/Los_Angeles. Older rows keep the AuthorDate previously recorded from git.

RELEASE	DATE (PT)	WHERE	CHANGE
0.2.0 / coda-2	2026- 10- 04/05	main @ 3cc9bed	Three-pane stage (drift · device 3D · cryostat); chat bubbles; high-contrast composer; Safari-safe bottom-third <code>.stage-blur</code> (solid bands); six workflow chips (readiness / status / improve Bell / diagnose); product rename to Coda 2. PR #25–#27.
0.1.0 / #19–#24	2026- 10-04	main	Cryostat plate added as the third pane (#19); cryostat layout polish and Safari frost fixes; larger mid-pane 3D device (#20–#24).
0.1.0 / #18	2026- 10- 03/04	main @ b05005c	Higher contrast on the left drift grid. iPadOS Split View–style resizable separator with a capsule drag handle on hover.
0.1.0 / #17	2026- 10- 03/04	main	Left drift grid aligned coplanar with the fidelity surface (shared tilted group).
0.1.0 / #16	2026- 10- 03/04	main	Richer right Device3D: chiplets, resonators, and a live fidelity / T1 / T2 strip.
0.1.0 / #15	2026- 10- 03/04	main	Drift framing shifted up and zoomed out.
0.1.0 / #14 / #13	2026- 10- 03/04	main	Zoom-out steps for the left drift camera.
0.1.0 / #12	2026- 10-02	main @ 6a1e168	True side-by-side stage: equal peer panes (left param-drift, right 3D device); no glass inset.
0.1.0 / #11	2026- 10-02	main	Combined drift+device stage (no tabs); glass drift inset; remove macOS traffic lights; founder-demo toolbar polish.
0.1.0 / #10	2026- 10-02	main	Device HUD: clear label overlaps; bottom-right StageStateChip (OFFLINE → DRIFT → CALIBRATING → RUNNING → READY/FAILED).
0.1.0 / #9	2026- 10-02	main	Narrator honors post-goal device.is_ready; READY affirmation when calibration succeeds.
0.1.0 / #2	2026- 10-02 10:27	main	3D device panel; hardened calibration; optional LLM planner with fallback; founder-demo Makefile polish.
0.1.0 / #1	2026- 10-01 23:22	main	Squash-merge: QPUAdapter + NoisySimulatorBackend, CalibrationService, Orchestrator (5 tools), FastAPI control plane, Next.js instrument UI (65/35, WebGPU drift), three metrics, demos, pytest.
0.1.0 (PR)	2026- 10-01	PR #1	Multiple commits squashed: Streamlit → Next.js+shadcn; ToolTrace observability; founder README; detuning card; demo guardrails; progressive-blur dock.
init	2026- 09-29 00:32	main	Initial commit (empty / scaffold).

Table R.1. Releases on main. “Where” is the git ref that landed the change. The #12 ref is the commit that landed that release. It is not the tip of this book.

PR #1’s squash commit message enumerates the delivered surface: adapter interface and toy 1–2 qubit simulator with drift; iterative gradient-free calibration targeting a fidelity threshold; orchestrator with deterministic planner (offline, no LLM keys); FastAPI endpoints for goals, jobs, metrics, health, SSE, and direct calibrate/circuit; Next.js UI replacing an earlier Streamlit prototype; demos for calibration and Bell; metrics `time_to_calibrated`, `calibration_success_rate`, `interface_latency`; pytest covering contract, convergence, API happy path, and failure paths.

Later amendments inside the same PR added structured ToolTrace emission on every orchestrator call, a detuning/drift narrative in the UI, cancel guardrails, and the instrument layout (stage hero + progressive-blur dock). The book documents the merged result only — not intermediate PR commits as separate releases.

PR #2 (merged 2026-10-02 PT) keeps the adapter seam and adds founder-demo outcomes: a React Three Fiber *device* tab beside the existing drift landscape; calibration hardening so typical drift seeds climb past 0.88; an optional LLM planner with JSON extraction and hard fallback to `plan_from_goal`; and Makefile targets for founder demos (including LLM and stub modes). Default path remains deterministic and offline. The device tab from PR #2 is historical. Current UI is the three-pane stage (#19 onward), not a tab flip.

Metrics this system is designed to expose

This system has a single shipping revision. The numbers below are the *contract* of what the control plane measures. Typical demo ranges come from the default noisy simulator (seeded). They are not a multi-week A/B study.

METRIC	DEFINITION	WHERE SET / READ	TYPICAL DEMO RANGE
time_to_calibrated	Wall seconds from calibrate() start to best_fid ≥ threshold (successes only)	CalibrationMetrics.total_time_to_cal_s / successes	Sub-second to a few seconds on sim
calibration_success_rate	successes / attempts (running)	CalibrationMetrics	Often high on sim; 0 when fid-capped
interface_latency	Seconds per apply_calibration_update roundtrip	sampled each iteration	Near-zero on in-process sim
readiness	all qubits readout_fidelity ≥ 0.82	DeviceState.is_ready	Boolean; notes string
service threshold	best_fid ≥ 0.88 for CalibrationResult.success	CalibrationService.fidelity_threshold	API wires 0.88 / max 60

Table R.2. Metrics contract. Sim timings are toy; real QPU wall times will dominate.

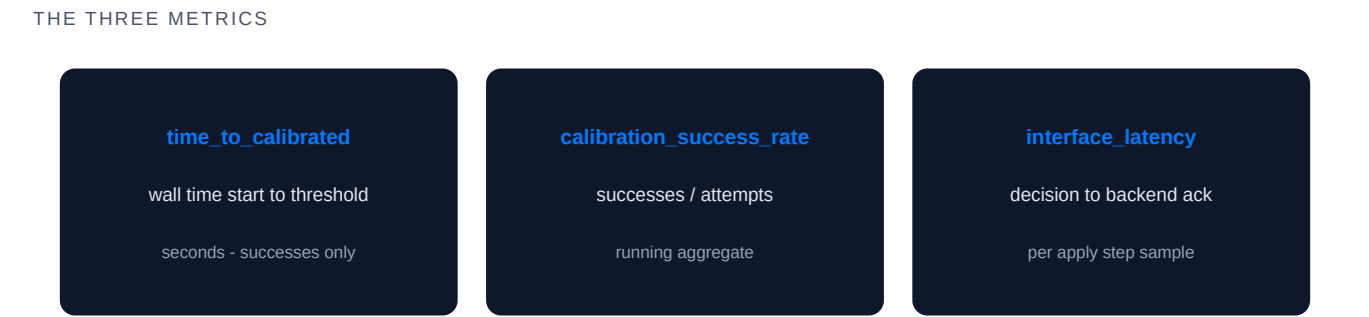


FIG. R.1 The three product metrics. CalibrationService owns the first two aggregates and the latency samples; MetricsAggregator additionally rolls latency percentiles and fidelity samples for the HTTP /metrics surface.

What is real vs simulated

HONESTY

The control plane (adapter contract, orchestrator, traces, FastAPI, Next.js UI, metrics) is real software. The device is a *NoisySimulatorBackend*: a toy 1–2 qubit model with a hidden true parameter vector, random-walk and sinusoidal drift, and a monotonic-but-noisy fidelity function of $|l_{\text{applied}} - t_{\text{ruell}}|$. Bell counts are a cheap stochastic simulation. There is no dilution refrigerator. There is no AWG. There is no pulse compiler. A `ConductorShapedAdapter` stub documents the shape of a real driver and refuses real work.

How to read this book

This manual is an engineering book for Coda 2, the AI → QPU integration layer. It is not a marketing deck. Start with **The instrument** (FIG. 0 — the live three-pane still). Then **Who this is for** (readers and the four tasks). Then the **Instrument walkthrough** (screenshots and flows). Then read Part 1 (mental model) and Part 2 (theory of the interface) before the reference chapters. Everything cites module paths under `src/conductor_qpu/` or `ui/` on main (three-pane chrome).

Part map

0	The instrument	three-pane still, multimodal, six chips
A	Who this is for	readers, four tasks, instrument map
W	Instrument walkthrough	screenshots, readouts, flows
1	Mental model	QPU architecture, seam, goals, loop
2	Theory of the interface	why this solution works
D	Diagrams	how language becomes physics
3	System architecture	components and data flow
4	Control plane / API	every FastAPI endpoint
5	Adapter & noisy simulator	contracts, noise, state
6	Calibration & metrics	algorithm, thresholds, failures
7	Orchestrator & tools	catalog, traces, planner
8	Operator UI	three-pane stage, dock, chips
9	Operations	install, Makefile, demos, tests
10	Worked walkthroughs	Calibrate Q0, Bell, fail paths
11	Reference	env, packages, glossary, figures

Conventions

MARK	MEANING
<code>path/to/file.py</code>	Exact path in the repo relative to package root or ui/
FIG. n.m	Captioned figure (SVG diagram or screenshot)
Table n.m	Reference table
Slate	Control plane / orchestrator in diagrams
Blue #007AFF	True / target / high fidelity (iOS system blue)
Orange #FF9500	Applied calibration / mid band / threshold line
Green #34C759	Success / READY / LIVE
Red #FF3B30	Failure / cancel / capped fidelity (UI diagrams)
Seafoam / olive / blue / red	Mid-century plate palette (#c5d5d8, #7a8a4a, #5a8fb5, #c41e1e) — Diagrams part + cover
Rule of thumb	Operator-facing design rule
Honesty	Explicit limit of this system (sim vs hardware)
Sources	Code paths and symbols that back the claim

Table C.1. Typographic and colour conventions used throughout.

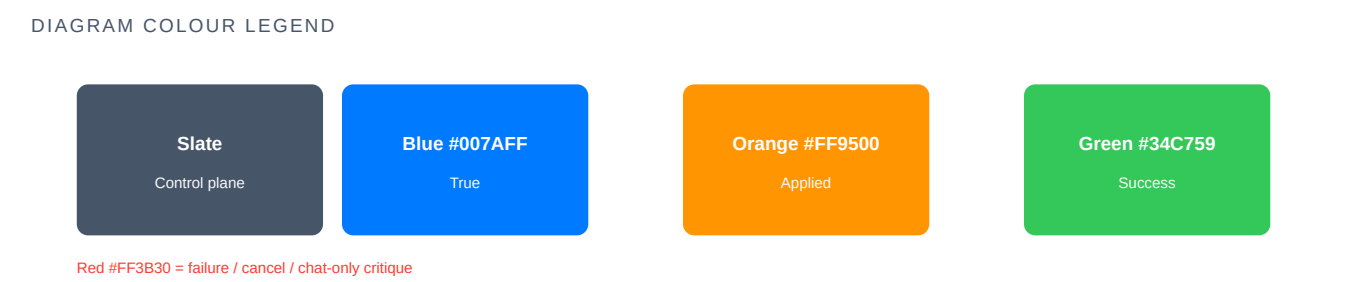


FIG. C.1 Diagram colour legend. Matches the UI's iOS system palette used in calibration-surface.tsx and the status chrome.

Numbers you will see often

- **0.82** — `READINESS_READOUT_FIDELITY_THRESHOLD` : device `is_ready` iff every qubit's readout fidelity is above this.
- **0.88** — default `CalibrationService.fidelity_threshold` and API wiring; also the planner's default `target_fidelity` for "bring to ready".
- **0.55** — lower clamp on simulator fidelity; also the stub's `measure_fidelity` return (permanently below threshold).
- **60** — API server's `max_iterations` for `CalibrationService`.
- **3 panes** — drift | device 3D | cryostat across the stage; the dock overlays the bottom third over a progressive frost (top ~2/3 stays sharp).

Sources. src/conductor_gpu/models/types.py (READINESS_*); calibration/service.py; api/server.py (_calibration construction); ui/app/page.tsx; ui/components/viz/calibration-surface.tsx.

Mental model

Start with the machine. Then look at the plug. A QPU is a set of two-level systems. Classical pulses steer them. Readout returns classical bits. The device drifts. This software sits above that stack. Natural-language goals and an orchestrator call a narrow `QPUAdapter`. The backend stands in for cryo and electronics. Four nouns remain clear once that seam is visible: goals, tools, jobs, and the calibration loop.

1.1 How a QPU works

A quantum processing unit is not a CPU with exotic opcodes. It is a small set of engineered two-level systems (qubits). Microwave and baseband electronics couple to those qubits. A cryogenic environment keeps thermal noise low so coherence lasts. Classical software decides which pulses to send. The physics decides what you get back. The primer below is the minimum model an operator or agent author needs.

1.1.1 Qubit modality — superconducting transmon

This manual uses a superconducting transmon as the example modality. A transmon is a Josephson junction with a large shunt capacitor. That circuit forms a weakly anharmonic oscillator. The two lowest levels are the computational states `|0⟩` and `|1⟩`. A readout resonator couples to the qubit. A microwave drive (XY) rotates the state. A flux or Z bias sets the qubit frequency when the junction is tunable. Other modalities (spin, trapped ion) differ in device physics. They still use pulses for control and a classical result for readout.

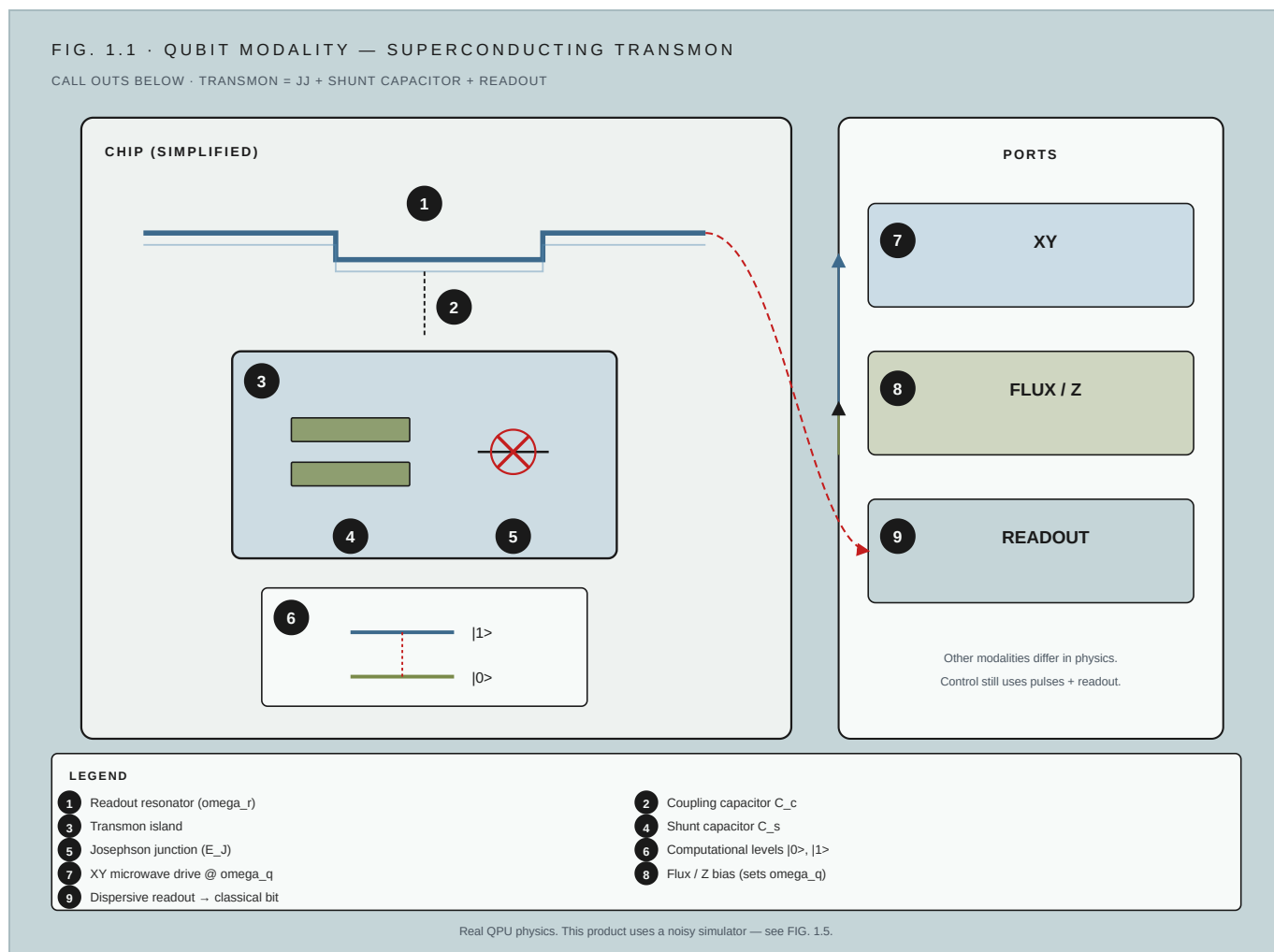


FIG. 1.1 Qubit modality — superconducting transmon with readout resonator, XY drive, and flux/Z bias.

The chip holds a transmon and a readout resonator. The Josephson junction sets the nonlinearity. The shunt capacitor sets the charging energy. XY microwave pulses rotate the state. Flux bias sets frequency on tunable devices. Readout probes the resonator and returns a classical bit. This figure shows real QPU physics. This product uses a noisy simulator — see FIG. 1.5.

Cite. P. Krantz et al., “A quantum engineer’s guide to superconducting qubits,” Appl. Phys. Rev. 6, 021318 (2019); arXiv:1904.06560 — <https://arxiv.org/abs/1904.06560>

1.1.2 Cryogenic and RF stack

Room-temperature racks hold the AWG, DAC, local oscillator, mixers, digitizer, and control PC. Drive lines go down through the dilution refrigerator. Stages typically run at about 300 K, 4 K, still (~ 1 K), and the mixing chamber (MXC, ~ 10 – 20 mK). Attenuators on the drive path cut thermal noise into the qubit. The readout path amplifies a weak response: a TWPA near the sample, then a HEMT at 4 K, then room-temperature electronics. The sample package sits at the MXC.

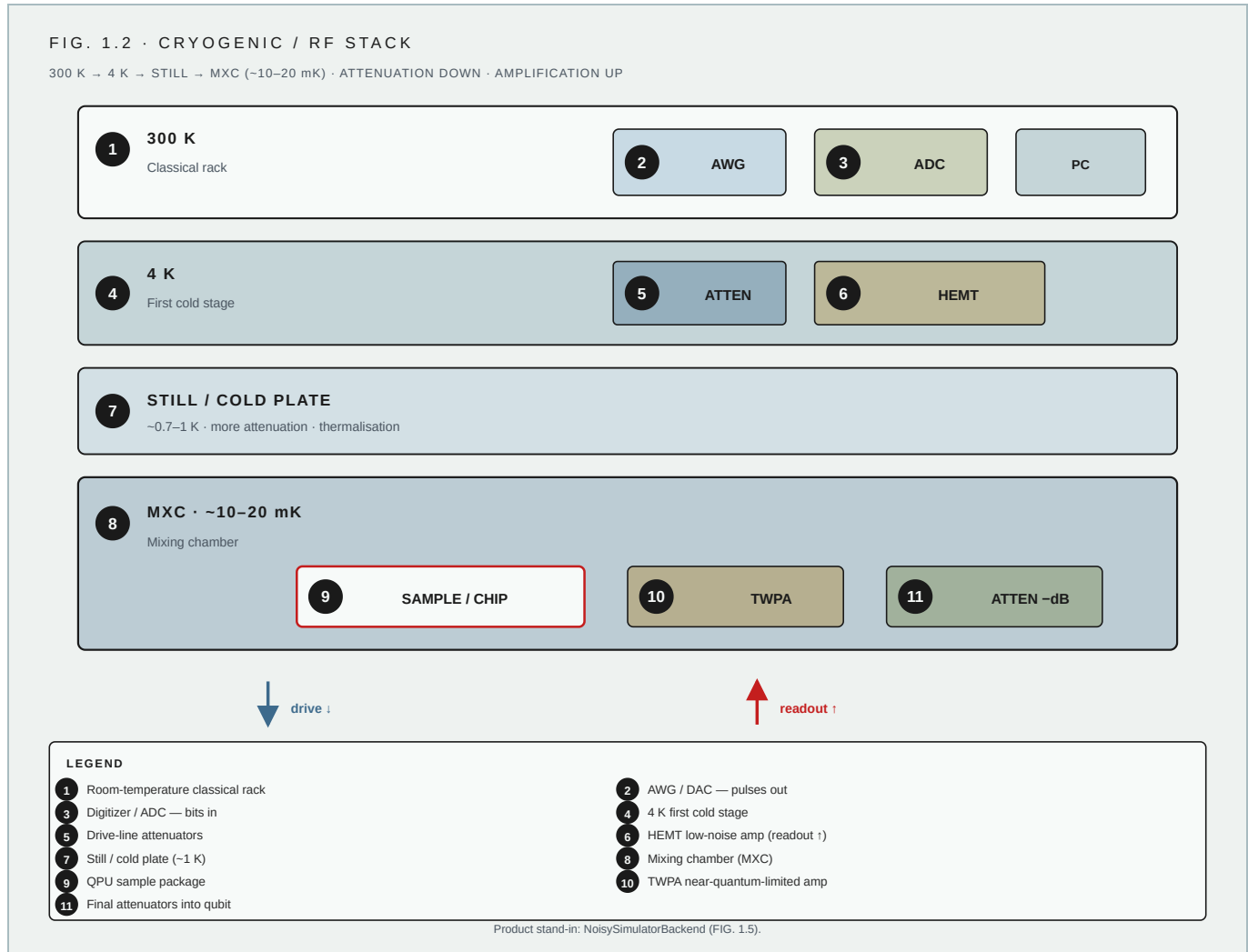


FIG. 1.2 Cryogenic / RF stack — 300 K rack through MXC, with attenuation down and amplification up.

Drive signals move down the fridge. Attenuators protect the qubit from room-temperature noise. Readout signals move up. A TWPA and a HEMT amplify the weak probe response. The digitizer turns that response into bits. This stack is real QPU infrastructure. The product stands in for it with `NoisySimulatorBackend` (FIG. 1.5).

Cite. P. Krantz et al., Appl. Phys. Rev. 6, 021318 (2019), arXiv:1904.06560; Keysight, “Characterizing Superconducting Qubits” application note — <https://www.keysight.com/us/en/assets/3120-1387/application-notes/Characterizing-Superconducting-Qubits.pdf>

1.1.3 Control and readout loop

Gates are the logical recipe (X, Y, CZ, Rz, and others). Pulses are the physical voltages and microwave envelopes. An AWG and LO emit those envelopes. Fridge wiring carries them to the qubit. The qubit state evolves. A readout chain amplifies the resonator response. A digitizer yields a classical bitstring. The classical PC classifies the bits, logs them, and decides the next pulse sequence. Fidelity is the rate at which those bits match the intended result.

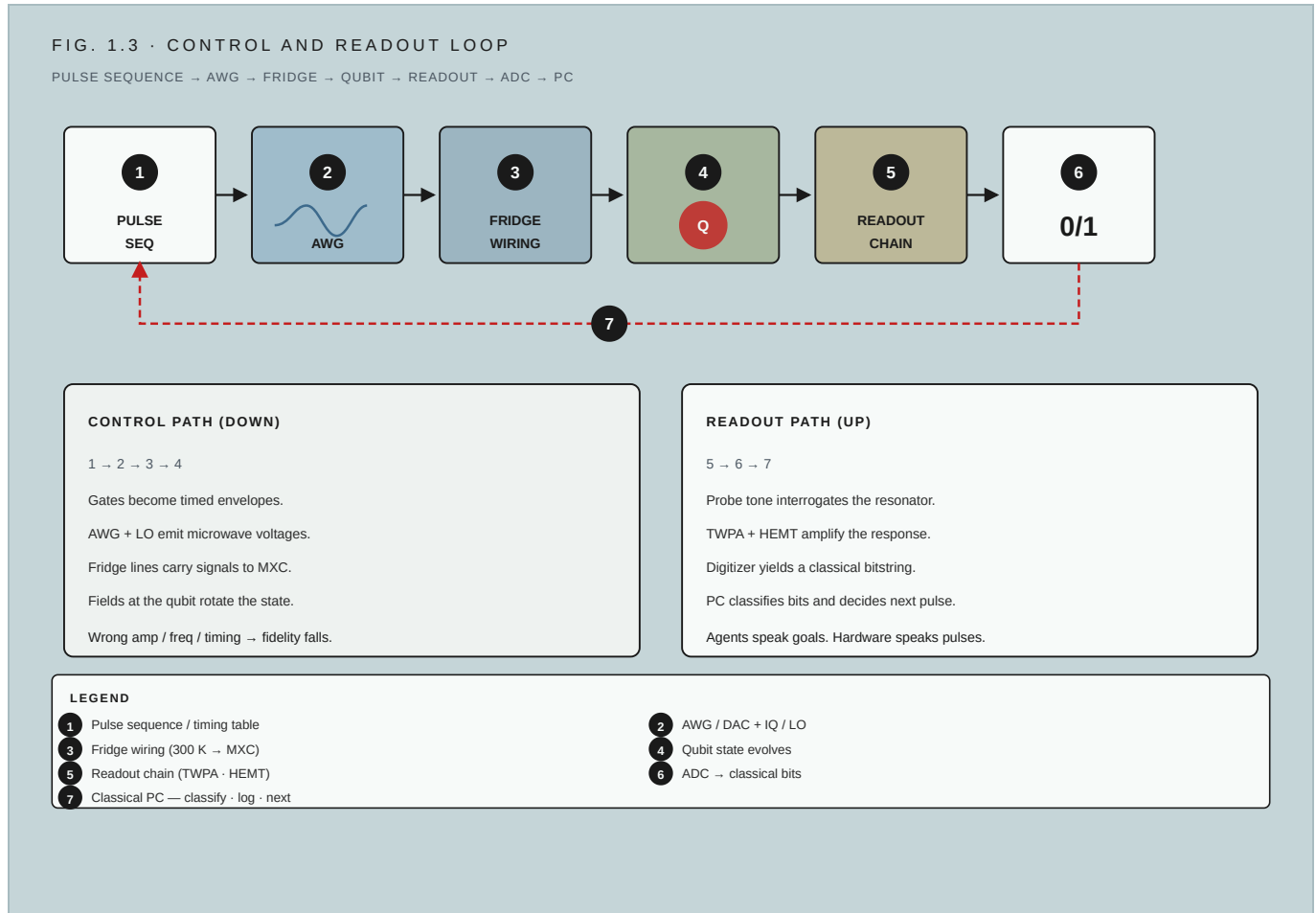


FIG. 1.3 Control and readout loop — pulse sequence → AWG → fridge → qubit → readout chain → digitizer → PC.

Control is continuous and analog on the wire. Readout is discrete and classical in the log. An agent does not speak in volts. It speaks in goals. Every successful goal becomes a pulse schedule and a measured bitstring. Wrong amplitude, frequency, or timing rotates the wrong amount. Fidelity falls.

Cite. P. Krantz et al., arXiv:1904.06560; IBM Qiskit Pulse (backend control / Schedule) — <https://docs.quantum.ibm.com/api/qiskit/0.46/pulse>; Keysight, “Characterizing Superconducting Qubits” — <https://www.keysight.com/us/en/assets/3120-1387/application-notes/Characterizing-Superconducting-Qubits.pdf>

1.1.4 Calibration as a closed loop on that stack

The device moves. Temperature, LO phase, flux bias, and cable response wander. Applied parameters drift away from the true sweet spot. That gap is detuning. As detuning grows, gate fidelity falls. Circuits stop being useful. Calibration is a recurring loop: measure, decide whether to move knobs, apply an update, and remeasure. Knobs include amplitude, frequency, and timing. Observables include gate fidelity, readout fidelity, and (on hardware) T1/T2-class metrics. This simulator reports a fidelity kernel instead of full T1/T2.

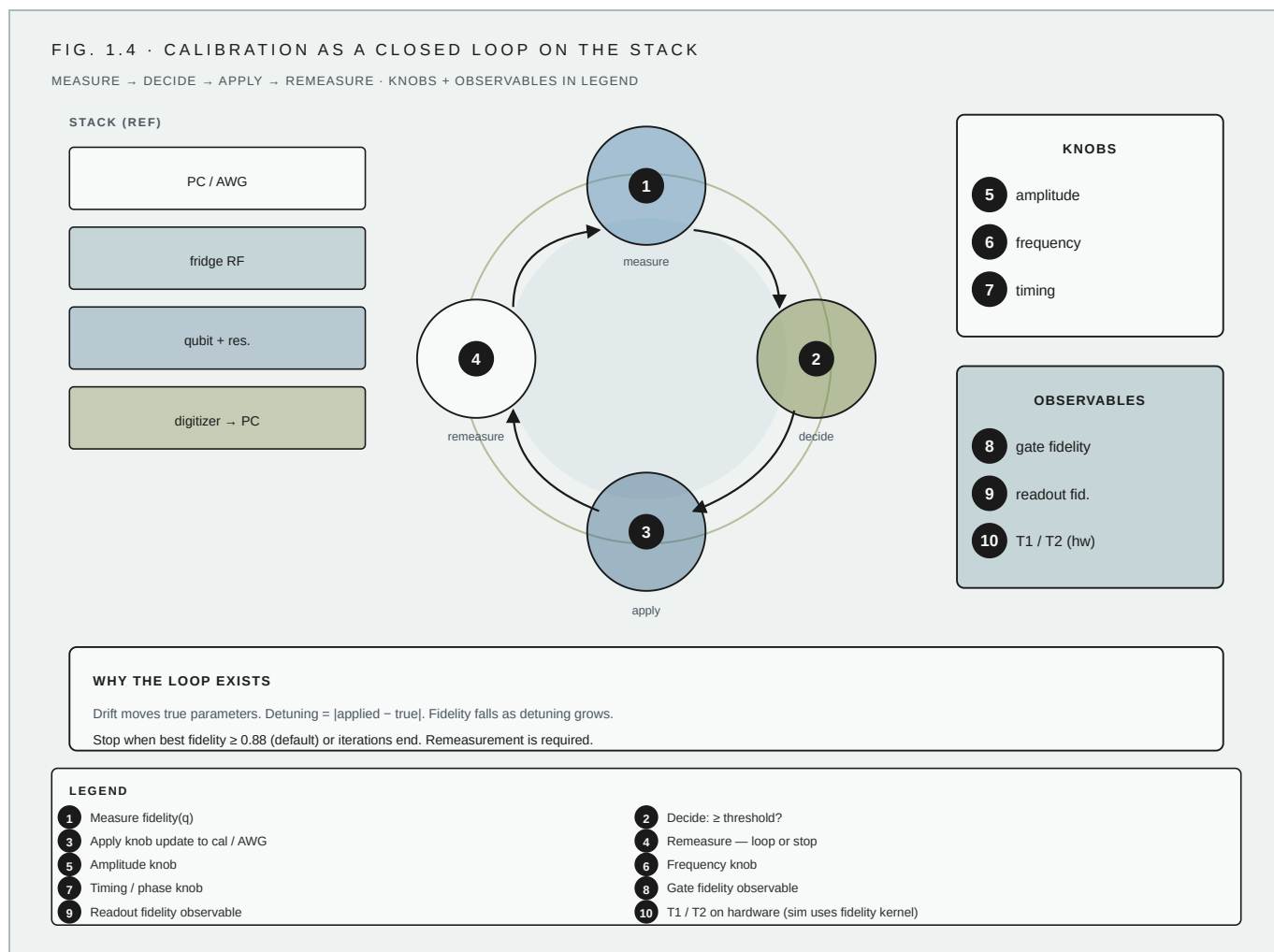


FIG. 1.4 Calibration as a closed loop — knobs (amplitude, frequency, timing) and observables (fidelity, readout fidelity; T1/T2 on hardware).

Drift moves true parameters. Detuning is the measurable gap. Fidelity decay is why you care. Stop when best fidelity meets the threshold (default 0.88) or iterations end. Without remeasurement against a threshold, calibration is not an agent tool. It is a hope.

Cite. N. Wittler, F. Roy, K. Pack, M. Werninghaus, et al., “Integrated Tool Set for Control, Calibration, and Characterization of Quantum Devices Applied to Superconducting Qubits,” Phys. Rev. Applied 15, 034080 (2021) — <https://journals.aps.org/prapplied/abstract/10.1103/PhysRevApplied.15.034080>

1.2 Where we plug in

This system does not replace the fridge. It plugs in above classical control. It sits at the software ↔ device boundary. Natural-language goals and the orchestrator sit with the operator layer. The `QPUAdapter` is the contract. Six typed methods carry all device talk. In this prototype, `NoisySimulatorBackend` stands in for cryo plus electronics. It is a toy 1–2 qubit model with hidden true parameters, drift, and a climbable fidelity kernel. Closed-loop calibration uses measure → decide → apply through that same adapter. The instrument UI keeps physics in view. Three resizable panes: a high-

contrast drift grid on the left, a denser 3D device in the middle, and an isometric cryostat plate on the right. Two resizable splitters (capsule handles on hover; about 22–28% minimum per pane) separate them. There are no tabs. PR #2 used tabs labeled drift | device. That flip is past. The 3D cryo device panel is a UI surface. It is not a real fridge. The dock holds the agent ledger.

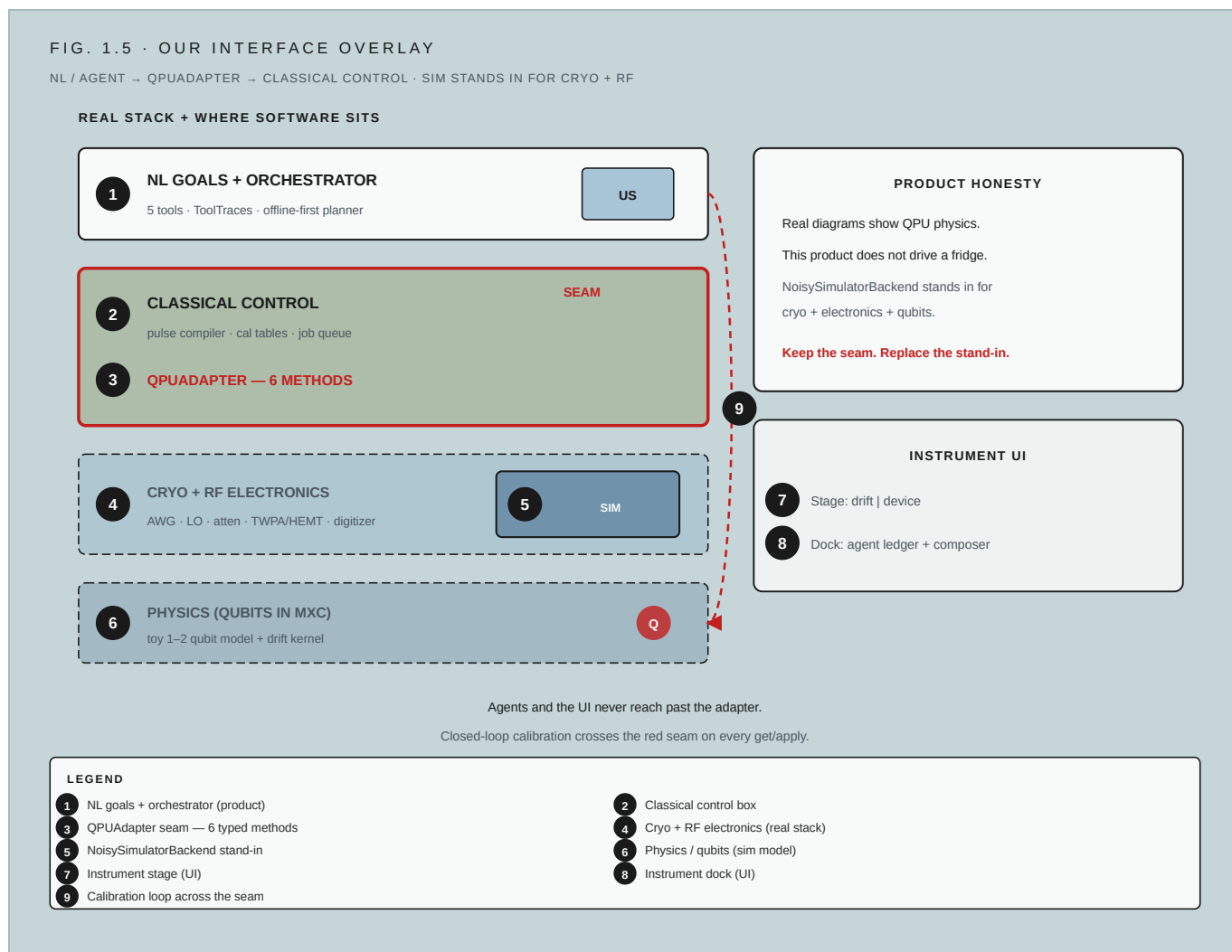


FIG. 1.5 Interface overlay — NL/agent/orchestrator map onto the classical-control box; NoisySimulatorBackend stands in for cryo and electronics.

The red bar is the product seam. Agents and the UI never reach past it. Swap the noisy simulator for a real AWG driver later. Keep the tools, traces, metrics, and instrument. The dashed loop is calibration. Every get or apply of cal params crosses the adapter. Each apply samples interface latency. The 3D device panel is UI, not hardware.

Cite. Product seam (this repo). Pulse-schedule concept: IBM Qiskit Pulse — <https://docs.quantum.ibm.com/api/qiskit/0.46/pulse>. Calibration-loop pattern: Wittler / Werninghaus et al., Phys. Rev. Applied 15, 034080 (2021).

1.3 Goals

A *goal* is a short natural-language string. An operator or agent submits it. Examples the deterministic planner accepts: “Bring qubit 0 to ready”, “Run a Bell pair and report fidelity”, “Report device health and temperature status”, “Check qubit 0 health and readout status”, “cancel <job-id>”. Goals enter through `POST /goals` or the UI composer, chips, or $\mathbb{K}$ palette. They are not LLM prompts by default. The planner in `orchestrator/planner.py` maps them to a list of `ToolCall` records. Optional LLM planning exists behind `CONDUCTOR_ENABLE_LLM`. It is off by default. The product path is offline-first.

1.4 Tools

The orchestrator registers five tools: `calibrate_qubit`, `run_bell_pair`, `get_device_state`, `get_job_status`, `cancel_job`. Each call produces a `ToolResult` and a `ToolTrace` (tool name, args, latency, ok, summary). Traces are the observability product. They show real control-plane decisions. They are not narrative fiction. Heavy paths (`calibrate_qubit`, `run_bell_pair`) cross the adapter. The other tools give visibility and guardrails.

ORCHESTRATOR TOOL CATALOG (5)

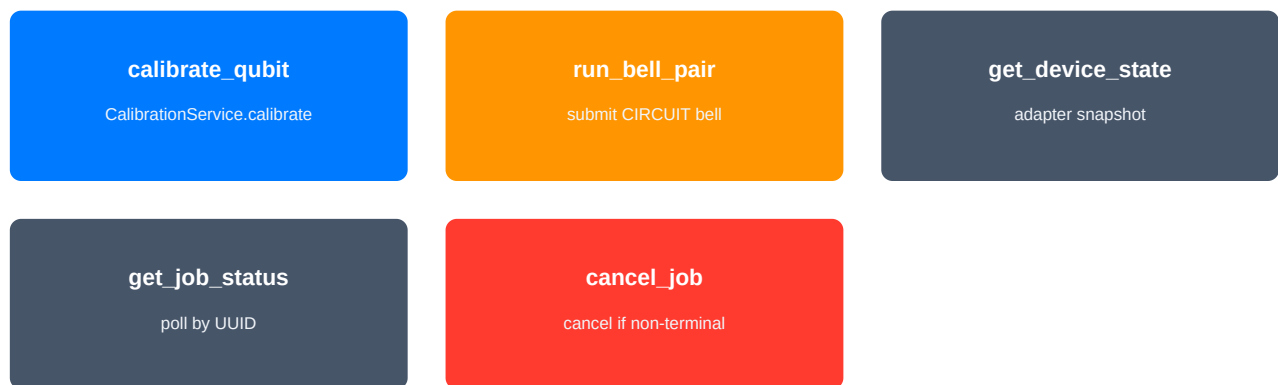


FIG. 1.6 The five built-in tools. `calibrate_qubit` and `run_bell_pair` are the heavy paths. The others give visibility and guardrails.

1.5 Jobs

A `QPUJob` is a typed record. It has a `JobType` (calibration | circuit | diagnostic) and a `JobStatus` (queued → running → succeeded | failed | cancelled). The adapter owns submission and polling. An `InMemoryJobStore` exists as a seam for durable history later. Jobs survive a UI refresh for the lifetime of the API process.

1.6 The calibration loop

Calibration is measure → decide → apply → remeasure. Continue until `best_fid ≥ threshold` or iterations end. That loop makes an agent interface viable. Each step has a typed result, a latency sample, and a fidelity point the UI can plot. Without the loop, “calibrate” is a fire-and-forget RPC with no story. Every apply goes through `QPUAdapter.apply_calibration_update` — the seam in FIG. 1.5.

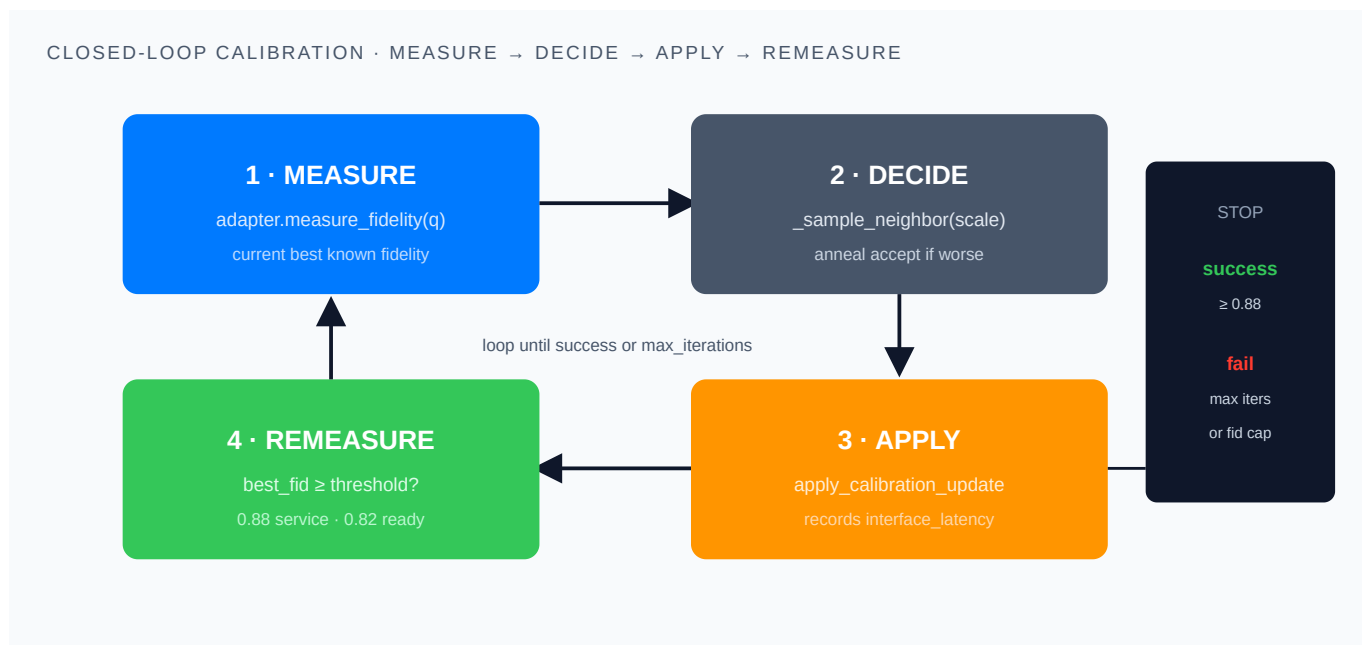


FIG. 1.7 Closed-loop calibration. Interface latency is sampled on every apply. time_to_calibrated is wall time to success. success_rate counts attempts.

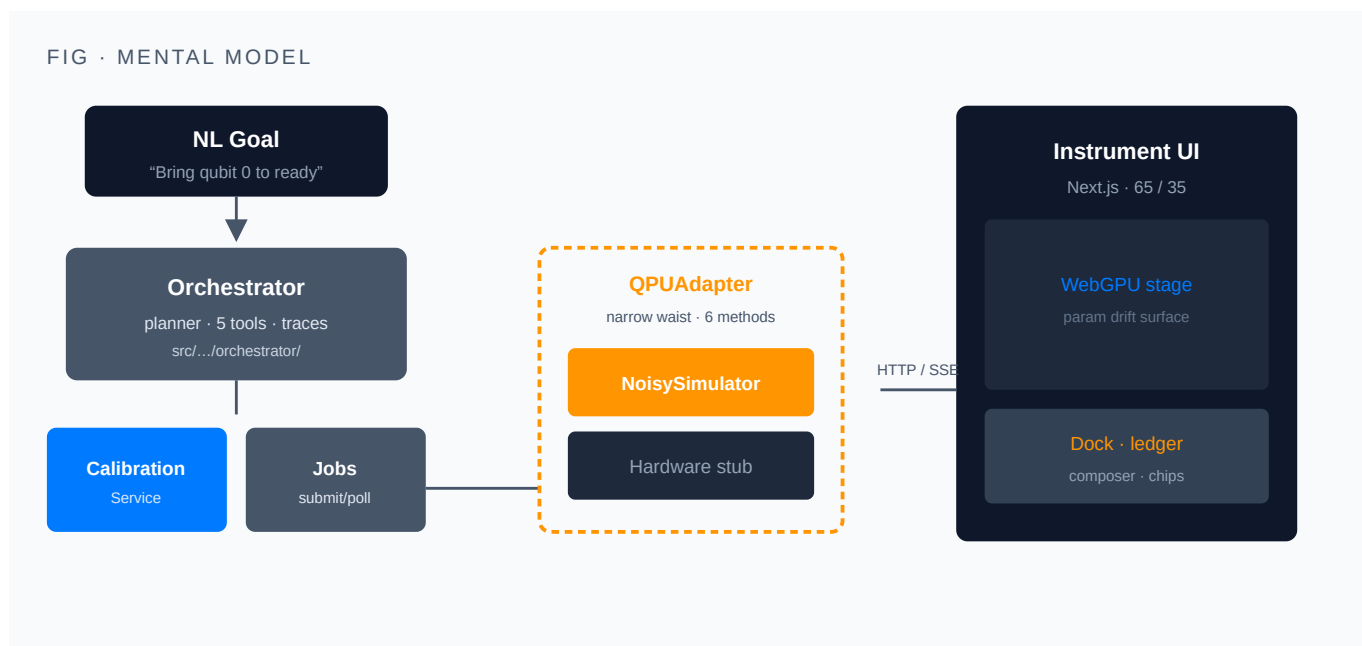


FIG. 1.8 Software stack map: NL goal → orchestrator → calibration/jobs → QPUAdapter → simulator or stub. UI talks HTTP/SSE to the control plane.

1.7 Readiness vs calibrated

Two predicates matter. *Device readiness* (`is_ready`) requires every qubit’s readout fidelity ≥ 0.82 . *Calibration success* requires the service’s best fidelity ≥ 0.88 (default). A device can be “ready” by the weak predicate while a strict calibration goal still runs. The UI shows both: READY/CAL NEEDED in the toolbar, and the climb HUD threshold line

at 0.88. The stage's 3D device panel (tab **device**) gives a spatial reading of the same health story. The dock ledger narrates that story in tool traces.

TWO THRESHOLDS - READINESS VS CALIBRATION SUCCESS

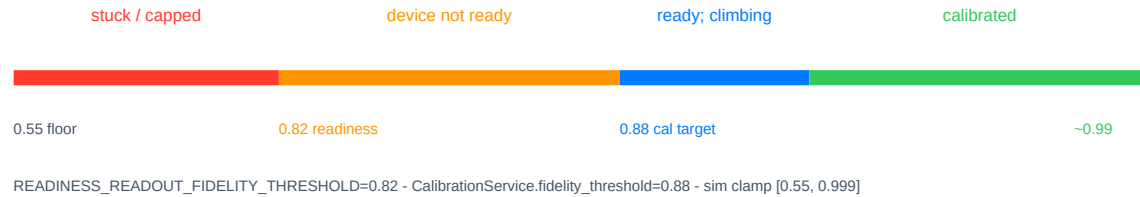


FIG. 1.9 The fidelity number line. The simulator clamps reported fidelity to [0.55, 0.999]. Demo caps can pin the ceiling near 0.69 to force failure.

Sources. adapter/base.py (QPUAdapter); adapter/noisy_sim.py; orchestrator/orchestrator.py (tools, ToolTrace); orchestrator/planner.py (plan_from_goal); models/types.py (Job*, READINESS_*); calibration/service.py; ui/app/page.tsx; ui/components/viz/Device3D*.tsx. Architecture literature: P. Krantz et al., Appl. Phys. Rev. 6, 021318 (2019), <https://arxiv.org/abs/1904.06560> (FIGS. 1.1–1.3); IBM Qiskit Pulse, <https://docs.quantum.ibm.com/api/qiskit/0.46/pulse> (FIGS. 1.3, 1.5); N. Wittler / M. Werninghaus et al., Phys. Rev. Applied 15, 034080 (2021) (FIG. 1.4); Keysight, “Characterizing Superconducting Qubits,” <https://www.keysight.com/us/en/assets/3120-1387/application-notes/Characterizing-Superconducting-Qubits.pdf> (FIGS. 1.2–1.3).

Theory of the interface

This part states why an NL-goal and tool-using agent layer on a narrow QPU adapter is the right seam for calibration-heavy quantum ops. It also states where this system’s honesty line cuts. It is the intellectual core of the book.

2.1 The problem the interface solves

Quantum hardware drifts. Before a circuit is worth running, someone — human or agent — must decide whether to calibrate, which knobs to move, whether the move helped, and how that correlates with downstream job quality. Vendor pulse APIs expose the knobs. They do not expose the *decision loop* as a first-class, observable object. Chat-only LLM UIs expose language. They do not expose continuous detuning, fidelity climb, or a tool ledger operators can audit.

The AI → QPU integration layer asserts a third surface: **goals in, typed tool traces out, device physics behind an adapter**. Calibration is not a side effect of “run circuit”. It is an explicit tool with a success predicate and three metrics.

RULE OF THUMB

If an operator cannot point at a ToolTrace row and a fidelity history for a calibration attempt, the control plane is not yet agent-ready. It is still a pulse console with a chatbot taped on.

2.2 NL goals and tool-using agents vs bare pulse APIs

Bare pulse APIs are necessary and not enough. They are the contract a driver must honour (load waveforms, arm triggers, commit cal tables). They are the wrong API for an agent that reasons in tasks: “bring Q0 to ready”, “don’t run Bell until ready”, “cancel the stuck diagnostic”. Agents need a small catalog of tools with stable names and JSON-shaped results. Humans need the same catalog as chips and a composer, not as a 200-method SDK.

This system’s planner is deliberately simple and deterministic. That is a feature for a founder demo: the mapping from goal text to tools is auditable in one function. When `CONDUCTOR_ENABLE_LLM=1`, an optional OpenAI planner may emit the same `ToolCall` schema. The execution path, traces, and metrics do not change. The interface’s value is the tool surface and the loop, not the quality of the NL parser.

THREE CONCERNS · ONE DIRECTION

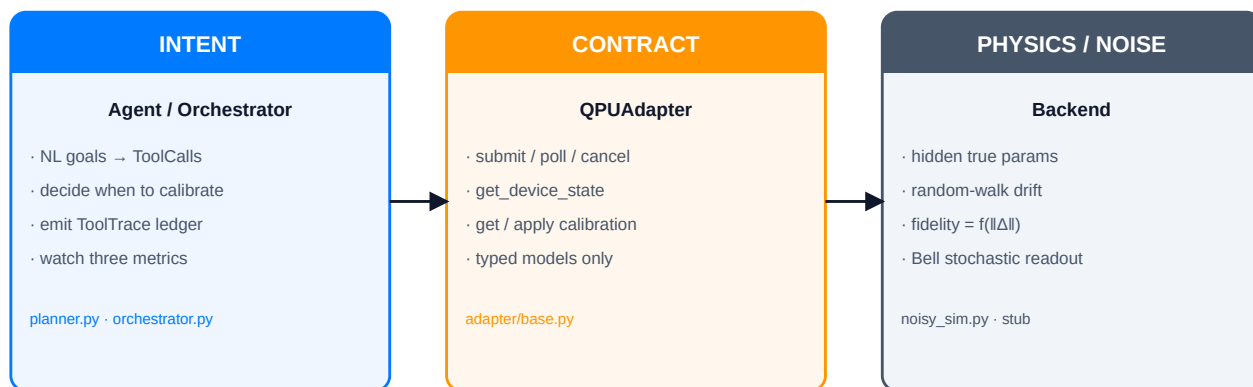


FIG. 2.1 Three concerns, one direction. Intent (agent) decides. Contract (adapter) admits only typed operations. Physics/noise (backend) is replaceable.

2.3 Intent vs device contract vs physics

Separating the three layers is the design claim.

1. **Intent** lives in the orchestrator and planner. It answers: what should we do next, given a goal? It records ToolTraces. It must not embed vendor pulse details.
2. **Device contract** is `QPUAdapter`: six methods. Anything that can sit under those methods — noisy sim today, AWG driver tomorrow — is a valid backend. The factory (`adapter/factory.py`) is the only place that chooses.
3. **Physics / noise** lives in the backend. Drift rates, fidelity kernels, Bell contrast, firmware temperatures — none of these belong in the planner. On the stub, physics is explicitly “not connected”.

This separation lets a founder say: “keep the UI and the orchestrator; swap the adapter.” Without it, every UI widget hard-codes simulator assumptions. The project then fails at the first real fridge.

QPUADAPTER - SIX-METHOD CONTRACT

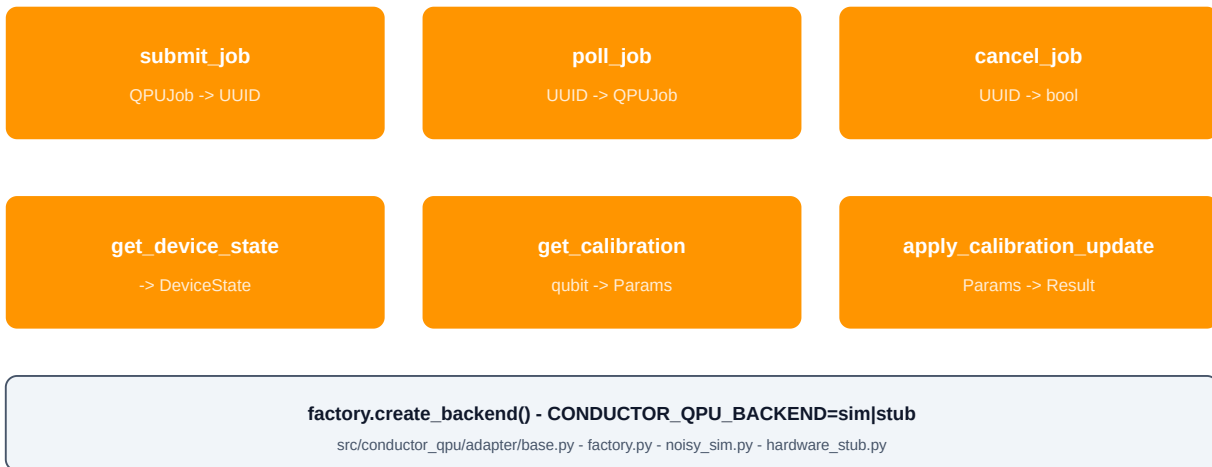


FIG. 2.2 The six-method waist. submit/poll/cancel for jobs; get_device_state for health; get/apply calibration for the loop.

2.4 Closed-loop calibration as the viability reason

Open-loop “set these params” is not enough for agents. Agents need a stop condition and a progress signal. The CalibrationService supplies both: a threshold (default 0.88) and a per-iteration fidelity history. The algorithm is gradient-free on purpose — coordinate jitter with an annealing-flavoured acceptor — because this system’s job is to exercise the *control-plane loop*, not to win a tuner bake-off. Strategy objects can replace the sampler later. The metrics and traces stay.

Each apply samples `interface_latency`. Successes accumulate `time_to_calibrated`. Attempts accumulate the success rate. Those three numbers are the dials a skeptical quantum+ML founder should watch in the demo. If they move for the wrong reasons (or never move), the loop is fake.

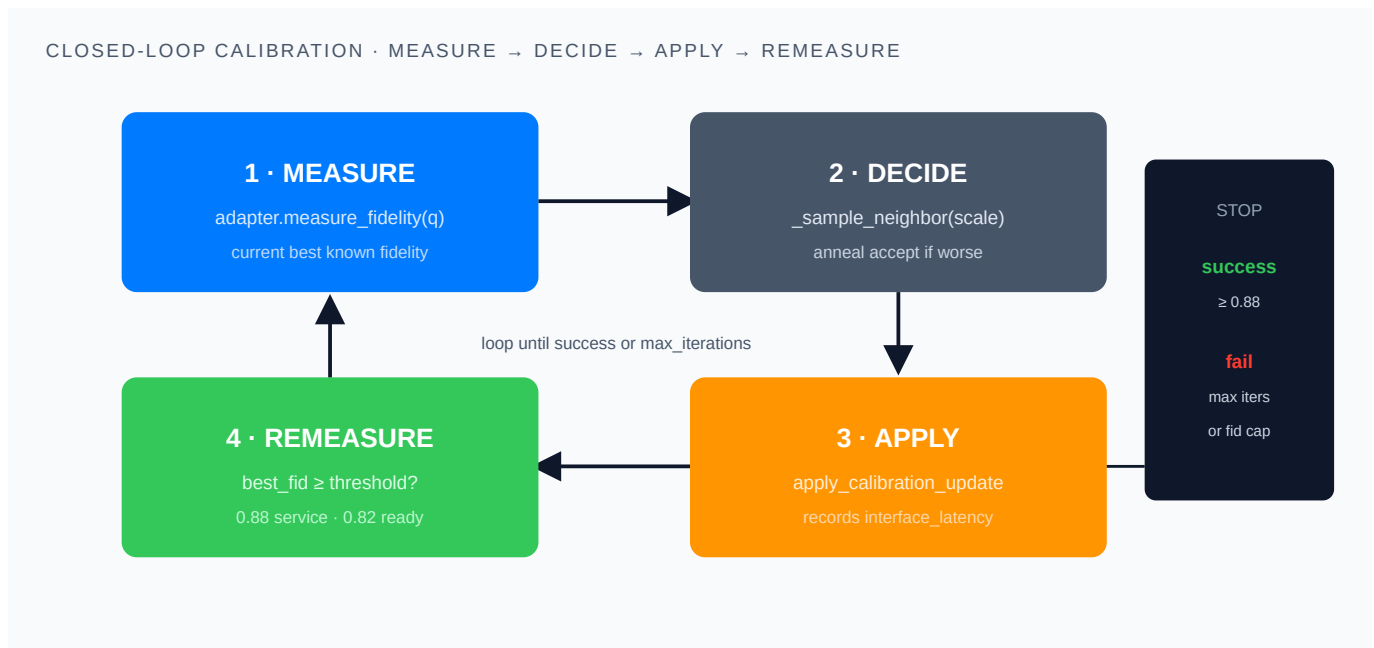


FIG. 2.3 Viability diagram: without remeasurement against an explicit threshold, calibration is not an agent tool — it is a hope.

2.5 Observability and trust surfaces

Trust in a control plane is not a green checkmark. It is a set of surfaces an operator can reconcile:

- **Fidelity history** — climb chart with threshold reference line (UI HUD + in-rail details).
- **mK / coherence / readout** — device state snapshot from `get_device_state`.
- **Param drift / detuning** — `get_detuning` exposes $|\text{applied} - \text{true}|$ on the sim (the problem calibration solves). The WebGPU surface places cyan (true) and amber (applied) markers on a $\Delta\text{freq} \times \Delta\text{amp}$ landscape.
- **Tool ledger** — ToolTrace rows: which tool, which args, latency, summary. The dock ledger is this list rendered as turns.

If any one of these is missing, agents can lie politely. With all four, a failure (fid cap, max iters, stub backend) is legible.

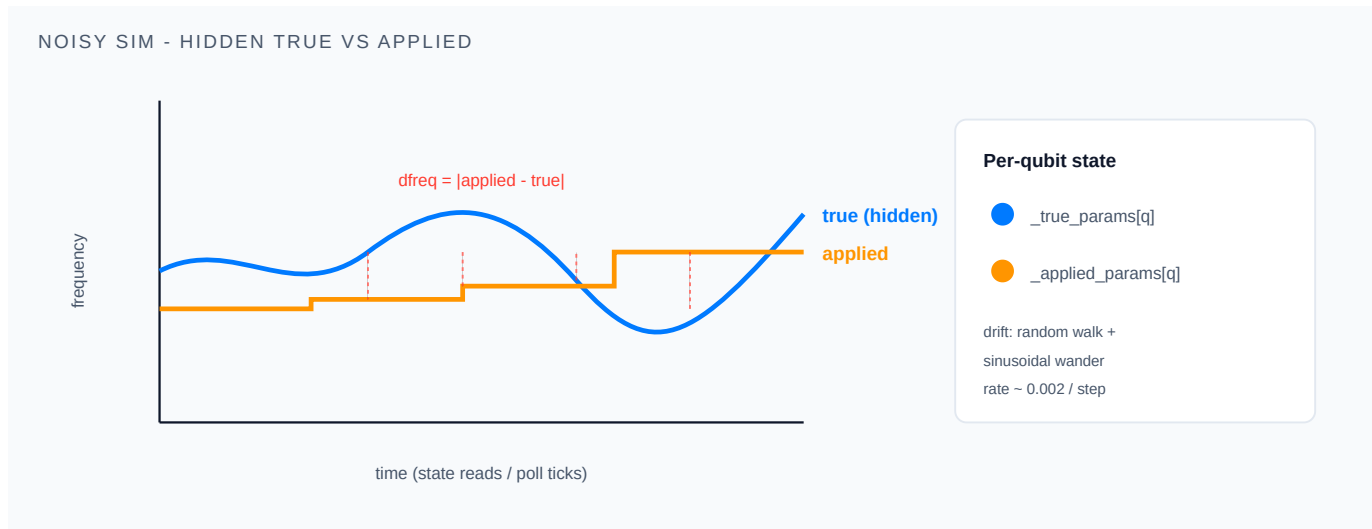


FIG. 2.4 Drift model on the noisy simulator. True params wander. Applied params step when calibration commits. Detuning is the gap.

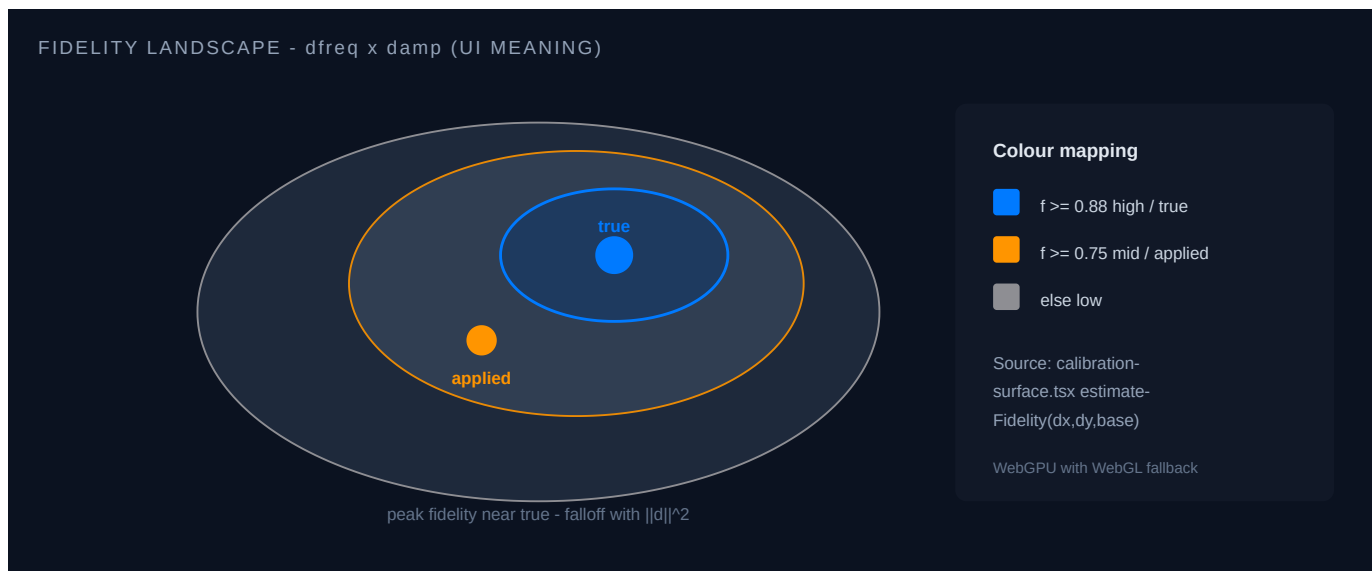


FIG. 2.5 What the instrument stage means. The landscape peaks at the hidden true target. Colour bands follow the iOS palette thresholds.

2.6 Why instrument UI (stage + dock) beats ChatGPT-only chat

Chat is a fine composer. It is a poor primary surface for a drifting physical system. Quantum ops need continuous state in the visual field: readiness, temperatures, detuning, the spatial relationship of applied vs true. A chat transcript buries those in prose and scrolls them away. The system's UI makes the stage the hero (~65% of the vertical field, full-bleed under the dock) with tabs for the WebGPU drift landscape and the 3D device panel. Language stays in a bottom-third progressive dock (~33vh) with chat bubbles, expanded suggestion chips (Calibrate Q0, Bell pair, Q0 readiness, Device status, Improve Bell, Diagnose Q0), and a high-contrast composer flush at the bottom. Frost uses a dedicated `.stage-blur` sibling with solid stacked bands (no `backdrop-filter` on the dock) so Safari keeps the upper panes sharp.

Details expand in-rail. Nothing steals the stage with a modal. ⌘K is available for power users but does not animate open (Emil Kowalski skill constraint). Toasts (Sonner) confirm terminal outcomes without becoming the narrative channel.

WHY INSTRUMENT UI BEATS CHAT-ONLY FOR QUANTUM OPS

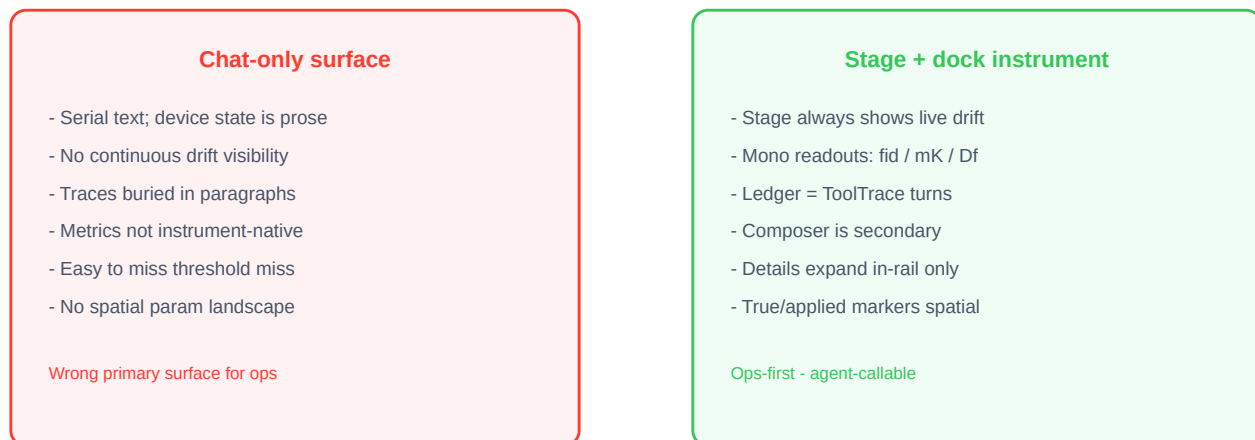


FIG. 2.6 Chat-only vs instrument. The right column is what this system ships in ui/app/page.tsx.



FIG. 2.7 Layout sketch of the instrument: frosted toolbar, stage with HUD, progressive-blur dock with ledger and composer.

2.7 Limits: sim vs hardware, toy timings, what changes on a real QPU

HONESTY

This is a control-plane demo, not a physics engine. Fidelity is a smooth function of parameter error with a wide basin so a simple search can climb. Drift is accelerated for demo feel. Interface latency on an in-process simulator is not predictive

of RPC/firmware latency. Bell “shots” are a Python loop, not ADC captures. Temperatures in mK are a theatrical function of fidelity, not a sensor.

On a real QPU, the following change — and the design claims they can change behind the adapter without rewriting the orchestrator or UI:

- **apply_calibration_update** commits a versioned cal table / AWG params and may run a verification sequence taking seconds to minutes.
- **get_device_state** reads monitor services, FPGA temps, PLL lock, link status — the stub already sketches these fields in its notes string.
- **get_detuning** comes from spectroscopy / Ramsey, not from a hidden vector the process itself owns. The sim’s omniscient detuning is a teaching tool.
- **submit_job** for circuits validates schedules, loads waveform memory, arms triggers, waits for completion.
- **Timings and success rates** become operational SLOs, not sub-second toys.
- **Persistence** — InMemoryJobStore must become Postgres (or equivalent) with retention and goal_id correlation.

RULE OF THUMB

Judge this system by whether a founder can see calibration as a recurring decision with observable cost. Judge a production port by whether the same ToolTraces and metrics still make sense when each apply takes real wall time on real hardware.

CALIBRATION FAILURE MODES



FIG. 2.8 Failure modes the interface must render honestly: demo fidelity cap, max-iteration exhaustion, and the always-cold hardware stub.

2.8 Summary claim

The interface works because it (1) lifts NL goals into a tiny tool catalog, (2) confines device diversity to a six-method adapter, (3) makes calibration a closed loop with explicit thresholds and metrics, (4) publishes trust surfaces (fidelity, mK, drift, 3D device view, ledger), and (5) uses an instrument UI that keeps physics in the visual field while language stays secondary. Everything else in this book is the engineering of that claim on main @ `b05005c`.

Sources. README.md (thesis; real vs simulated); adapter/base.py; adapter/noisy_sim.py; adapter/hardware_stub.py; adapter/factory.py; calibration/service.py; observability/metrics.py; orchestrator/*.py; api/server.py; ui/app/page.tsx; ui/components/viz/calibration-surface.tsx; ui/lib/api.ts

DIAGRAMS

How language becomes physics

Six mid-century plates. Each figure has a caption and a short explanation. A CEO or engineer can read each once. The visual language matches the cover: seafoam, olive, translucent blues, one red accent, extended sans, geometric clarity.

These plates restate the system claim without dark UI chrome. Natural language becomes typed decisions. Decisions become numbers. Numbers cross a narrow adapter firewall into replaceable physics. Fidelity and traces flow back into an instrument. The stage (physics) stays above the dock (agency).

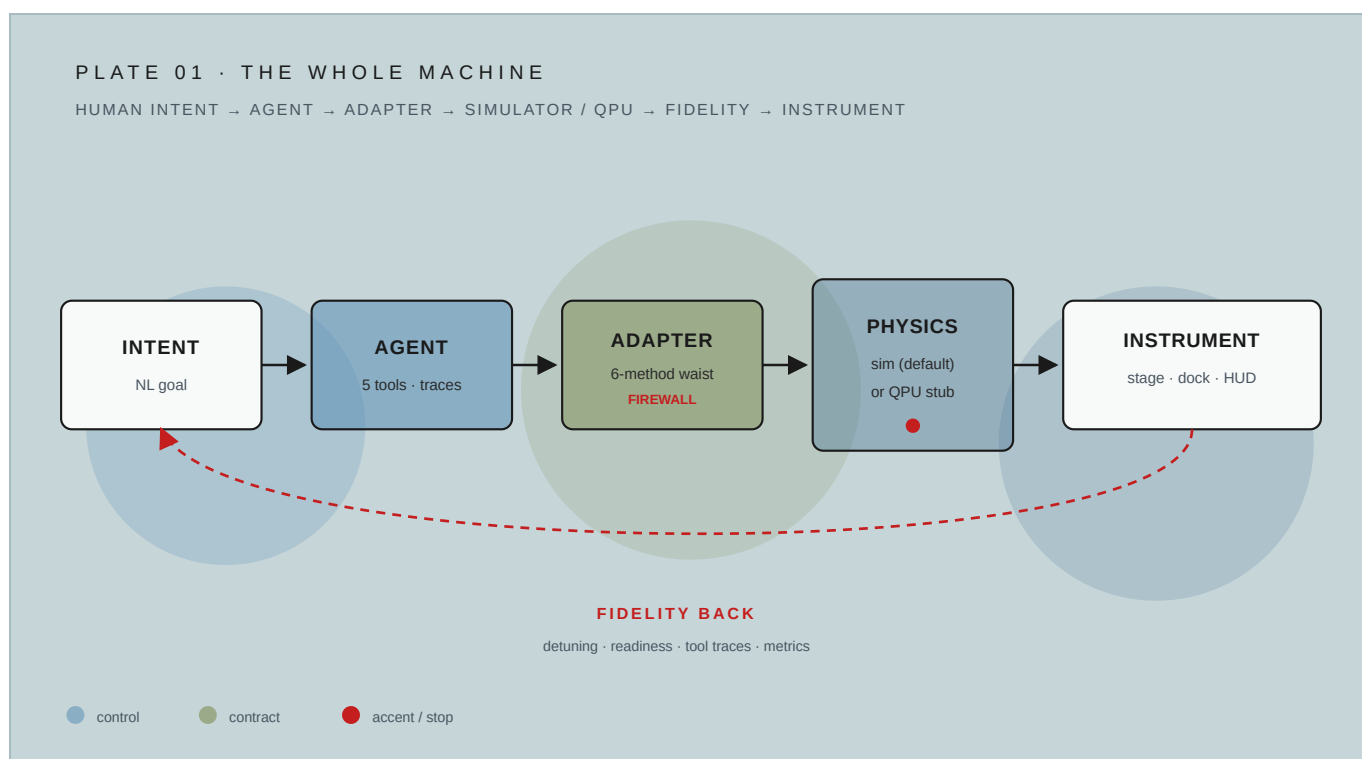
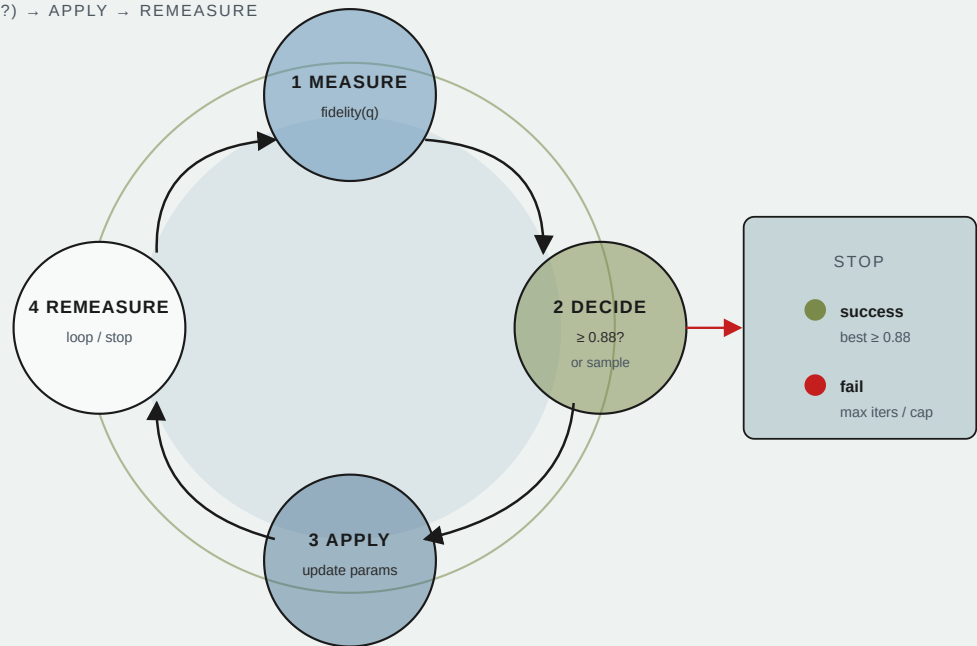


PLATE 01 The whole machine — intent to instrument, with fidelity back.

An operator states a goal. An agent maps it onto five tools and emits traces. The adapter is the waist: only six typed methods reach the backend. The backend may be the noisy simulator (default) or a hardware-shaped stub. Results — fidelity, detuning, readiness — return into the instrument UI. The human can see whether the loop worked. The dashed red arc is the product feedback path.

PLATE 02 · CALIBRATION LOOP

MEASURE → DECIDE ($\geq 0.88?$) → APPLY → REMEASURE**PLATE 02** Closed-loop calibration — measure, decide against 0.88, apply, remeasure.

Calibration is viable as an agent tool only because it is a loop. Measure fidelity. Decide whether best fidelity has crossed the service threshold (0.88) or whether to sample a neighbor. Apply an update through the adapter (this samples interface latency). Remeasure. Exit on success, exhausted iterations, or a demo fidelity cap. Hardened calibration (PR #2) widens the sim kernel and strengthens early exploration. Founder-demo seeds converge. The adapter seam does not change.

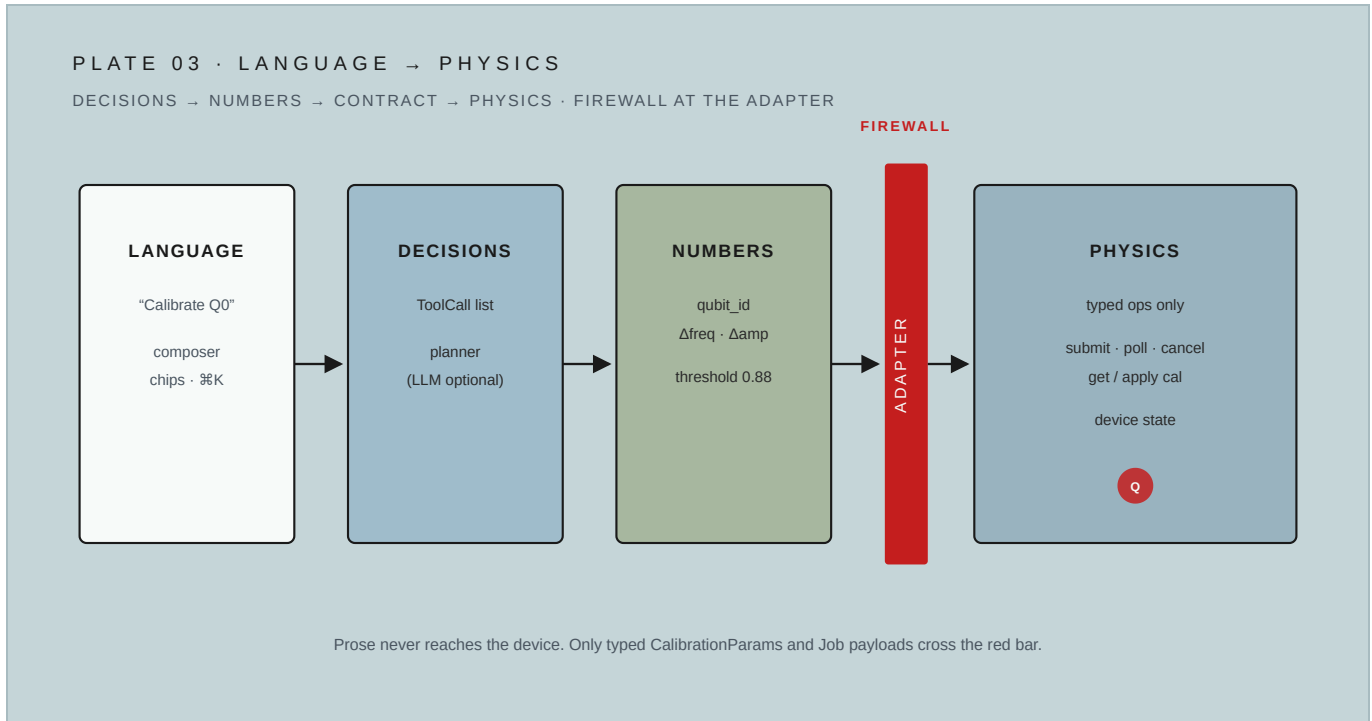
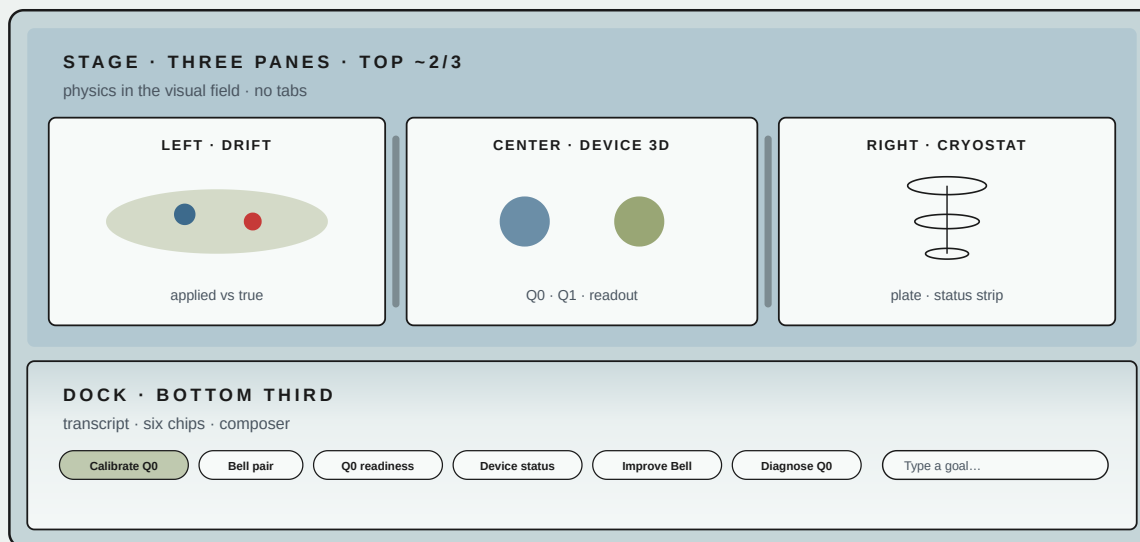


PLATE 03 Language → decisions → numbers → contract → physics. Firewall at the adapter.

Prose stops at the red bar. The planner (deterministic by default; optional LLM behind `CONDUCTOR_ENABLE_LLM`) turns a goal into ToolCalls. Tools produce numbers — qubit ids, parameter deltas, thresholds. Only those typed payloads cross the QPUAdapter. That firewall lets backends swap without rewriting the agent. It also stops the agent from inventing illegal pulse calls.

PLATE 04 · STAGE VS DOCK

PHYSICS IN THE VISUAL FIELD · AGENCY IN THE LEDGER



Left drift · center device 3D · right cryostat. Dock holds agency under physics.

PLATE 04 Stage holds physics; dock holds agency. On current main the stage is three panes — drift | device 3D | cryostat — with two resizable splitters. No tabs.

Chat is a fine composer. It is a poor primary surface for a drifting device. The instrument keeps continuous state in the visual field. The stage shows three panes at once. Left: a high-contrast drift grid, coplanar with the fidelity surface, with true/applied markers. Center: the 3D device (qubits, chiplets, resonators) with a readout card for fidelity, T1, T2, and mK. Right: the isometric dilution-fridge plate with a status strip and the state chip. Two iPadOS Split View–style separators, with capsule handles on hover, resize the panes. There is no glass inset and no tab flip. PR #2 first used tabs labeled drift | device. That control is past. The device and cryostat panes are UI. They are not a fridge sensor. The dock over the bottom-third frost holds the transcript, workflow chips, and composer — agency under physics, not instead of it. See FIG. A.0 for the live screenshot.

PLATE 05 · METRICS SCOREBOARD

TIME-TO-CAL · SUCCESS RATE · INTERFACE LATENCY

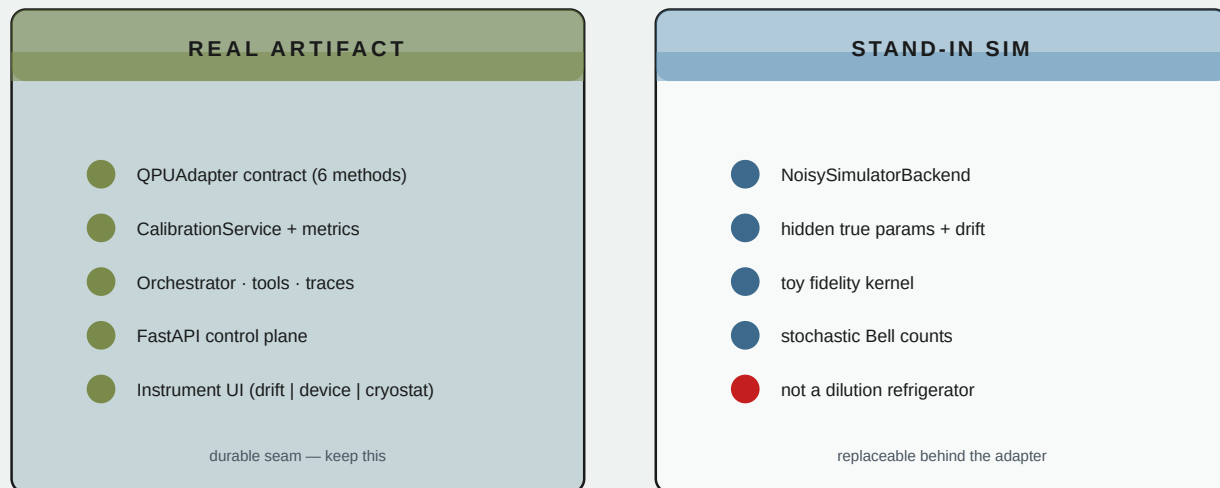


PLATE 05 The three founder-facing dials: time-to-cal, success rate, interface latency.

If calibration is a product, its cost must be observable. *Time-to-calibrated* is wall seconds from loop start to best fidelity $\geq$ threshold (successes only). *Calibration success rate* is successes over attempts. *Interface latency* is the seconds per `apply_calibration_update` roundtrip. `CalibrationService` owns the aggregates. The HTTP `/metrics` surface rolls percentiles for operators and demos.

PLATE 06 · REAL ARTIFACT VS STAND-IN

WHAT SHIPS AS SOFTWARE · WHAT STANDS IN FOR THE FRIDGE

**PLATE 06** Real software artifact versus the stand-in noisy simulator.

Keep the left column. The adapter contract, calibration loop, orchestrator traces, FastAPI plane, and instrument UI (including the 3D device panel) are the durable artifact. The right column is honest scaffolding: a toy 1–2 qubit model with hidden true parameters, random-walk drift, and a climbable fidelity kernel — not a fridge, not an AWG, not a pulse compiler. A shaped stub documents the driver shape and refuses real work. Replace the stand-in behind the same six methods.

Sources. `adapter/base.py`; `adapter/noisy_sim.py`; `calibration/service.py`; `orchestrator/*`; `api/server.py`; `ui/app/page.tsx`; `ui/components/viz/calibration-surface.tsx`; `ui/components/viz/Device3D*.tsx` (PR #2); cover palette from `assets/cover-bloch.svg`. Part 1 architecture plates: `gen_qpu_plates.py`.

System architecture

The system is a FastAPI control plane process plus a Next.js instrument client. Between them: REST and SSE. Under the API sit the orchestrator, calibration service, job store, and a backend. An environment variable selects the backend.

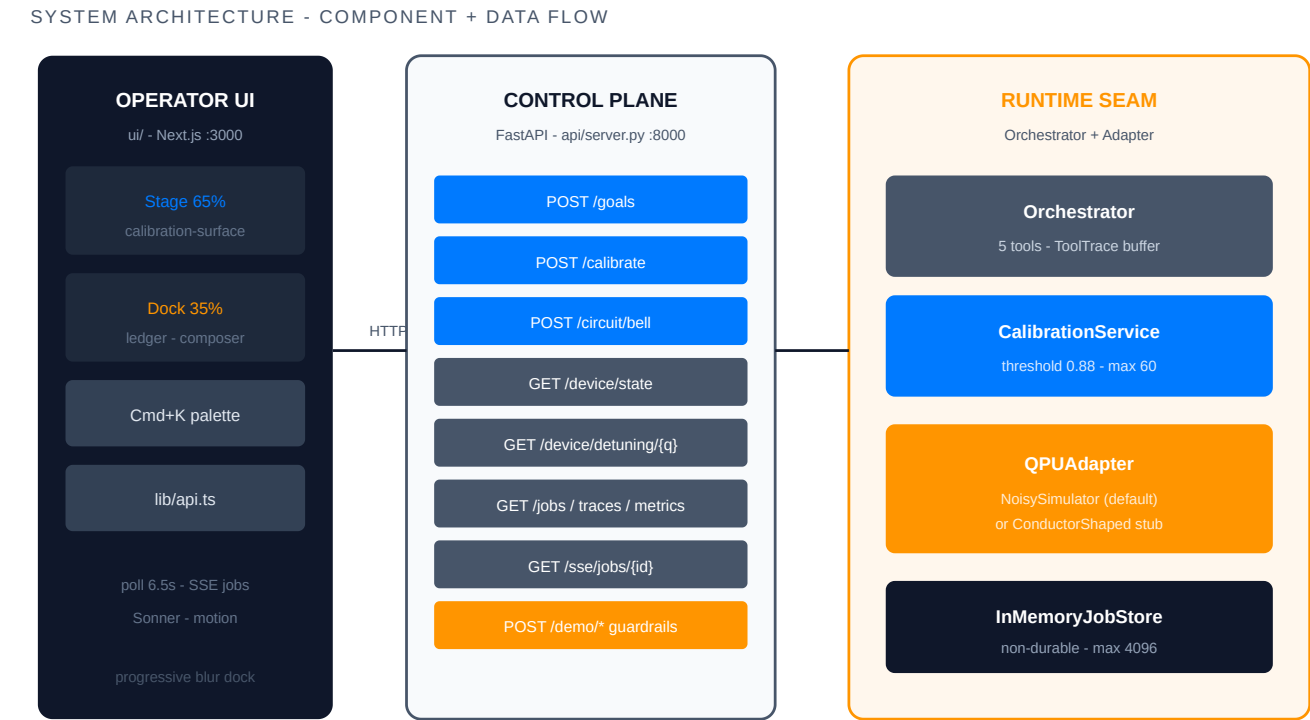


FIG. 3.1 Component diagram. UI on :3000; API on :8000; runtime seam holds Orchestrator, CalibrationService, QPUAdapter, InMemoryJobStore.

PACKAGE LAYOUT - src/conductor_qpu

```

conductor_qpu/
adapter/ QPUAdapter + sim + stub + factory
api/ FastAPI server (CORS, SSE, demos)
calibration/ CalibrationService + metrics
jobs/ InMemoryJobStore
models/ types.py (Job*, DeviceState, Params)
observability/ MetricsAggregator
orchestrator/ Orchestrator + planner
ui/ legacy Python UI entry (optional)

ui/ Next.js instrument (separate tree)
tests/ pytest contract + convergence
demo_scripts/ founder / cal / circuit demos

```

FIG. 3.2 Package layout under src/conductor_qpu and the sibling ui/ tree.

GOAL TO TOOLS TO ADAPTER TO DEVICE STATE

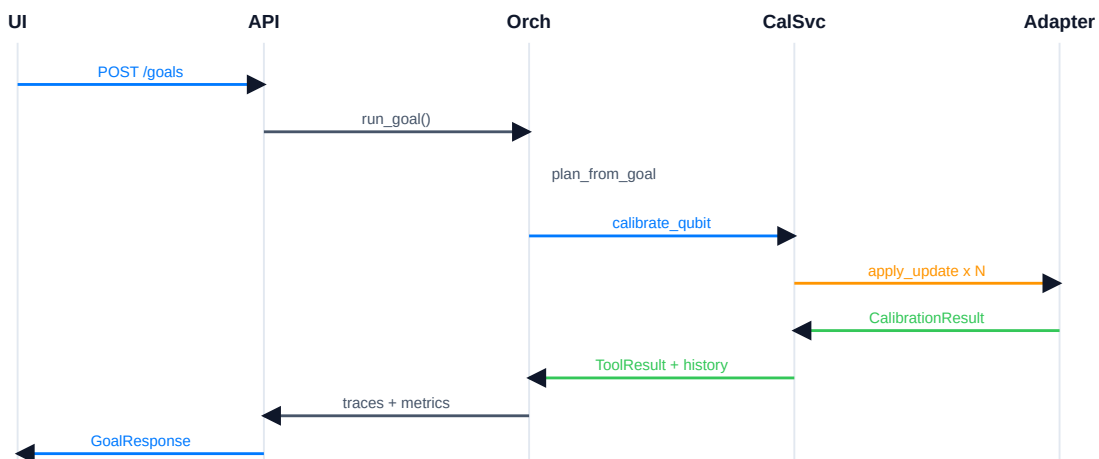


FIG. 3.3 Request path for a calibration goal from UI through API, orchestrator, calibration service, and adapter.

3.1 Process topology

Two processes in development:

1. `python -m conductor_qpu.api` (Makefile `run-api`) — uvicorn serving `conductor_qpu.api.server:app` on `0.0.0.0:8000`.
2. `cd ui && npm run dev` (Makefile `run-ui-dev`) — Next.js on `:3000`. CORS on the API allows localhost:3000.

Python demos (`make demo`, `make founder-demo`) construct adapter + calibration + orchestrator in-process and need no HTTP server.

3.2 Object graph at API startup

Module import of `api/server.py` builds singletons:

```
_backend      = create_backend()          # env CONDUCTOR_QPU_BACKEND
_calibration  = CalibrationService(
    adapter=_backend,
    fidelity_threshold=0.88,
    max_iterations=60)
_job_store    = InMemoryJobStore()
_orchestrator = Orchestrator(
    adapter=_backend,
    calibration=_calibration,
    job_store=_job_store)
_metrics      = MetricsAggregator()
```

There is no dependency-injection container. Tests construct their own graphs. The UI never imports Python — it speaks HTTP via `ui/lib/api.ts`.

3.3 Data stores

All state is in-memory for this prototype:

- Adapter job dict (`NoisySimulatorBackend._job_store` or stub equivalent).
- `InMemoryJobStore` (bounded to 4096) — present for the seam; the API's `/jobs` list currently reads the backend's `list_recent_jobs`.
- Orchestrator `_last_traces` buffer — last goal's traces.
- CalibrationMetrics counters on the service instance.
- MetricsAggregator dequeues (latency ms, fidelities) with maxlen 512.

HONESTY

Nothing survives process restart. The README's week-1 plan calls out Postgres (or equivalent) with retention and `goal_id` correlation as the persistence swap.

3.4 Backend selection

CONDUCTOR_QPU_BACKEND	CLASS	BEHAVIOUR
sim (default)	NoisySimulatorBackend	2-qubit drifting toy; full demo path
stub hardware shaped conductor	ConductorShapedAdapter	Shape-only; circuits fail; fidelity 0.55; never ready

Table 3.1. Factory mapping in `adapter/factory.py`.

Sources. `api/server.py`; `adapter/factory.py`; `jobs/store.py`; `ui/lib/api.ts`

Control plane / API

This part lists every HTTP endpoint from `src/conductor_qpu/api/server.py`. This book describes main `@b05005c`. PR #13–#18 change stage framing only. They do not revise this route list. Schemas are Pydantic v2 models or ad-hoc dicts as noted. Errors use FastAPI `HTTPException`.

4.1 Cross-cutting

- **CORS** — `allow_origins` includes `http://localhost:3000` and `:8000` (and `127.0.0.1` variants); credentials allowed; methods/headers *.
- **OpenAPI** — `docs_url="/docs"`, title field still reads the package-era string “Conductor QPU”, version 0.1.0 (the product name is Coda 2).
- **Auth** — none. Local prototype only.

4.2 Health and readiness predicate

GET `/health`

Returns `{status: ok, service: conductor-qpu}`. Used by the UI connection check.

GET `/readiness_predicate`

Returns the exact predicate name `all_qubits_readout_fidelity_above`, threshold 0.82, and description string. Founders can point at numbers, not vague claims.

4.3 Goals

POST `/goals`

Body `GoalRequest { goal: str }`. Runs `_orchestrator.run_goal`, records latencies into `MetricsAggregator`, returns `GoalResponse { goal, results[], traces[], metrics }`. Each result is `{ok, data, latency_s, error}`. Traces are the structured `ToolTrace` dicts for this execution.

4.4 Calibration and circuits (direct)

POST `/calibrate`

Body `{ qubit_id?: int=0, target_fidelity?: float }`. Calls `_calibration.calibrate` directly (bypasses planner, still uses the service). Records fidelity in aggregator. Returns success, fidelity, iterations, `duration_s`, params subset, history.

POST /circuit/bell

Body { shots?: int=1024, qubits?: [0,1] }. Builds a CIRCUIT QPUJob, submits to backend, polls once, returns job_id, counts, metrics — or error.

4.5 Device

GET /device/state

Snapshot: is_ready, readiness_score, qubits, temperatures_mk, coherence_us, readout_fidelity, notes, timestamp, readiness_predicate object.

GET /device/detuning/{qubit_id}

Returns detuning dict (frequency_error, amplitude_error, phase_error, readout_error_delta) plus applied params. 404 if unknown qubit. On the stub this raises NotImplementedError → 500 unless caught; the sim implements it.

4.6 Jobs, traces, metrics

GET /jobs

Query limit (default 50). Lists recent jobs from backend list_recent_jobs. Documented non-durable.

GET /jobs/{job_id}

400 invalid UUID; 404 unknown. Returns status, type, timestamps, result data, metrics, error.

GET /traces

Most recent orchestrator tool traces (get_last_traces()).

GET /metrics

Union of orchestrator.get_metrics(), MetricsAggregator.snapshot(), and calibration.metrics.to_dict().

4.7 SSE

GET /sse/jobs/{job_id}

Server-Sent Events. Polls backend every 200 ms, emits event: update when status changes, stops on terminal status, or event: timeout after ~24 s (120 iterations). event: error if job missing.

4.8 Demo / guardrail endpoints

POST /demo/fail_next_cal

If backend has set_demo_fid_cap, sets cap to 0.69 so subsequent calibration cannot reach 0.88. Returns {ok, fid_cap, note}.

POST /demo/start_long_job

Starts a long-running diagnostic that stays RUNNING until cancel (sim helper). Returns `{job_id}`. Used to demo cancel in the UI.

4.9 Error table

SITUATION	STATUS	DETAIL
Malformed job UUID	400	invalid job_id
Unknown job	404	job not found
Unknown qubit (detuning)	404	ValueError text
Backend down (UI)	—	UI sets backendDown; toast; banner “make run-api”
Calibration capped	200 with success:false	Not an HTTP error — domain failure in body
Stub circuit	job FAILED	error explains hardware not connected

Table 4.1. Error and failure signalling. Domain failures are often 200 + ok:false / success:false so agents can read them as tool results.

Sources. `api/server.py` (all routes); `ui/lib/api.ts` (client mapping)

PART 5

Adapter & noisy simulator

The adapter is the narrow waist. This part documents the abstract contract, the noisy simulator's dynamics, and the hardware-shaped stub.

QPUADAPTER - SIX-METHOD CONTRACT

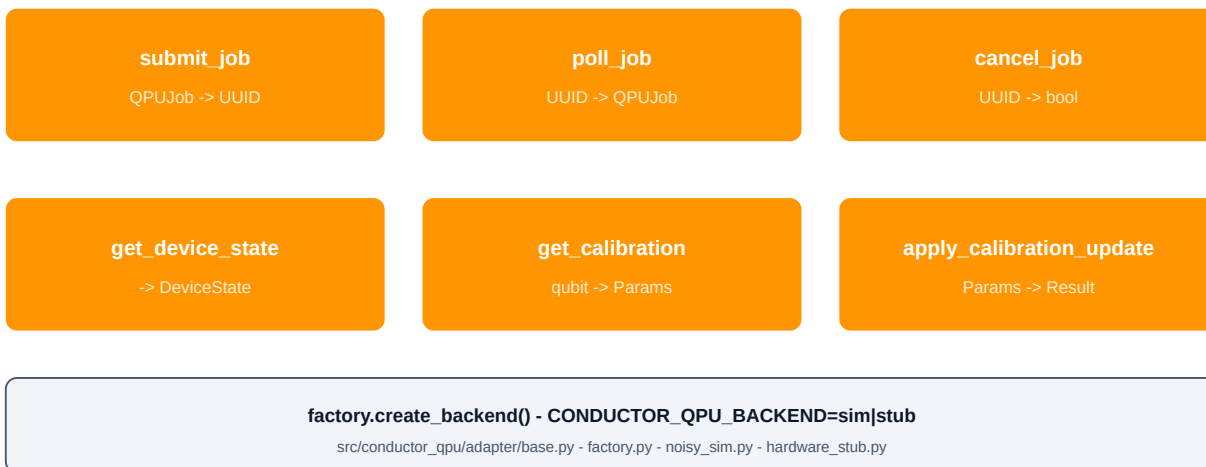


FIG. 5.1 Contract reminder.

5.1 QPUAdapter (base.py)

Abstract methods: `submit_job`, `poll_job`, `cancel_job`, `get_device_state`, `get_calibration`, `apply_calibration_update`. Synchronous by prototype choice; production ports may go async without changing the orchestrator's conceptual model if a thin sync wrapper remains.

5.2 CalibrationParams and DeviceState

FIELD	ROLE
<code>qubit_id</code>	Which qubit the params apply to
<code>frequency</code>	Fake GHz-scale drive frequency
<code>amplitude</code>	Drive amplitude
<code>phase</code>	Radians
<code>t1, t2</code>	Relaxation / dephasing (μ s, theatrical)
<code>readout_error</code>	Bit-flip probability on measurement

Table 5.1. CalibrationParams fields (models/types.py). Units are illustrative; only relative distance to true params matters on the sim.

`DeviceState` carries temperatures_mk, coherence_us (T1,T2) per qubit, readout_fidelity, is_ready, notes.
`readiness_score()` averages readout fidelities into 0..1.

5.3 NoisySimulatorBackend

Construction: `num_qubits` clamped to 1..2, seed (default 42), drift_rate (default 0.002). Maintains `_true_params` and `_applied_params` per qubit, an RLock, and an in-memory job store.

Drift

`_apply_drift` runs on state reads / polls when ≥ 50 ms elapsed. True params take a Gaussian step scaled by drift_rate and a sinusoidal term $\sin(\text{now} * 0.7 + q)$, with clamps per field. This is the lab dynamic the UI visualises: calibration chases a moving target.

Fidelity kernel

`_fidelity_from_params` builds a squared-error penalty across frequency, amplitude, phase, T1/T2 shortfalls, and readout error, then maps through $0.94 * \exp(-1.8 * \text{err}) + 0.05$, clamped to [0.55, 0.999]. Optional `_demo_fid_cap` can further ceiling the value for failure demos.

Bell simulation

`_simulate_bell_readout` draws ideal 00/11, applies readout flips, and depolarises by contrast derived from joint fidelity. Returns counts dict.

Job handling

Calibration jobs run a small local loop (threshold 0.88, max 25) when submitted directly; the preferred path is the CalibrationService via the orchestrator (full metrics + traces). Circuit jobs run Bell (or stub success for unknown circuit names). Cancel flips non-terminal jobs to CANCELLED.

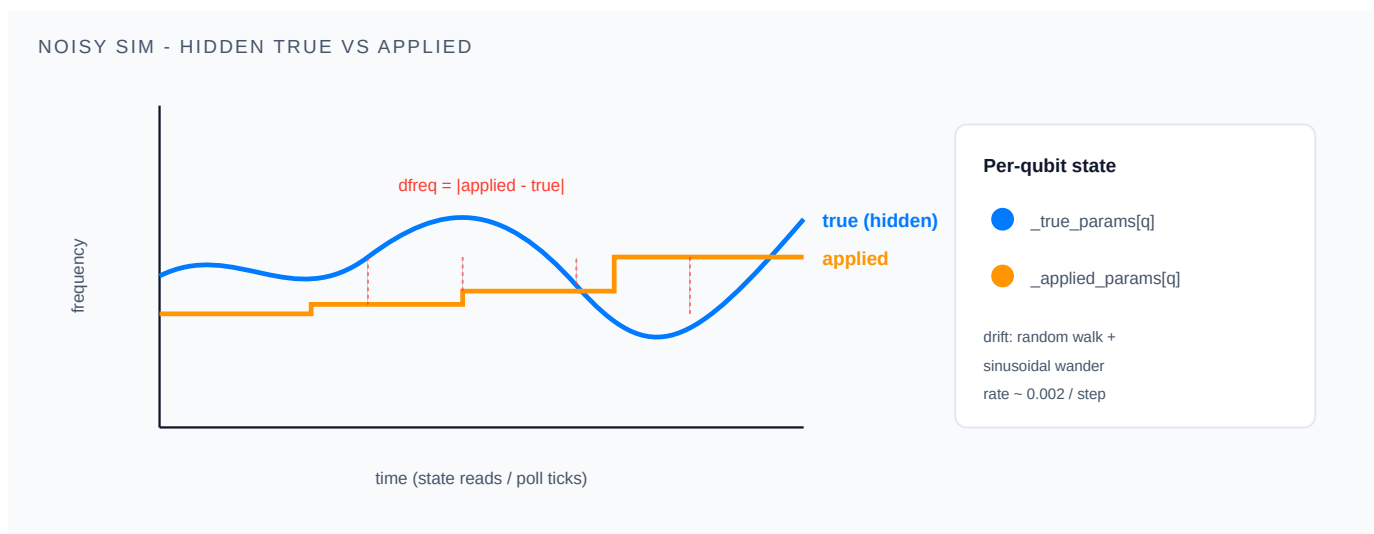


FIG. 5.2 True vs applied over time on the simulator.

5.4 ConductorShapedAdapter

Documents what a real AWG/DAC/FPGA driver would look like. Circuits fail with an explicit `NotImplemented` error string. Calibration “succeeds” at fidelity 0.90 but notes that nothing was committed. `is_ready` is always `False`. `measure_fidelity` returns 0.55. `get_detuning` raises `NotImplementedError`. Use `CONDUCTOR_QPU_BACKEND=stub` and `make_founder-demo-stub` to exercise the seam.

Sources. `adapter/base.py`; `adapter/noisy_sim.py`; `adapter/hardware_stub.py`; `adapter/factory.py`; `models/types.py`; `tests/test_adapter_contract.py`; `tests/test_adapter_selection.py`

Calibration & metrics

CalibrationService is an iterative, gradient-free tuner that records the three founder metrics. This part specifies the algorithm, thresholds, failure modes, and what tests assert.

THE THREE METRICS

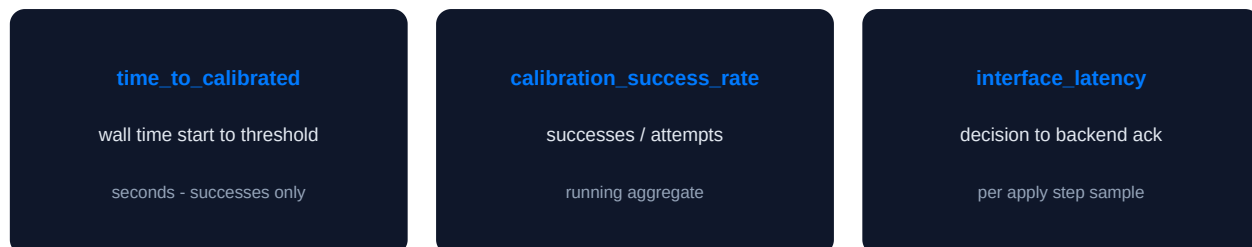


FIG. 6.1 Metric definitions again — they are the product dials.

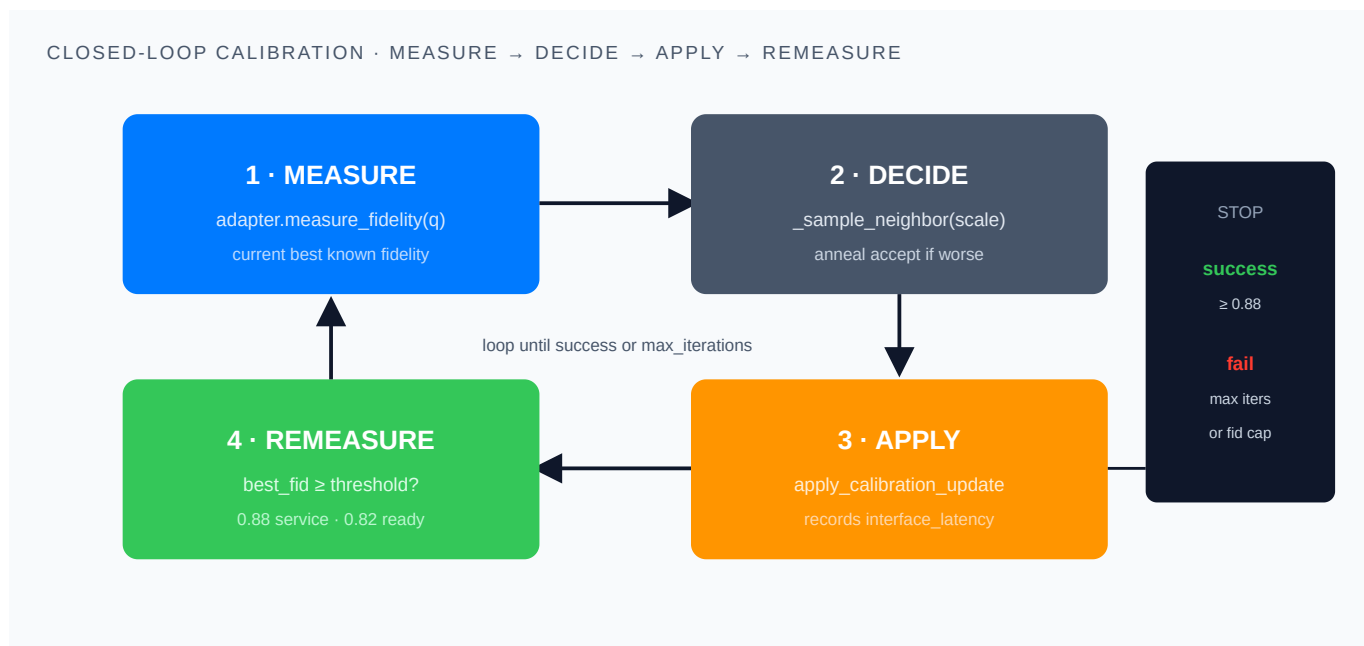


FIG. 6.2 Loop structure as implemented in calibration/service.py.

6.1 Algorithm

1. Read current params; measure initial fidelity; history starts at (0, fid).
2. For iteration 1..max_iterations:
 - Compute temperature schedule `temp = max(0.015, 0.55*(1-progress))`.
 - Choose exploration scale (wide early; widen further if patience half-exhausted).

- Sample neighbour via Gaussian jitter on frequency, amplitude, phase, t1, t2, readout_error with field clamps.
- Call `adapter.apply_calibration_update(candidate)` ; record interface latency.
- Accept if better than best; else accept worse with probability `exp((f-best)/temp)` .
- Update best/no_improve; append history; optional on_step callback.
- Break if `best_fid ≥ threshold`.
- If `no_improve ≥ patience`: restart around best, sometimes with a wide redraw.

3. Update CalibrationMetrics attempts/successes/time.

6.2 Defaults (API process)

PARAMETER	API DEFAULT	NOTES
fidelity_threshold	0.88	Planner default target matches
max_iterations	60	Tests may use 30–80
patience	12	Service default
seed	123	Service default RNG
READINESS threshold	0.82	Separate predicate on DeviceState

Table 6.1. Thresholds and loop limits.

6.3 Failure modes

CALIBRATION FAILURE MODES

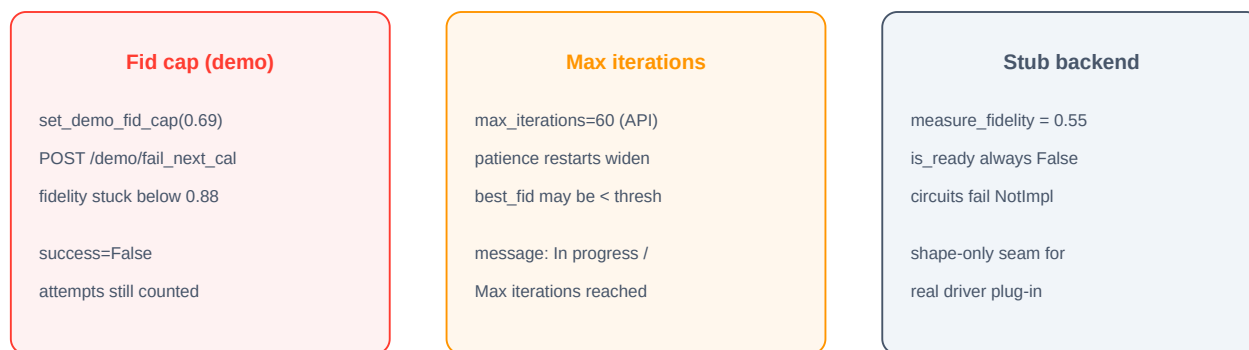


FIG. 6.3 Three classes of failure the demos and tests cover.

- **Fidelity cap** — `POST /demo/fail_next_cal` or `set_demo_fid_cap` . Service reports `success=False`; fidelity stays below 0.88; attempts still increment.
- **Max iterations / patience cycling** — if the basin cannot be climbed in time (or cap intervenes), final message reflects non-convergence.
- **Stub backend** — `measure_fidelity 0.55` permanently; useful to prove UI/orchestrator don't assume sim omniscience.

6.4 Test and demo numbers

`tests/test_calibration_converges.py` asserts fidelity can reach ≥ 0.80 on the noisy sim with threshold 0.86, and that metrics fields are populated across multi-qubit attempts. `tests/test_failure_paths.py` asserts capped runs end with `success=False` and that long-running jobs cancel to CANCELLED. Demos print the three metrics to stdout; they do not claim statistical significance across seeds.

HONESTY

Historical flakiness: early in development, fidelity could stick near the 0.55 floor while the threshold sat at 0.88 — the wide-basin kernel and exploration schedule exist specifically so the founder demo climbs reliably without becoming a fake always-success path. Caps remain to demonstrate honest failure.

Sources. `calibration/service.py`; `observability/metrics.py`; `tests/test_calibration_converges.py`; `tests/test_failure_paths.py`; `demo_scripts/run_calibration_demo.py`; `demo_scripts/founder_demo.py`

Orchestrator & tools

The orchestrator is a thin agent runtime. It registers tools, calls tools, runs goals through a planner, and exposes traces and metrics. No LLM is required on the default path.

ORCHESTRATOR TOOL CATALOG (5)

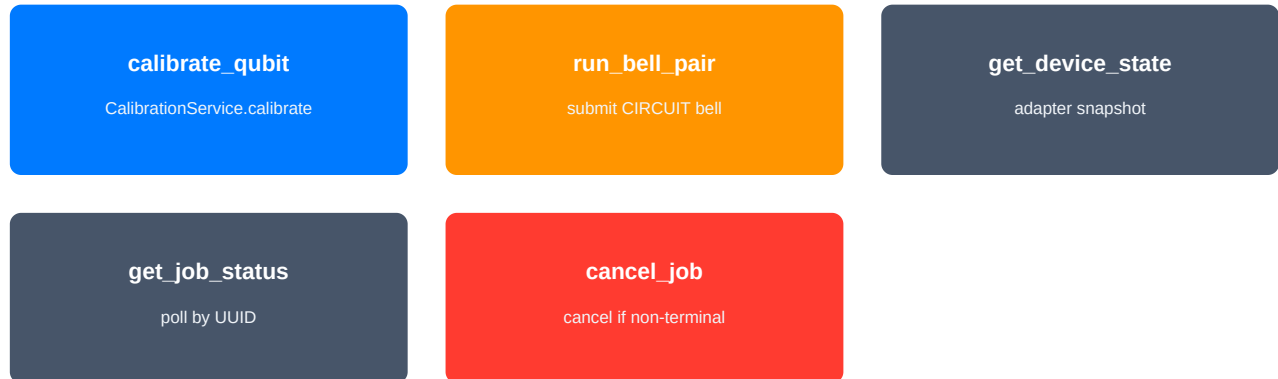


FIG. 7.1 Tool catalog.

7.1 ToolCall, ToolResult, ToolTrace

TYPE	FIELDS	ROLE
ToolCall	tool, args	Planner output step
ToolResult	ok, data, latency_s, error?	Tool return value
ToolTrace	timestamp, tool, args, latency_s, ok, summary	UI timeline / ledger observability

Table 7.1. Orchestrator dataclasses (orchestrator/orchestrator.py).

Summaries are derived: fidelity=..., 00/11 counts, ready=... score=..., or first keys. Traces reset at the start of each `run_goal`.

7.2 Planner rules

`plan_from_goal` normalises whitespace/case and matches keywords:

INTENT KEYWORDS	TOOL PLAN
bring, calibrat, ready, tune	calibrate_qubit(qubit_id parsed 0 1, target 0.88 or 0.92 if high/strict)
bell, circuit, entangl, pair	run_bell_pair(shots 1024 256 few/quick 4096 many/precise)
state, health, status, ready?, temperature	get_device_state
cancel ...	cancel_job(job_id token)
default (no match)	get_device_state then calibrate_qubit(0, 0.85)

Table 7.2. Deterministic planner mapping (planner.py).

`plan()` tries `maybe_llm_plan` first when `CONDUCTOR_ENABLE_LLM` is truthy and `openai` imports; otherwise `plan_from_goal`. LLM failures return `None` and fall through. The API's `POST /goals` uses `Orchestrator.run_goal`, which imports `plan_from_goal` as its default planner function (note: the server also imports `plan` but `run_goal` defaults to `plan_from_goal` unless a planner callable is passed).

7.3 Early stop

If a step's tool is `calibrate_qubit` or `run_bell_pair` and `ok` is `False`, `run_goal` breaks the plan early. Visibility tools do not abort the sequence.

7.4 Metrics surface from orchestrator

`get_metrics()` returns calibration metrics dict, per-tool call counts and avg latency, optional adapter `get_stats()` (`total_jobs`, `total_shots` on the sim), and an `interface_latency_p50_hint` aliased from avg interface latency.

Sources. `orchestrator/orchestrator.py`; `orchestrator/planner.py`

PART 8

Operator UI

The UI is a Next.js App Router instrument panel. It is not a chat app with a chart widget. The stage is the product: three panes — **drift** | **device 3D** | **cryostat**. The dock is the agent surface.

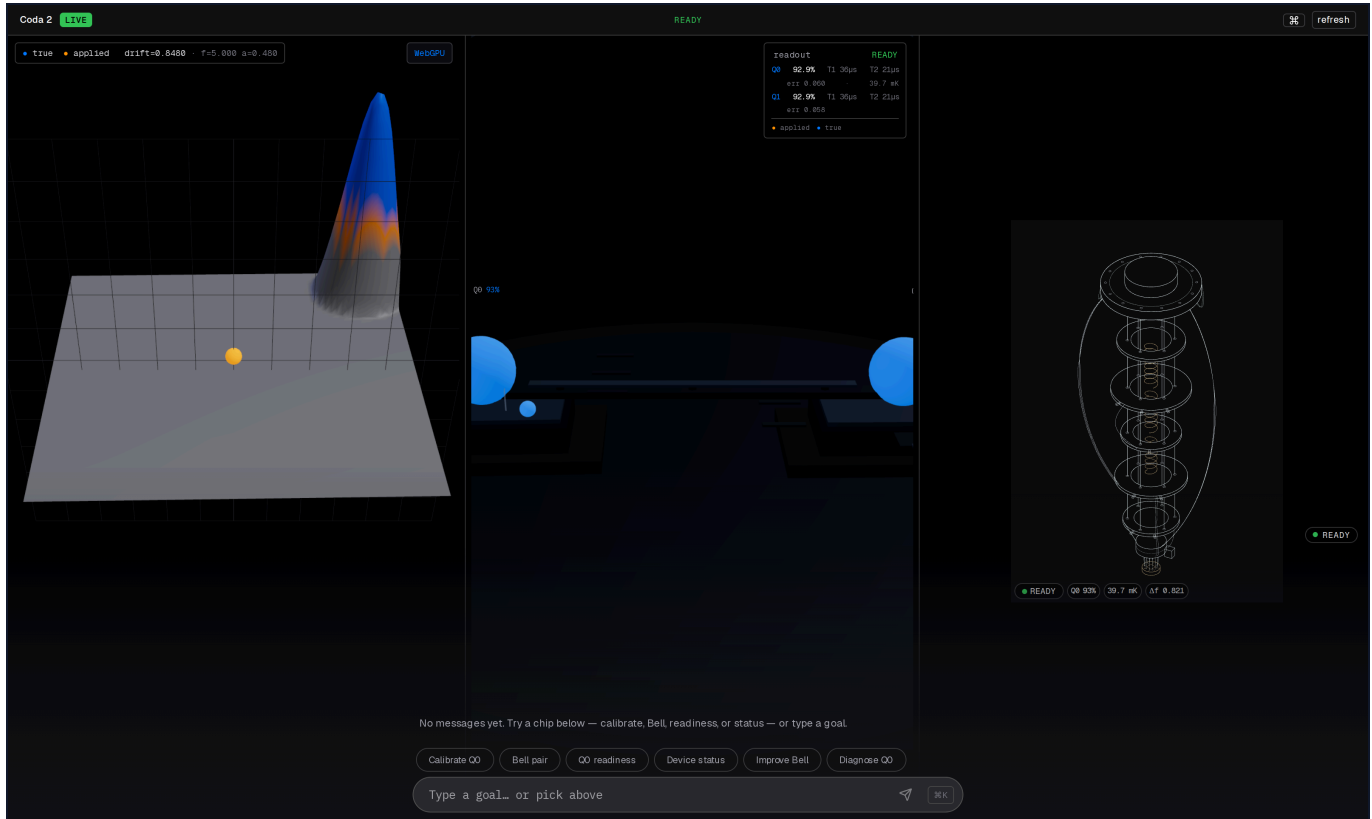


FIG. 8.1 Screenshot: the three-pane instrument at rest. Left: parameter-drift landscape (orange applied, blue true). Center: 3D device with qubit spheres and the readout card. Right: isometric dilution-fridge plate with its status strip and the StageStateChip. Bottom: dock with empty-state hint, six workflow chips, and the composer.

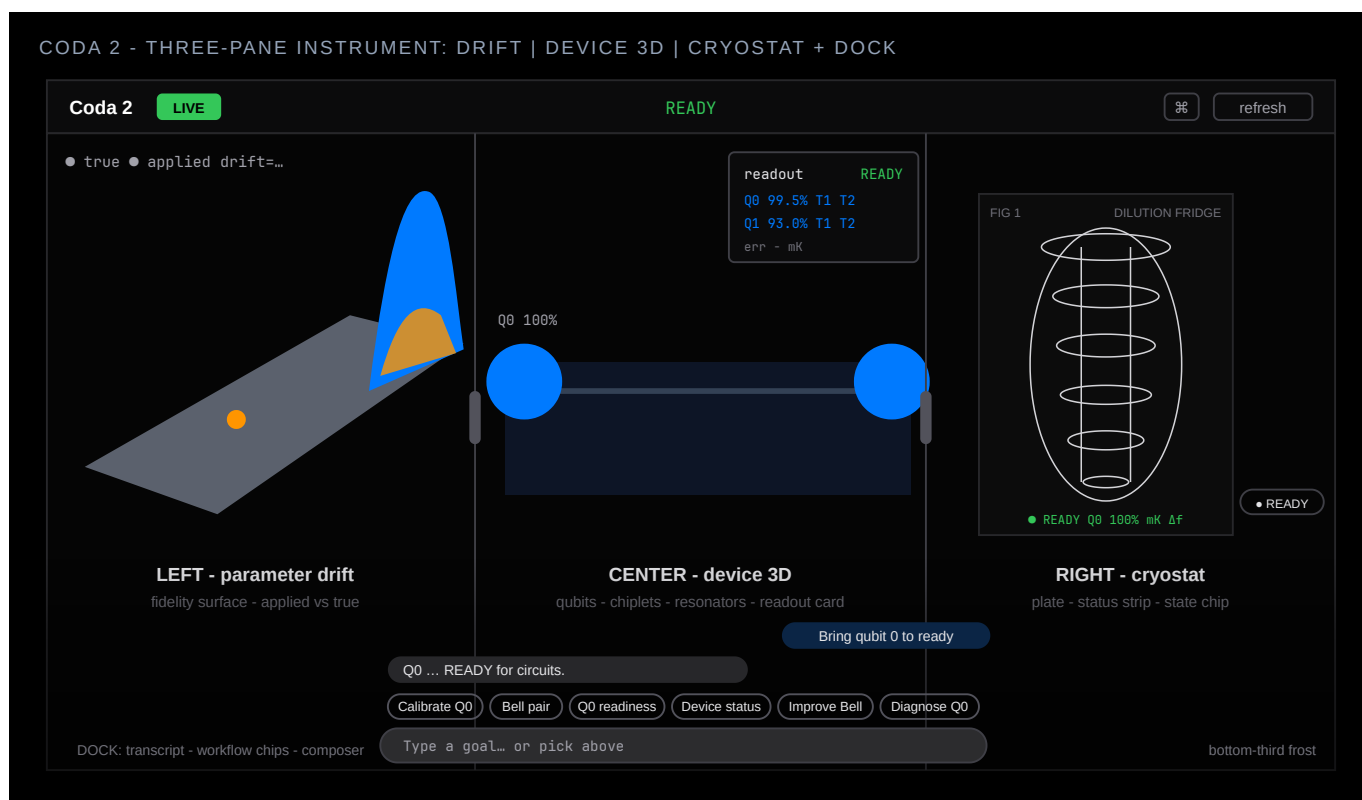


FIG. 8.2 Schematic of the same chrome. Toolbar on top. Three panes separated by two splitters. Dock over the bottom-third frost.

Current UI (`main` , three-pane chrome through PR #25–#27): three panes at rest. Left is a high-contrast drift grid, coplanar with the fidelity surface. Center is Device3D: qubit spheres, chiplets, resonators, and a readout card (Q0/Q1 fidelity, T1, T2, err, mK). Right is the isometric cryostat plate with a status strip (state · Q0 % · mK · Δf). Two iPadOS Split View–style separators resize the panes; hover shows a capsule drag handle; each pane keeps a minimum width (roughly 22–28% of the stage). No traffic lights. No glass inset. No tabs. Earlier releases shipped a single drift stage (65/35) and then a two-pane drift | device split; both are historical.

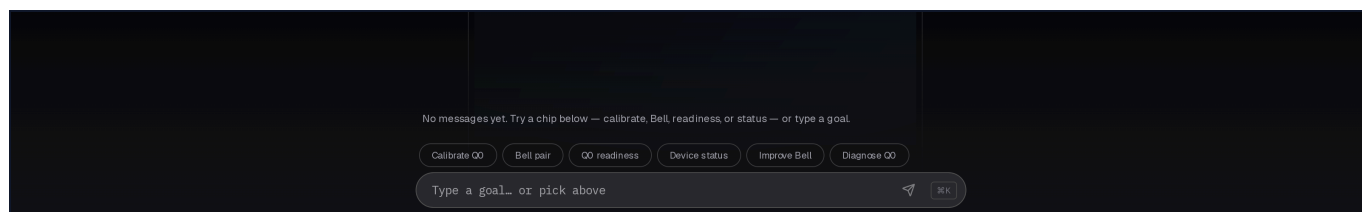


FIG. 8.3 Screenshot crop: dock region — empty-state hint, six workflow chips (Calibrate Q0 · Bell pair · Q0 readiness · Device status · Improve Bell · Diagnose Q0), and the high-contrast composer with ⌘K.

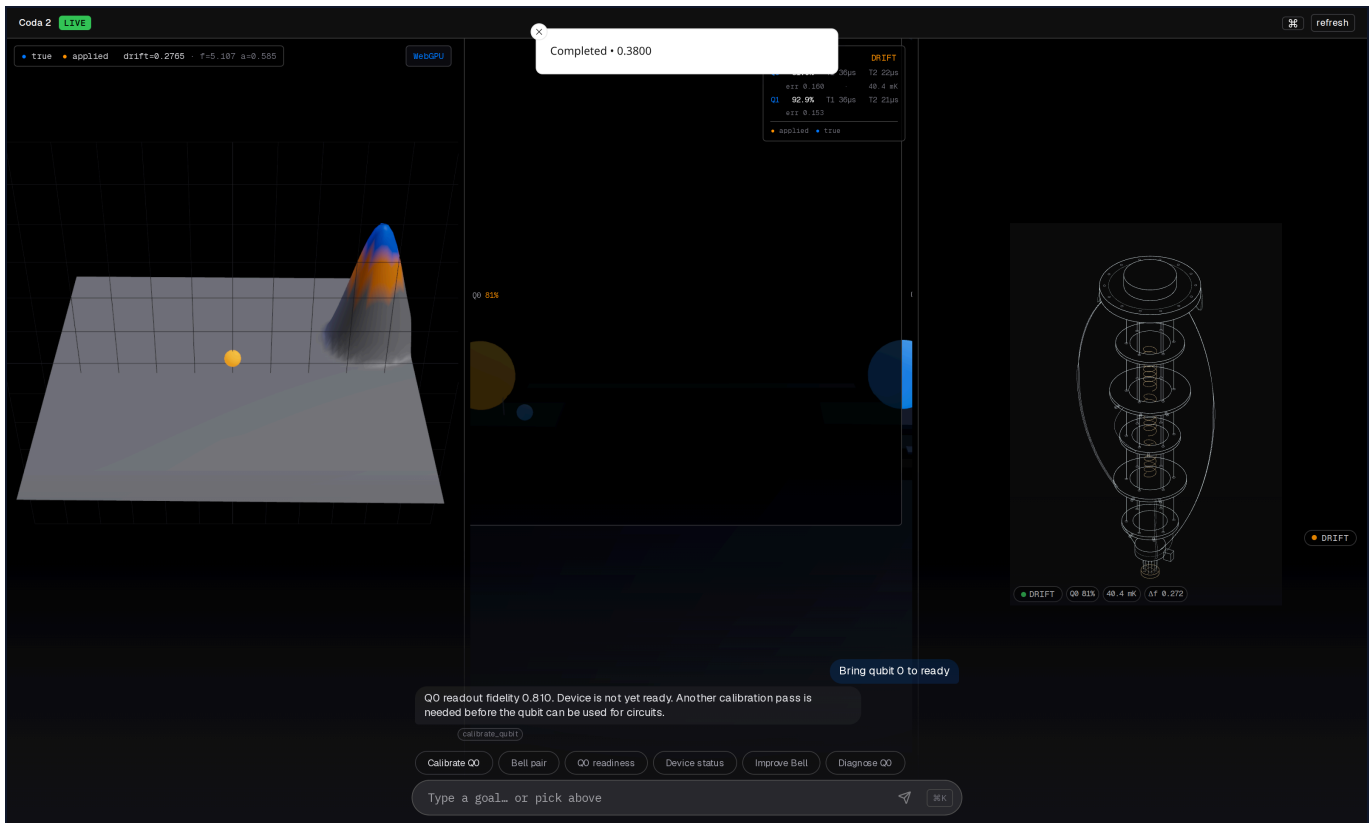


FIG. 8.4 Screenshot: a Calibrate Q0 turn on the three-pane stage. All three panes stay live. The you-bubble sits right; the agent reports fidelity and readiness. The Calibrate Q0 path is unchanged.

8.1 Layout rules

- Full-viewport black shell (`h-screen w-screen`).
- Toolbar without traffic lights: brand, LIVE badge, READY centered, ⌘K and refresh on the right.
- Three-pane stage at rest. Left: high-contrast drift grid, coplanar with the fidelity surface. Center: Device3D (qubits, chiplets, resonators, readout card). Right: isometric cryostat plate with status strip.
- Two iPadOS Split View–style separators. Hover shows a capsule drag handle. Drag to resize. Each pane keeps a minimum width.
- No glass inset. No tabs. (PR #2 labels drift | device were tabs. That control is gone.)
- Bottom-right `StageStateChip` (OFFLINE → DRIFT → CALIBRATING → RUNNING → READY / FAILED).
- Content area: stage absolute inset-0; dock overlaid at bottom over a Safari-safe bottom-third frost (`.stage-blur` , no `backdrop-filter`) so the canvases extend underneath and the top two-thirds stay sharp.
- Composer max width ~42rem, centered; six workflow chips above it.
- Dock transcript uses you (right) / agent (left) bubbles above chips, with small tool pills under agent replies. No card wall on first paint.

8.2 Drift visualisation

`calibration-surface.tsx` builds a PlaneGeometry heightfield from `estimateFidelity(dx,dy,base)`, peaked at the true offset implied by live detuning. Cyan marker = true; amber = applied at origin of the applied frame. Colours: ≥ 0.88 blue, ≥ 0.75 orange, else gray. On current main the left grid is high contrast and shares the tilted group with that surface, so the grid and the mountain sit coplanar. WebGPU via async `three/webgpu` renderer when `navigator.gpu` exists; else WebGL + banner.



FIG. 8.5 Semantic map of the stage surface.

8.3 Composer, chips, ledger, details

Workflow chips: Calibrate Q0, Bell pair, Q0 readiness, Device status, Improve Bell, Diagnose Q0 — each submits a goal string through `api.postGoal`. Turns append to local state with status running $\rightarrow$ succeeded/failed, traces, fidelityHistory, bellCounts. The dock transcript stacks you/agent bubbles above chips; selecting a turn expands details (fidelity climb, tool traces, optional counts). The primary operator path stays first-class: a natural-language goal or a chip, then calibrate, then READY, then a Bell circuit.

8.4 Command palette and demos

⌘K opens cmdk actions: goals, refresh, force fail next calibration, start long job. Demo actions hit `/demo/*` endpoints. Cancel uses goal `cancel {jobId}` or stop button on running jobs.

8.5 Polling

Device + metrics refresh every 6.5 s when connected. Active running jobs poll every 850 ms; SSE also attaches via EventSource to `/sse/jobs/{id}`.

8.6 Design stack

CONCERN	LIBRARY
Command palette	cmdk (no open animation)
Toasts	sonner
Motion	motion (transform/opacity, ease [0.23,1,0.32,1])
Charts	recharts
3D	three + @react-three/fiber + drei
State	zustand
Primitives	shadcn/ui + Radix

Table 8.1. UI dependencies (see ui/README.md).

Sources. ui/app/page.tsx; ui/components/viz/calibration-surface.tsx; ui/components/viz/device-3d.tsx; ui/components/viz/cryostat-plate.tsx; ui/components/stage/stage-state-chip.tsx; ui/components/command/command-palette.tsx; ui/lib/api.ts; ui/README.md

Operations

Install, run, demo, and test this system on a developer machine. Python ≥ 3.11 , Node ≥ 18 , no API keys required for the default path.

9.1 Setup

```
git clone https://github.com/mariodelgado/ai-to-qpu-spike
cd ai-to-qpu-spike
make setup          # pip install -e ".[dev]" + cd ui && npm install
```

9.2 Run full stack

```
# Terminal A
make run-api          # FastAPI http://localhost:8000
# Terminal B
make run-ui-dev       # Next.js http://localhost:3000
# open http://localhost:3000
```

Optional: `NEXT_PUBLIC_API_BASE=http://localhost:8000` . Production UI: `make run-ui` (build + start).

9.3 Makefile targets

TARGET	ACTION
setup	Python editable install + npm install
install	pip install -e ".[dev]"
demo	calibration + circuit Python demos
demo-calibration / demo-circuit	Individual demos
founder-demo	Scriptable orchestrator demo
founder-demo-stub	Same against ConductorShapedAdapter
test	pytest tests/ -v
lint / format	ruff check / format
run-api	uvicorn API :8000
run-ui / run-ui-dev	Next.js prod / dev :3000
clean	Build artifacts, caches, demo_outputs

Table 9.1. Makefile surface.

9.4 Tests

Expected areas: adapter contract, adapter selection via env, API happy path, calibration convergence, failure paths (cap, cancel). Guarantees stated in README: pytest green, `make demo` works, offline, no keys.

9.5 Environment variables

VARIABLE	DEFAULT	EFFECT
CONDUCTOR_QPU_BACKEND	sim	sim vs stub/hardware/shaped/conductor
CONDUCTOR_ENABLE_LLM	0	Enable optional OpenAI planner
CONDUCTOR_LLM_MODEL	gpt-4o-mini	Model when LLM planner on
NEXT_PUBLIC_API_BASE	http://localhost:8000	UI API base
FOUNDER_DEMO_FAIL	—	If 1, founder_demo caps fidelity

Table 9.2. Environment knobs.

Sources. Makefile; pyproject.toml; ui/package.json; README.md

Worked walkthroughs

Step sequences for the founder demo paths. Pair with the screenshots in Part 8.

WALKTHROUGH - CALIBRATE Q0

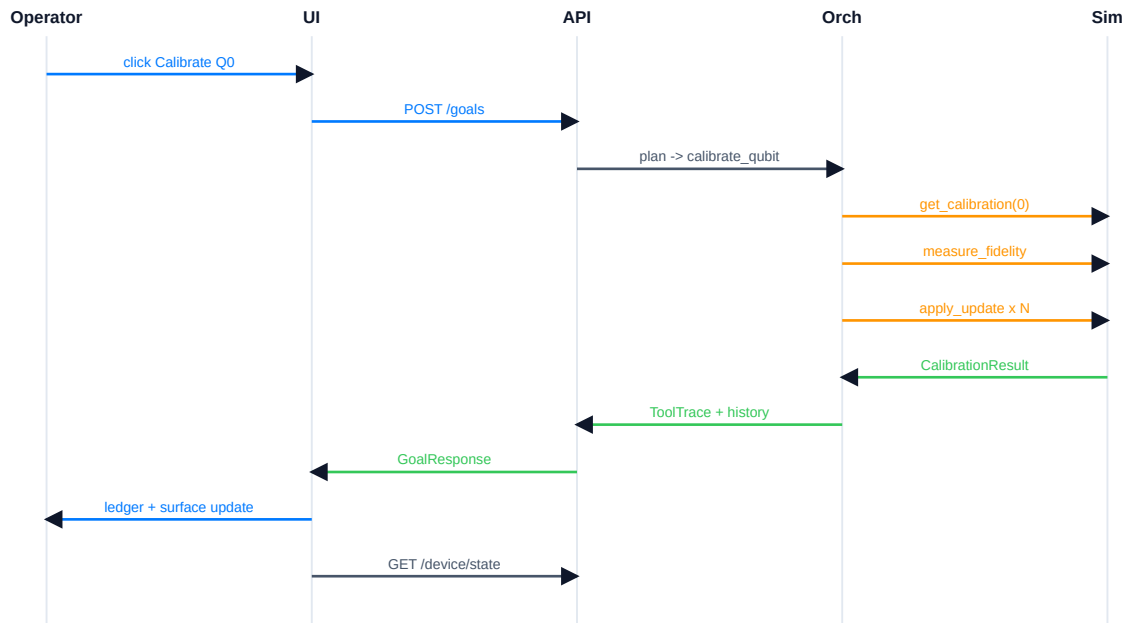


FIG. 10.1 Sequence: Calibrate Q0 from chip click through GoalResponse and surface update.

10.1 Calibrate Q0

1. API and UI running; toolbar shows LIVE.
2. Observe cyan/amber separation on the stage (drift).
3. Click **Calibrate Q0** (goal: “Bring qubit 0 to ready”).
4. Ledger turn appears running; fidelity climb HUD may populate as history returns.
5. On completion, toast “Reached threshold” if last fidelity $\geq$ threshold; surface markers should move closer if the loop helped.
6. Click the ledger row: in-rail chart + tool traces (`calibrate_qubit` summary fidelity=...).

WALKTHROUGH - BELL PAIR

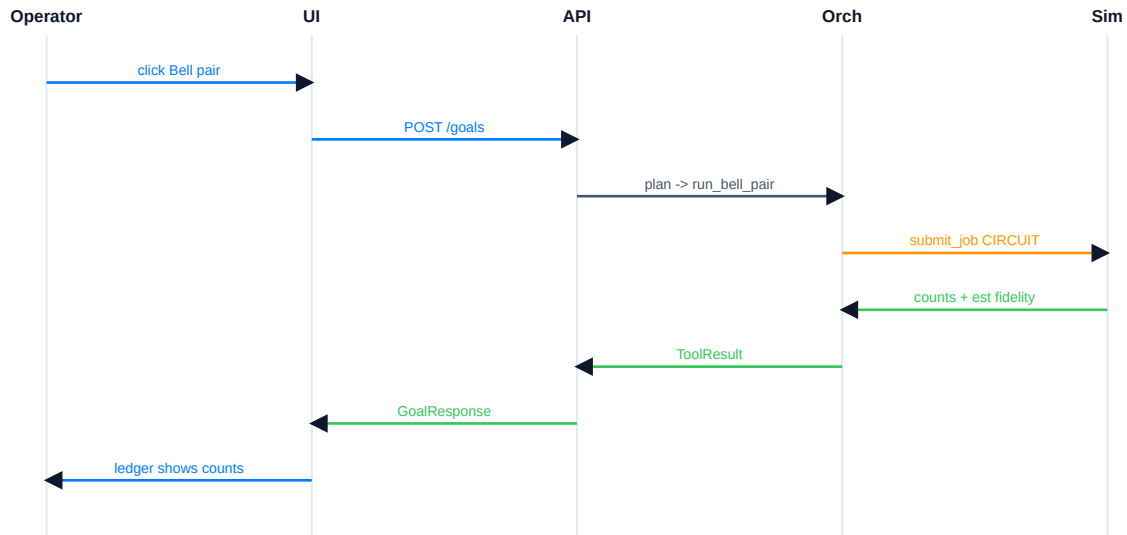


FIG. 10.2 Sequence: Bell pair.

10.2 Bell pair

1. Click **Bell pair**.
2. Orchestrator plans `run_bell_pair`; adapter returns counts.
3. Ledger shows 00/01/10/11 summary; toast “Bell complete”.
4. Status bar continues to reflect latest device snapshot on poll.

10.3 Forced calibration failure

1. ⌘K → “Force fail next calibration” (POST /demo/fail_next_cal, cap 0.69).
2. Run Calibrate Q0.
3. Expect success=false domain result; fidelity below 0.88; toast shows completed fidelity or failure path; metrics attempts increment.

10.4 Cancel long job

1. ⌘K → “Start long job (cancel me)”.
2. Turn shows running; stop button available.
3. Stop/cancel → job CANCELLED; toast “Cancel requested”.

10.5 Scriptable founder demo (no UI)

```
make founder-demo
# or
CONDUCTOR_QPU_BACKEND=stub make founder-demo-stub
python -m demo_scripts.founder_demo --fail
```

Prints steps, traces, three metrics, readiness; exits 0.

Sources. README.md (2-minute founder demo); demo_scripts/founder_demo.py; ui/app/page.tsx (submitGoal, commandActions)

Reference

11.1 Python package

`conductor-qpu` 0.1.0, MIT, requires Python ≥ 3.11 . Runtime deps: `numpy`, `scipy`, `fastapi`, `uvicorn`, `pydantic`, `httpx`, `python-multipart`. Dev: `pytest`, `pytest-asyncio`, `ruff`, `mypy`. Optional llm: `openai`. Scripts: `conductor-qpu-api`, `conductor-qpu-ui`.

11.2 Glossary

TERM	DEFINITION
Adapter	QPUAdapter implementation sitting under the control plane
Applied params	Last calibration vector written to the backend
True params	Hidden simulator state; source of detuning
Detuning	Per-field $ \text{applied} - \text{true} $ exposed for teaching/ops
Goal	NL string mapped to ToolCalls
ToolTrace	Structured record of one tool invocation
Readiness	$\text{all readout_fidelity} \geq 0.82$
Calibration success	$\text{best_fid} \geq \text{service threshold (0.88 default)}$
<code>time_to_calibrated</code>	Wall time to successful threshold cross
<code>interface_latency</code>	<code>apply_calibration_update</code> roundtrip sample
Stage	Primary WebGPU/WebGL visual surface
Dock	Bottom progressive-blur agent/ledger band
Stub	<code>ConductorShapedAdapter</code> — shape only, not connected

Table G.1. Glossary.

11.3 Figure index

FIGURE	TITLE
A.1–A.2	Front plates: who this is for; four tasks
A.3–A.7	Front stills: idle home, typed goal, READY, Bell ledger, instrument map
R.1	The three metrics
C.1	Colour legend
1.1–1.5	QPU primer plates: Bloch, control/readout, why calibrate, stack, our seam
1.6–1.9	Tools, cal loop, software stack map, thresholds
2.1–2.8	Theory diagrams (layers, contract, loop, drift, surface, UI rationale, layout, failures)
D.01–D.06	Mid-century plates: whole machine, cal loop, language firewall, stage/dock, metrics, real vs sim
3.1–3.3	Architecture, package, dataflow
5.1–5.2	Adapter contract, drift
6.1–6.3	Metrics, loop, failures
7.1	Tool catalog
8.1–8.5	UI layout, screenshots, surface semantics
10.1–10.2	Calibrate Q0 and Bell sequences

Table F.1. Figure index (primary).

11.4 Week-1 port checklist (from README)

1. Implement QPUAdapter against vendor SDK / internal driver.
2. Persist jobs + traces (Postgres); add goal_id.
3. Calibration policy object (strategies, cost model, multi-qubit readiness, calibrate-or-not recommendation).
4. Narrow plan+execute+observe API or MCP tool surface.
5. Real detuning from diagnostics.
6. Guardrails: kill switches, max iterations, backoff, refuse circuit below fidelity X.

11.5 Closing

This manual describes the AI → QPU integration layer on main @ `b05005c` (2026-10-03/04 PT, through PR #18). It is a control-plane integration layer with a noisy simulator behind a real seam. The theory of the interface — goals, tools, closed-loop calibration, trust surfaces, instrument UI — is the durable claim. The simulator numbers are not.

Theory addendum — design alternatives rejected

A short record of alternatives considered and rejected while shaping the AI → QPU interface. This is part of the theory of the interface. Knowing what was not chosen clarifies what was.

2B.1 “Just wrap the vendor SDK in LangChain tools”

Rejected as the primary architecture. Vendor SDKs expose dozens to hundreds of calls with pulse-level detail. Dumping them into an LLM tool list creates brittle agents that invent illegal parameter combinations and hide calibration as an incidental side effect of “run_experiment”. This system instead offers five tools and a planner, with calibration as an explicit tool that returns history and metrics. A production system may still wrap vendor calls — but behind the adapter, not as the agent’s entire world.

2B.2 “ChatGPT as the only UI”

Rejected for ops. Language is excellent for composing goals and explaining failures after the fact. It is poor for continuous physical state. Detuning is a spatial relationship; fidelity climb is a time series against a threshold; readiness is a predicate over multiple qubits. Those belong on an instrument stage. Chat remains available via the composer and ⌘K, subordinated to the stage.

WHY INSTRUMENT UI BEATS CHAT-ONLY FOR QUANTUM OPS

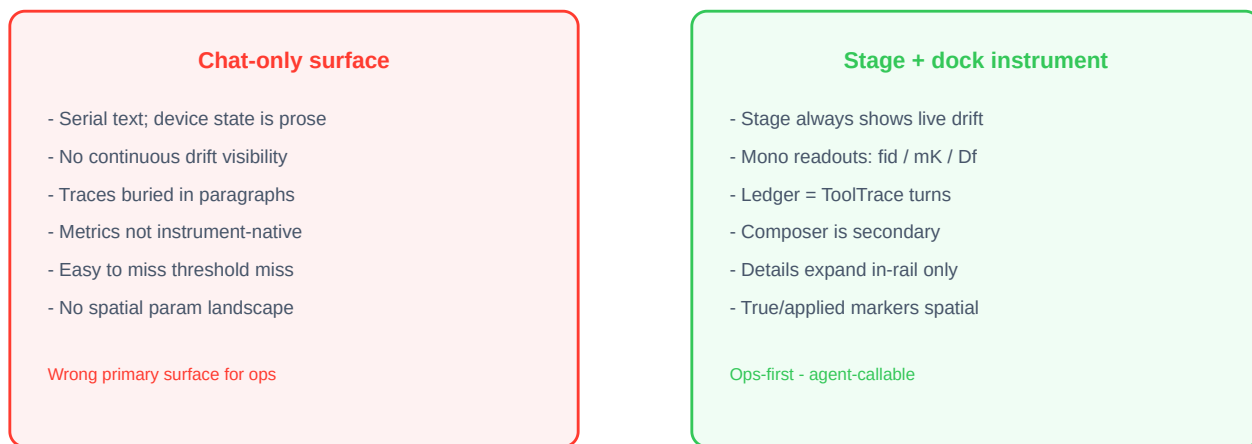


FIG. 2B.1 Reprise: chat-only vs instrument — the rejected alternative on the left.

2B.3 “One-shot Bayesian optimisation as the calibration product”

Rejected for this system’s thesis. Advanced tuners matter and should plug in as CalibrationService strategies. Shipping a fancy optimiser first would confuse the demo: founders would argue about acquisition functions instead of asking whether the control plane makes calibration a recurring, observable decision. The gradient-free loop is intentionally

simple; the metrics and traces are the product.

2B.4 “Hide the simulator’s true params forever”

Partially rejected. Omniscient detuning is unrealistic for hardware — and the stub correctly refuses it. For teaching and founder demos, exposing `get_detuning` on the sim makes the problem visible on the stage (cyan vs amber). The honesty line in the README states this clearly: the dynamics are representative; the oracle is not.

2B.5 “Require LLM keys for the happy path”

Rejected. Offline-first is a constraint of this system: `make demo` and the UI must work with zero external services. Optional LLM planning is gated. This keeps CI deterministic and founder laptops unblocked.

RULE OF THUMB

Prefer a boring planner and an honest loop over a clever planner and an opaque loop. Agents can get smarter later; fake observability cannot be fixed by a better model.

API schemas and payload examples

Concrete JSON shapes as returned by the FastAPI surface. Field names match `api/server.py` and `ui/lib/api.ts`.

4B.1 POST /goals

```
// request
{ "goal": "Bring qubit 0 to ready" }

// response (shape)
{
  "goal": "Bring qubit 0 to ready",
  "results": [
    {
      "ok": true,
      "data": {
        "fidelity": 0.9012,
        "iterations": 14,
        "params": {"frequency": 5.02, "amplitude": 0.51, "readout_error": 0.04},
        "duration_s": 0.0123,
        "history": [[0, 0.71], [1, 0.74], /* ... */],
        "initial_fidelity": 0.71,
        "threshold": 0.88
      },
      "latency_s": 0.013,
      "error": null
    }
  ],
  "traces": [
    {
      "ts": 1727827200.1,
      "tool": "calibrate_qubit",
      "args": {"qubit_id": 0, "target_fidelity": 0.88},
      "latency_s": 0.013,
      "ok": true,
      "summary": "fidelity=0.9012"
    }
  ],
  "metrics": { /* MetricsAggregator.snapshot() */ }
}
```

4B.2 GET /device/state

```
{
  "is_ready": false,
  "readiness_score": 0.812,
  "qubits": [0, 1],
  "temperatures_mk": {"0": 24.1, "1": 23.8},
  "coherence_us": {"0": [41.2, 24.5], "1": [40.1, 23.9]},
  "readout_fidelity": {"0": 0.81, "1": 0.814},
  "notes": "Calibration recommended",
  "timestamp": "2026-10-01T23:00:00.000000",
  "readiness_predicate": {
    "name": "all_qubits_readout_fidelity_above",
    "readout_fidelity_threshold": 0.82,
    "description": "All qubits must have readout_fidelity ≥ threshold."
  }
}
```

4B.3 GET /device/detuning/0

```
{
  "qubit_id": 0,
  "detuning": {
    "frequency_error": 0.0312,
    "amplitude_error": 0.014,
    "phase_error": 0.08,
    "readout_error_delta": 0.01
  },
  "applied": {
    "frequency": 5.0,
    "amplitude": 0.48,
    "phase": 0.01,
    "readout_error": 0.06
  }
}
```

4B.4 GET /metrics

```
{
  "orchestrator": {
    "calibration": {
      "attempts": 2.0,
      "successes": 1.0,
      "calibration_success_rate": 0.5,
      "avg_time_to_calibrated_s": 0.02,
      "avg_interface_latency_s": 0.0001
    },
    "tools": {
      "calibrate_qubit": {"calls": 2.0, "avg_latency_s": 0.015}
    },
    "adapter": {"total_jobs": 3.0, "total_shots": 1024.0},
    "interface_latency_p50_hint": 0.0001
  },
  "aggregator": {
    "interface_latency_ms": {"count": 2, "p50": 12.0, "p95": 15.0},
    "fidelity": {"count": 1, "avg": 0.9012, "last": 0.9012},
    "job_outcomes": {"succeeded": 0, "failed": 0, "cancelled": 0},
    "timestamp": 1727827200.0
  },
  "calibration": { /* same shape as orchestrator.calibration */ }
}
```

4B.5 SSE event shapes

```
event: update
data: {"job_id":"...", "status":"running", "result":null, "metrics":null}

event: update
data: {"job_id":"...", "status":"succeeded", "result":{"counts":{"..."}}, "metrics":{"..."}}

event: timeout
data: {"job_id":"..."}

event: error
data: {"error":"not found"}
```

4B.6 Direct calibrate / bell

ENDPOINT	NOTABLE RESPONSE FIELDS
POST /calibrate	success, fidelity, iterations, duration_s, params{qubit_id,frequency,amplitude,readout_error}, history
POST /circuit/bell	job_id, counts{00,01,10,11}, metrics{estimated_fidelity,shots,p00,p11} or error
POST /demo/fail_next_cal	ok, fid_cap=0.69, note
POST /demo/start_long_job	job_id

Table 4B.1. Direct and demo endpoints — response highlights.

Simulator physics notes

Additional detail on the noisy simulator's fidelity kernel, drift update, and Bell contrast — enough to reimplement or replace without reading the UI.

5B.1 Error terms

Let `p` be applied params and `t` true params for qubit `q`. Define:

- `df = |p.frequency - t.frequency|`
- `da = |p.amplitude - t.amplitude|`
- `dp =` absolute wrapped phase difference on `[0, π]`
- `dt1 = max(0, t.t1 - p.t1) / max(1, t.t1)` (penalise claiming too-long T1)
- `dt2` analogous for T2
- `dr = |p.readout_error - t.readout_error|`

Composite error:

```
err = 0.9*df2 + 0.7*da2 + 0.5*dp2 + 0.35*dt12 + 0.35*dt22 + 0.85*dr2
fid = clamp(0.94*exp(-1.8*err) + 0.05, 0.55, 0.999)
```

Optional demo cap: `fid = min(fid, cap)` before clamp upper bound still applies via min with cap then clamp.

5B.2 Drift step

On each drift application with elapsed `dt` and `steps = max(1, int(dt*20))`:

```
f_drift = drift_rate * (1 + 0.3*sin(now*0.7 + q))
frequency += N(0, f_drift * steps) # clamped [4.2, 5.9]
amplitude += N(0, 0.0008 * steps) # clamped [0.25, 0.75]
phase      += N(0, 0.002 * steps)
t1, t2, readout_error similarly with their sigmas and clamps
```

5B.3 Bell contrast

```
joint_f = (f0 + f1) / 2
contrast = clamp(0.5 + 0.48*(joint_f - 0.6)/0.4, 0.02, 0.98)
```

Ideal bits are 00 or 11 with equal probability; readout flips each bit with the applied `readout_error`; with probability `1 - contrast` the outcome is forced to odd parity (01 or 10).

5B.4 Device state theatre

Temperature: $18 + (1-f)*35 + \text{noise}$ mK. Coherence scales with fidelity. Readout fidelity card: $1 - \text{readout_error}*(1.3 - 0.3*f)$ clamped. Ready iff all readout fidelities > 0.82 .

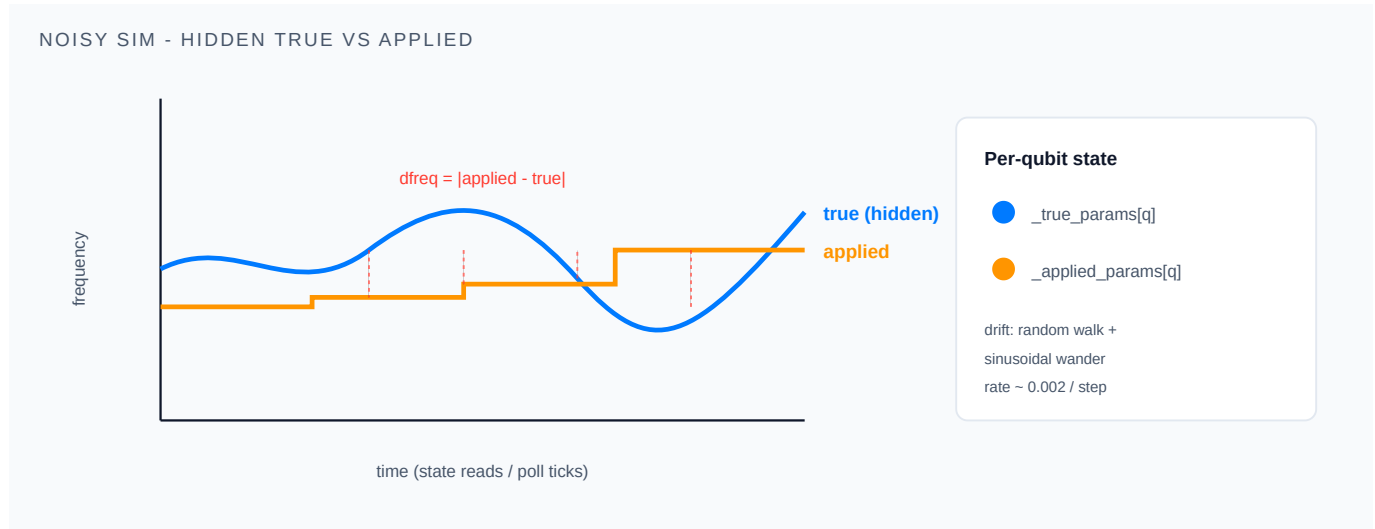


FIG. 5B.1 Drift cartoon corresponding to §5B.2.

Sources. `adapter/noisy_sim.py` (`_apply_drift`, `_fidelity_from_params`, `_simulate_bell_readout`, `get_device_state`)

Calibration search parameters

Exact neighbour sampling and acceptance rules as coded — for anyone replacing the strategy while keeping metrics.

FIELD	REL STEP (scale>1)	REL STEP (else)	CLAMP
frequency	0.032	0.018	[4.20, 5.90]
amplitude	0.065	0.035	[0.24, 0.76]
phase	0.28	0.28	$[-\pi, \pi]$
t1	0.10	0.10	[15, 88]
t2	0.11	0.11	[10, 68]
readout_error	0.22	0.13	[0.002, 0.16]

Table 6B.1. `_sample_neighbor` relative Gaussian scales (multiplied by scale and applied as abs delta).

6B.1 Scale schedule

- Iterations < 8: `base_scale` 1.6
- < 20: 1.1
- else: 0.85
- If `no_improve` > `patience/2`: force `base_scale` 1.8
- Final scale = `base_scale` * (0.6 + 0.8*U[0,1])

6B.2 Metrics update rules

- Every apply: `total_interface_latency_s += latency ; latency_samples += 1`.
- Every finished `calibrate()` call: `attempts += 1`.
- On success only: `successes += 1 ; total_time_to_cal_s += duration_s`.
- Averages divide by successes (time) or `latency_samples` (latency); success rate uses attempts.

UI chrome and progressive blur

Implementation notes for the instrument chrome beyond the React tree — so a designer or engineer can preserve the three-pane stage (drift | device 3D | cryostat) over a bottom-third dock.

8B.1 Why the stage extends under the dock

Progressive blur via `backdrop-filter` only works if there is content behind the frosted layer. The stage is therefore positionally full-bleed; the dock is an overlay with a gradient mask that is stronger near the composer and fades toward the stage. First paint must never be an empty chat void.

8B.2 Palette tokens (conceptual)

TOKEN	HEX	USE
--true / system blue	#007AFF	True marker, high fidelity, goal text accents
--applied / system orange	#FF9500	Applied marker, threshold lines, CAL NEEDED
--success	#34C759	LIVE badge, success fidelity text
--fail	#FF3B30	Failed turns, backend errors
label gray	#8E8E93 / zinc	Secondary labels, low fidelity band
hairline	white/10 borders	iOS-like separators

Table 8B.1. iOS system palette as used in UI and diagrams.

8B.3 Motion constraints

Animate only transform/opacity. Ease-out cubic-bezier(0.23, 1, 0.32, 1). ⌘K open has no animation. NumberFlow may be used on metrics cards in alternate layouts; the instrument toolbar uses mono text updates on poll.

8B.4 Connection failure UX

On health check failure: `connected=false`, `backendDown=true`, red banner “Cannot reach backend — make run-api”, composer disabled, toast on submit failure. This is a control-plane honesty surface: the UI must not pretend the device is live.

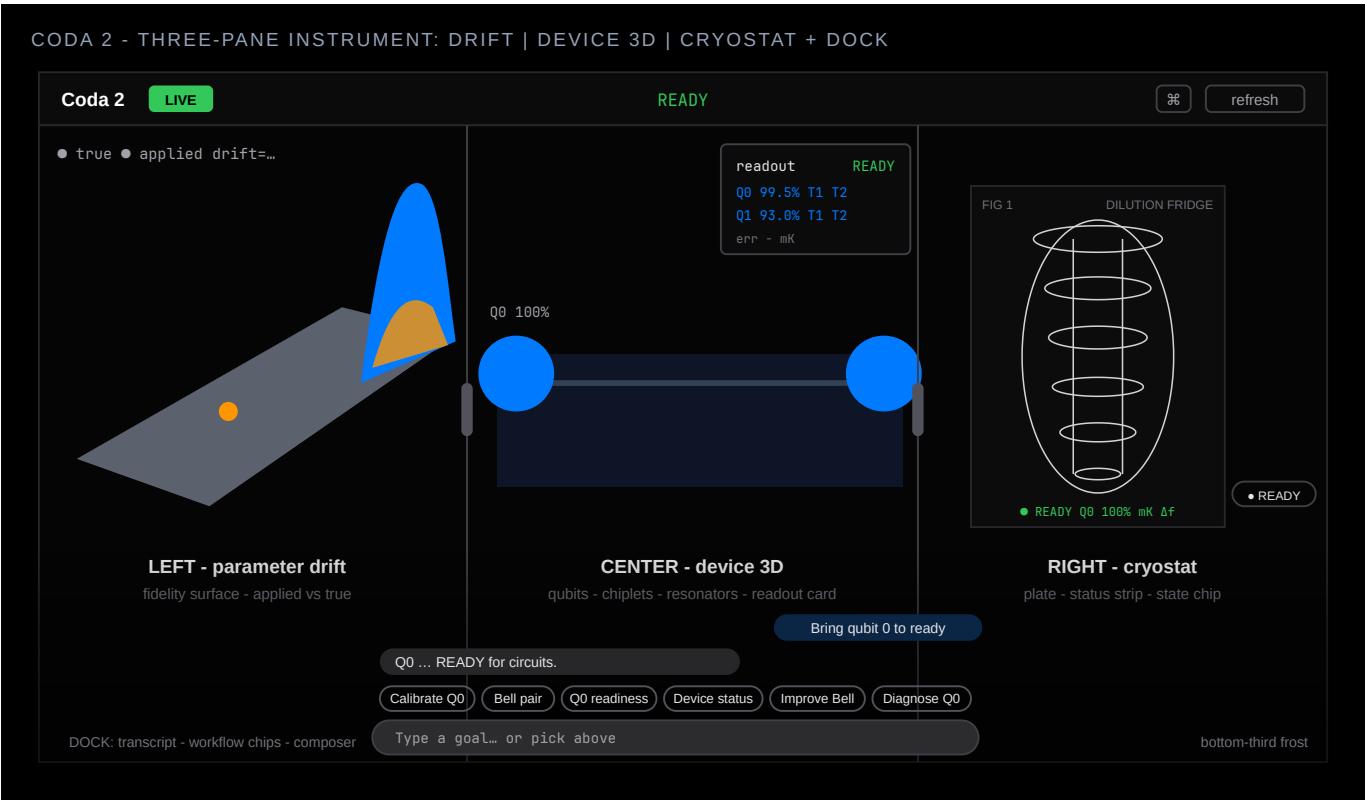


FIG. 8B.1 Layout reference for chrome work.

Test matrix

What each test module guards. Keep these green when changing the adapter or calibration loop.

MODULE	ASSERTIONS (SUMMARY)
test_adapter_contract.py	Adapter methods exist and behave: submit/poll/cancel, state, calibration apply returns fidelity
test_adapter_selection.py	factory env selects sim vs stub
test_api_happy_path.py	Health, goals, device, metrics HTTP paths succeed against TestClient
test_calibration_converges.py	Service can improve fidelity on noisy sim; metrics populated
test_failure_paths.py	Fid cap → success False; long job cancel → CANCELLED

Table 9B.1. Pytest modules under tests/.

9B.1 Running

```
make test
# equivalent
pytest tests/ -v --tb=short
```

asyncio mode auto via pyproject. Prefer short tracebacks in CI logs.

9B.2 Demo scripts as smoke tests

demo_scripts/run_calibration_demo.py and run_circuit_demo.py print human-readable climbs and counts. founder_demo.py exercises orchestrator goals end-to-end including optional `--fail`. They are not substitutes for pytest but catch packaging/import regressions quickly via `make demo`.

Endpoint catalog (complete)

METHOD	PATH	AUTH	PRIMARY CONSUMER
GET	/health	none	UI connection check
GET	/readiness_predicate	none	UI / founders
GET	/device/state	none	UI poll 6.5s
GET	/device/detuning/{qubit_id}	none	UI drift surface
POST	/goals	none	UI composer/chips/⌘K
POST	/calibrate	none	Direct/API clients
POST	/circuit/bell	none	Direct/API clients
GET	/jobs	none	UI job list
GET	/jobs/{job_id}	none	UI poll
GET	/sse/jobs/{job_id}	none	UI EventSource
GET	/traces	none	Debug / UI
GET	/metrics	none	UI metrics refresh
POST	/demo/fail_next_cal	none	⌘K demo
POST	/demo/start_long_job	none	⌘K demo
GET	/docs	none	OpenAPI Swagger UI

Table 4C.1. Complete FastAPI route list on server.py.

Numeric walkthrough — one calibration attempt

Illustrative numbers consistent with the model (not a captured golden run). Shows how history, metrics, and UI fields relate.

STEP	ACTION	FIDELITY	NOTES
0	measure initial	0.72	history (0, 0.72)
1–3	wide explores	0.74–0.79	some accepts worse via anneal
4–8	improve	0.83	crosses readiness 0.82; service still climbing to 0.88
9–12	tighten scale	0.87	near threshold
13	accept	0.901	success; break

Table 10B.1. Illustrative climb. UI threshold line at 0.88; READY may flip earlier based on readout card ≥ 0.82 .

After success: `attempts=1` , `successes=1` , `calibration_success_rate=1.0` , `avg_time_to_calibrated_s` $\approx$ `duration_s` , `latency_samples=13` . ToolTrace summary `fidelity=0.901` . Detuning `frequency_error` should shrink if applied moved toward true.

Under fid cap 0.69: steps continue to `max_iterations` or patience cycling; `best_fid` ≤ 0.69 ; `success=False`; `successes` unchanged; `attempts` still +1; `success_rate` drops.

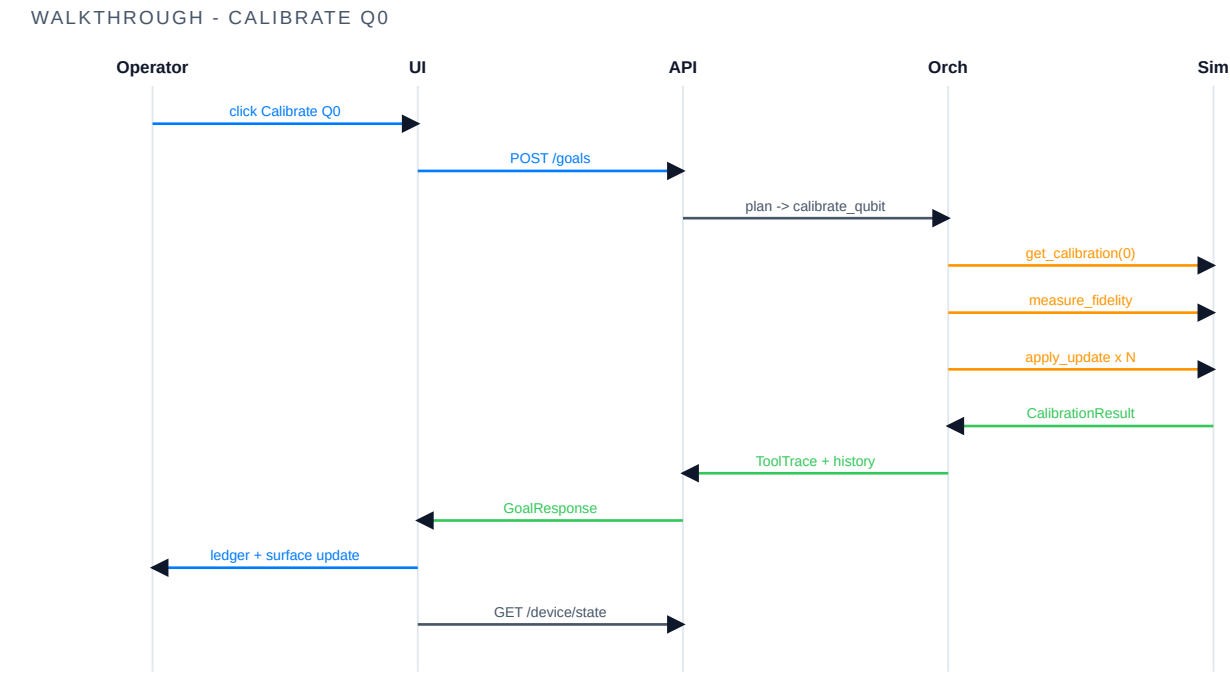


FIG. 10B.1 Sequence context for the numeric climb.

TWO THRESHOLDS - READINESS VS CALIBRATION SUCCESS

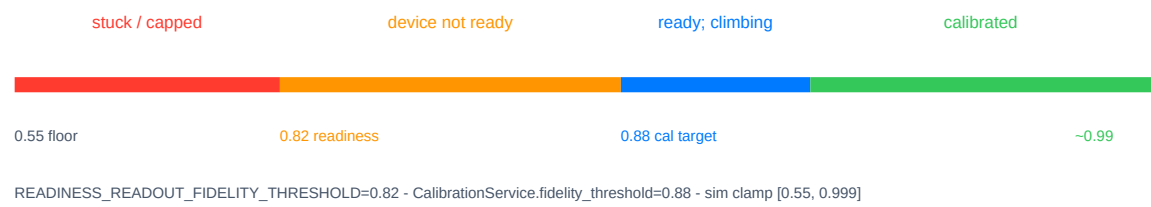


FIG. 10B.2 Where 0.82 and 0.88 sit on the number line during the climb.

Agent integration patterns

How an external agent should call this control plane without treating it as a chatbot.

7B.1 Preferred pattern

1. GET /device/state — decide if work is needed.
2. POST /goals with an explicit calibration or circuit goal — or POST /calibrate / /circuit/bell for direct control.
3. Read results[].ok and traces[] — do not invent narrative.
4. GET /metrics — update dashboards.
5. If running async jobs, follow GET /jobs/{id} or SSE.

7B.2 Anti-patterns

- Asking an LLM to “explain” device state instead of reading /device/state.
- Ignoring success=false because HTTP was 200.
- Skipping calibration because a previous session’s UI looked green (state is in-memory per process).
- Calling stub backend and treating fidelity 0.90 calibration stub success as hardware truth.

HONESTY

MCP / tool-gateway exposure is a week-1 item in the README, not shipped in 0.1.0. Today the HTTP surface is the agent API.

GOAL TO TOOLS TO ADAPTER TO DEVICE STATE

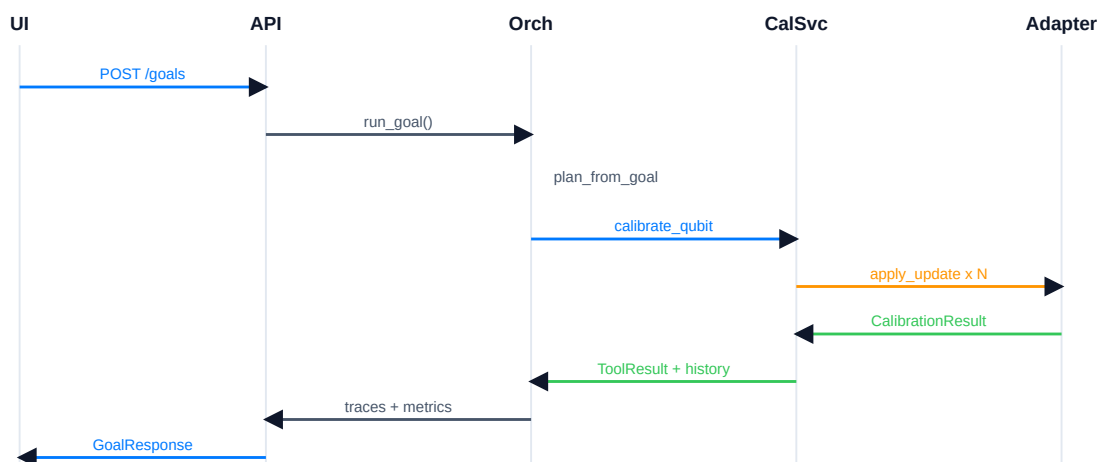


FIG. 7B.1 HTTP dataflow agents should follow.

Symbol and path index

SYMBOL / PATH	ROLE
conductor_qpu.adapter.base.QPUAdapter	Abstract six-method contract
conductor_qpu.adapter.noisy_sim.NoisySimulatorBackend	Default drifting toy device
conductor_qpu.adapter.hardware_stub.ConductorShapedAdapter	Shape-only stub
conductor_qpu.adapter.factory.create_backend	Env-based backend selection
conductor_qpu.calibration.service.CalibrationService	Iterative tuner + CalibrationMetrics
conductor_qpu.orchestrator.orchestrator.Orchestrator	Tools + traces + run_goal
conductor_qpu.orchestrator.planner.plan_from_goal	Deterministic NL → tools
conductor_qpu.api.server.app	FastAPI application
conductor_qpu.models.types	Job*, DeviceState, CalibrationParams, thresholds
conductor_qpu.observability.metrics.MetricsAggregator	Rolling latency/fidelity samples
conductor_qpu.jobs.store.InMemoryJobStore	Non-durable job history seam
ui/app/page.tsx	Instrument composition
ui/components/viz/calibration-surface.tsx	WebGPU/WebGL drift landscape
ui/lib/api.ts	Typed HTTP client
READINESS_READOUT_FIDELITY_THRESHOLD	0.82
CalibrationService.fidelity_threshold	0.88 (API)

Table 11B.1. Path and symbol index.

11B.1 Document control

FIELD	VALUE
Title	AI → QPU Integration Layer Technical Manual
Edition	Engineering Edition — October 2026
Describes	main @ b05005c (through PR #18, 2026-10-03/04 PT)
Source tree	/workspace/manual/ (HTML/CSS + diagrams)
PDF deliverable	/workspace/ai-to-qpu-technical-manual.pdf
Repo	https://github.com/mariodelgado/ai-to-qpu-spike

Table 11B.2. Document control block.

Architecture decision records

Short ADRs for this system. Each records context, decision, and consequences as shipped on main.

ADR-001 — Narrow adapter waist

Context. Real QPUs differ wildly in pulse programming models. **Decision.** Six synchronous methods on QPUAdapter. **Consequences.** Orchestrator/UI stay stable; week-1 work is implementing the six methods. Async/hardware batching is a future wrapper concern.

ADR-002 — Deterministic planner by default

Context. NL goals are convenient; LLM keys are not. **Decision.** Rule-based planner; optional LLM behind env flag. **Consequences.** Demo and CI are offline; goal coverage is intentionally small; expand rules or enable LLM later without changing tools.

ADR-003 — In-memory everything

Context. Weekend prototype. **Decision.** No database. **Consequences.** Refresh-safe only within process lifetime; persistence is an explicit next step.

ADR-004 — Instrument UI over dashboard cards

Context. Early Streamlit UI became a card wall. **Decision.** Replace with a Next.js instrument (65/35 at first; now a three-pane stage over a dock). **Consequences.** Higher front-end complexity; clearer founder narrative; WebGPU dependency with WebGL fallback.

ADR-005 — Expose sim detuning

Context. Calibration needs a visible problem. **Decision.** get_detuning on sim; stage markers. **Consequences.** Teaching oracle; stub must raise; production must replace with measured diagnostics.

ADR-006 — Domain failures as 200 + flags

Context. Agents should parse tool outcomes. **Decision.** Calibration failure returns HTTP 200 with success/ok false. **Consequences.** Do not key off HTTP status alone for domain errors; UUID errors still 4xx.

SYSTEM ARCHITECTURE - COMPONENT + DATA FLOW

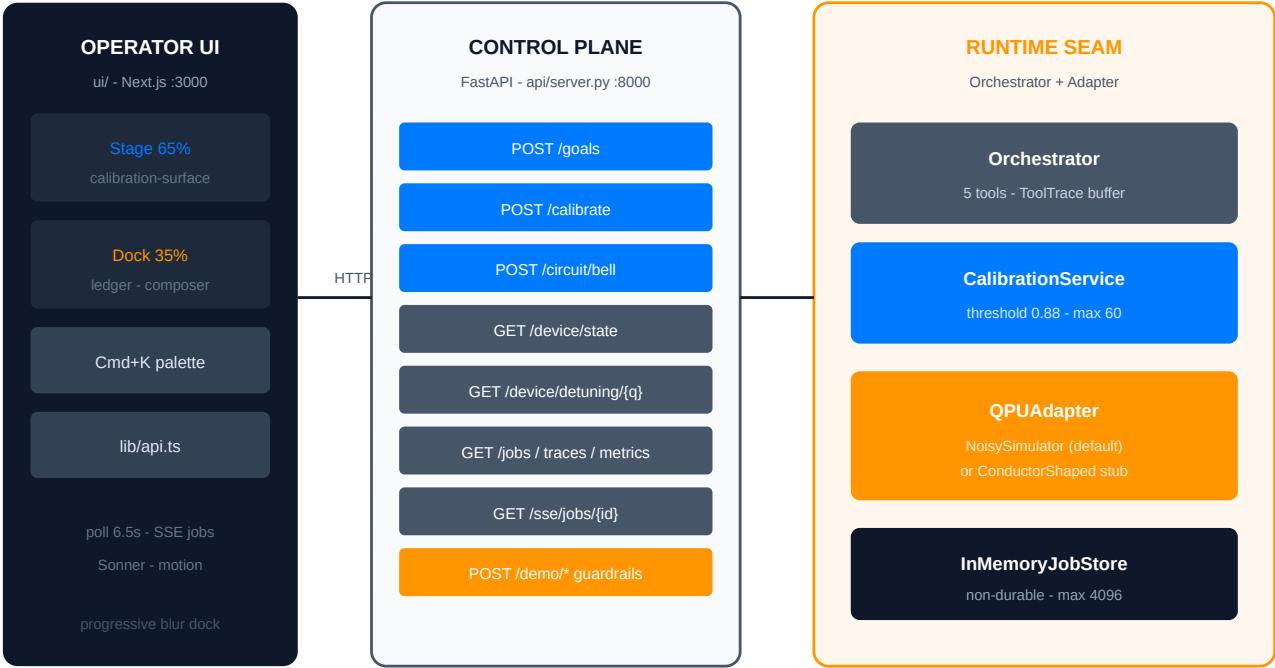


FIG. 3B.1 Architecture context for ADR-001/004.

Stub driver integration sketch

How a real hardware team should read `ConductorShapedAdapter` — as a checklist, not as runnable firmware.

METHOD	REAL DRIVER SHOULD...	STUB DOES...
<code>submit_job</code> (CIRCUIT)	Validate/lower pulse_program; allocate AWG; load; arm; collect shots	FAIL with <code>NotImplemented</code> message + <code>payload_shape</code>
<code>submit_job</code> (CALIBRATION)	Write versioned cal table; optional verify sequence	Accept shape; <code>apply_calibration_update</code> stub fidelity 0.90
<code>poll_job</code>	Return queue/run/complete from control system	Return in-memory record
<code>cancel_job</code>	Best-effort abort if non-terminal	Mark CANCELLED if non-terminal
<code>get_device_state</code>	Query monitors, FPGA, env sensors	Static snapshot; <code>is_ready=False</code> ; NOTES disconnected
<code>get_calibration</code>	Read committed table	Last applied or defaults
<code>apply_calibration_update</code>	Validate ranges; commit; verify; return measured fid	Store last applied; fidelity 0.90 message STUB
<code>get_detuning</code>	Spectroscopy / Ramsey estimate	<code>NotImplementedError</code>
<code>measure_fidelity</code>	Hardware estimate	Return 0.55

Table 5C.1. Stub vs real driver checklist (`hardware_stub.py` docstring).

```
# integration sketch
export CONDUCTOR_QPU_BACKEND=stub # proves seam
# then subclass:
class MyDriver(ConductorShapedAdapter):
    def submit_job(self, job):
        ... # talk to control stack
# register in factory.create_backend()
```

QPUADAPTER - SIX-METHOD CONTRACT

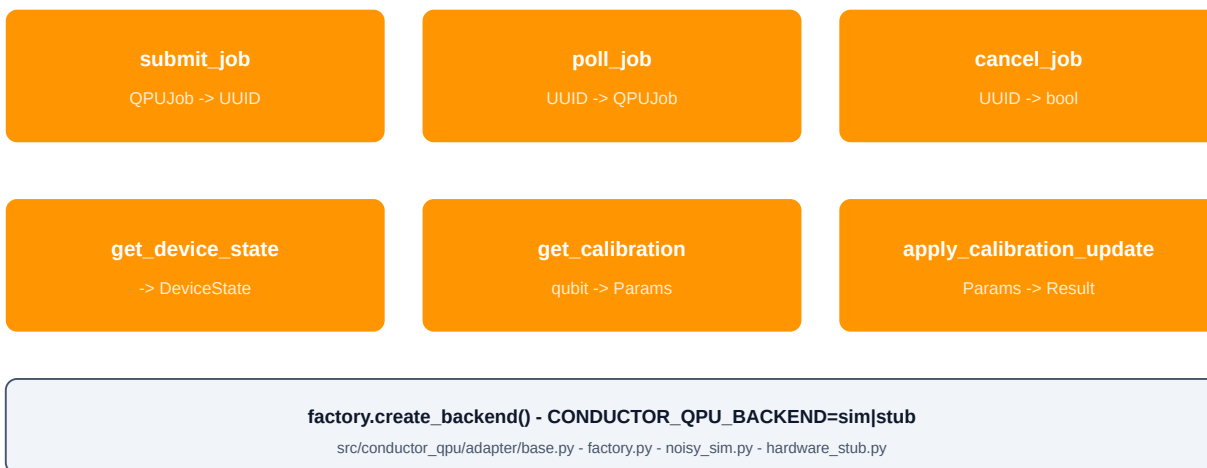


FIG. 5C.1 Methods the subclass must honour.

UI component map

File-level map of the Next.js tree that implements the instrument.

PATH	RESPONSIBILITY
ui/app/page.tsx	Root instrument: toolbar, three-pane stage (drift · device · cryostat), iPadOS splitters, bottom-third dock with bubbles + expanded chips, turns, polling
ui/app/layout.tsx	HTML shell / fonts
ui/app/globals.css	Tokens, toolbar, Safari-safe <code>.stage-blur</code> (solid bands), dock tint, high-contrast composer, hairlines
ui/components/viz/calibration-surface.tsx	R3F landscape + high-contrast coplanar grid + markers + GPU mode
ui/components/viz/device-3d.tsx	Center pane: qubits, chiplets, resonators, readout card (fidelity / T1 / T2)
ui/components/viz/cryostat-plate.tsx	Right pane: isometric dilution-fridge plate + status strip
ui/components/stage/stage-state-chip.tsx	Bottom-right StageStateChip lifecycle badge
ui/components/viz/calibration-surface-lazy.tsx	Dynamic import boundary
ui/components/command/command-palette.tsx	⌘K actions
ui/components/inspector/right-inspector.tsx	Legacy/alternate inspector pieces
ui/components/stage/hero-viewer.tsx	Stage helper
ui/components/timeline/filmstrip.tsx	Timeline alternate
ui/components/metrics-cards.tsx	Metrics presentation helper
ui/lib/api.ts	Fetch wrapper + types
ui/lib/store.ts	zustand (command open, etc.)

Table 8C.1. UI file map. `page.tsx` is the composition root for the shipped three-pane layout (drift · device · cryostat + dock).

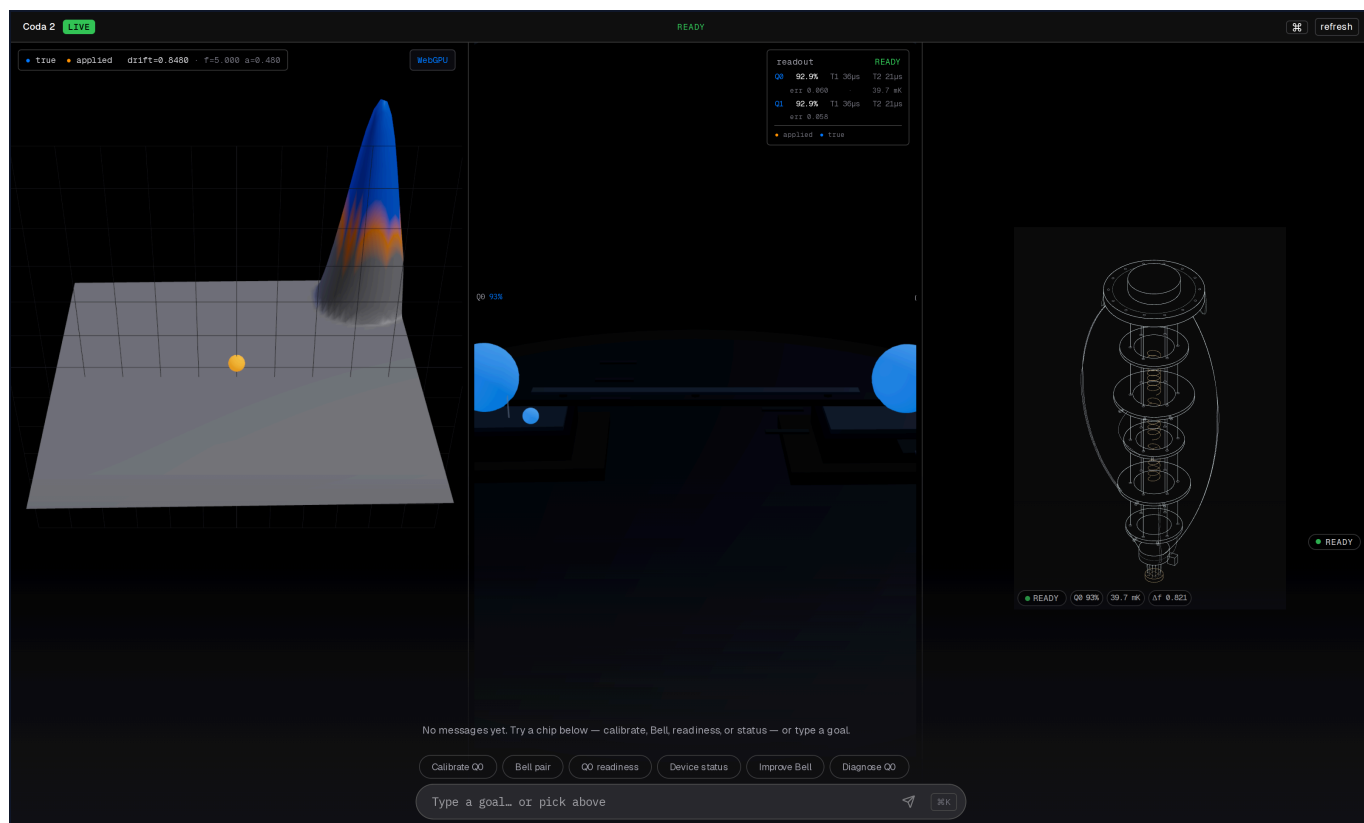


FIG. 8C.1 Full home screenshot for component orientation. Left: calibration-surface.tsx (drift). Center: device-3d.tsx. Right: cryostat-plate.tsx. Bottom: dock in page.tsx.

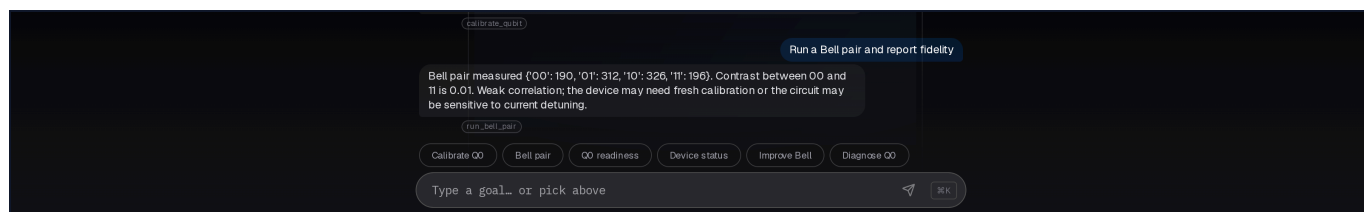


FIG. 8C.2 Dock after two turns — you / agent bubbles, tool pills (`calibrate_qubit` , `run_bell_pair`), workflow chips, composer.

8C.1 State fields on a Turn

FIELD	MEANING
id	Client-generated UUID for the ledger row
goal	Submitted string
status	running succeeded failed
traces	ToolTrace[] from GoalResponse
results	ToolResult-like array
fidelityHistory	{iter, fidelity}[] extracted from data.history
calThreshold	From data.threshold or 0.88
lastCalParams	params object if present
bellCounts	counts object if present
error	Failure message
createdAt	Client timestamp for ledger clock

Table 8C.2. Turn interface in page.tsx.

Operator runbook (90 seconds)

Compressed runbook matching the README's 2-minute founder demo, expanded with failure checks.

1. **Boot.** `make setup` once. Then `make run-api` and `make run-ui-dev`.
2. **Verify LIVE.** Toolbar badge green; if red banner, API is down.
3. **Watch drift.** Cyan vs amber; Δf readout moves slowly.
4. **Calibrate Q0.** Expect climb HUD; ledger turn; threshold toast if crossed.
5. **Inspect traces.** Open ledger row — tool names must be real (`calibrate_qubit`), not prose.
6. **Bell pair.** Counts appear; stage remains primary.
7. **Q0 readiness / Device status / Diagnose Q0.** `get_device_state` fills fidelity/readiness when Bell alone is not enough.
8. **Improve Bell.** Precise (more shots) Bell after calibrate.
9. **Fail path.** ⌘K force fail; recalibrate; confirm success=false behaviour.
10. **Cancel path.** Start long job; stop; confirm CANCELLED.
11. **Metrics.** Refresh; confirm attempts/successes moved.
12. **Stub sanity (optional).** Restart API with `CONDUCTOR_QPU_BACKEND=stub` ; confirm never-ready and circuit failure messaging.

WALKTHROUGH - CALIBRATE Q0

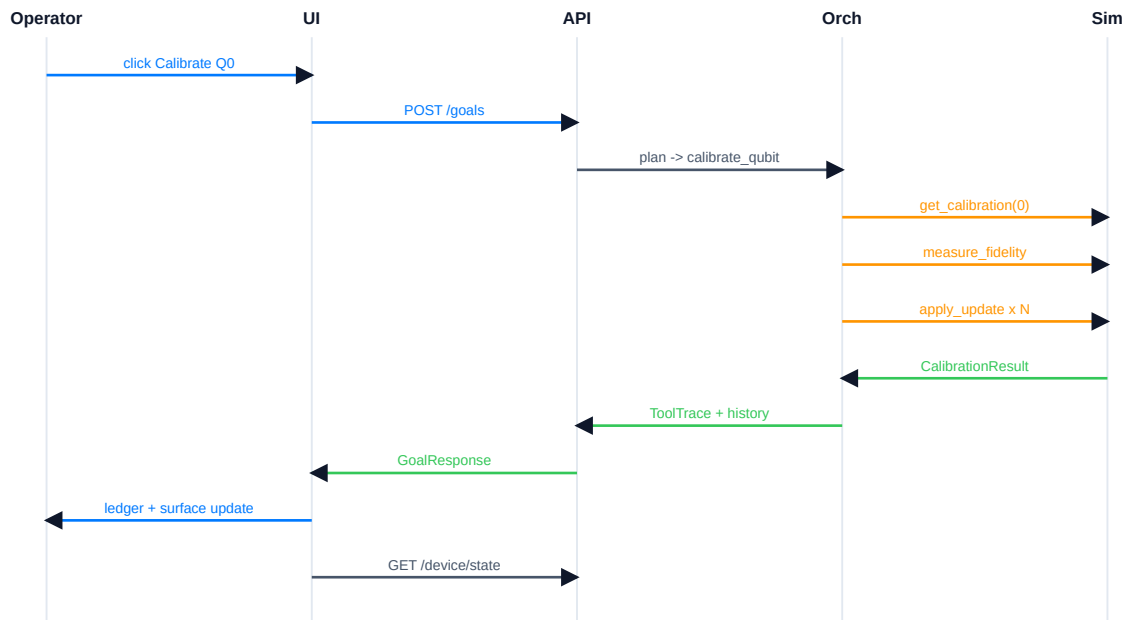


FIG. 10C.1 Happy-path calibrate sequence.

WALKTHROUGH - BELL PAIR

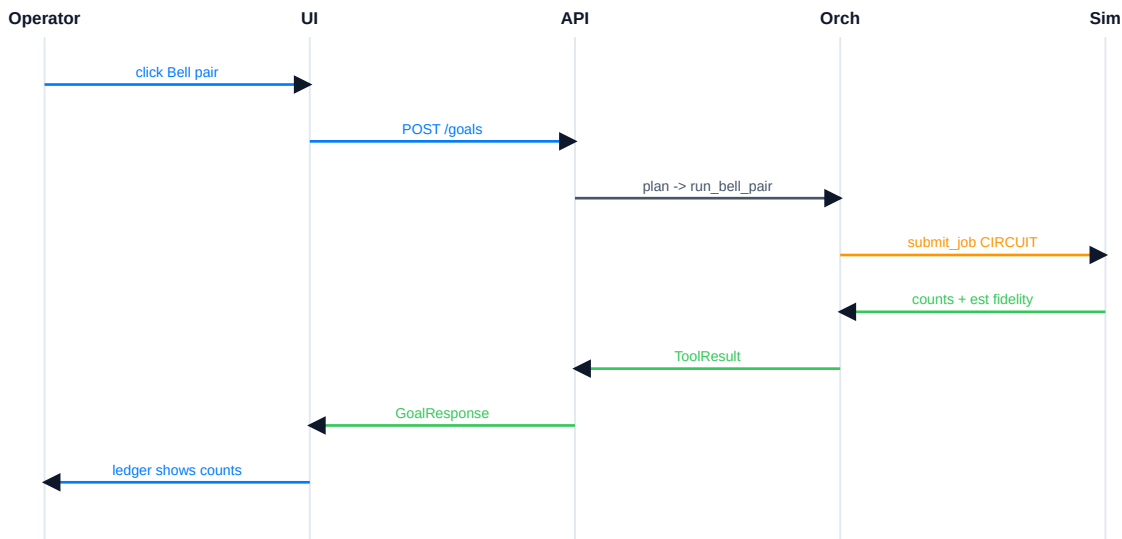


FIG. 10C.2 Bell sequence.

CALIBRATION FAILURE MODES



FIG. 10C.3 Failure modes to hit in steps 7–8 and stub check.

Theory — observability as a trust protocol

Extends Part 2: why traces + metrics + drift form a protocol, not a dashboard aesthetic.

2C.1 Three audiences, one ledger

A **founder** watching the demo needs to see that calibration costs time and sometimes fails. An **operator** needs to see whether the device is ready and what the last tool did. An **agent** needs machine-readable ToolResults and stable tool names. The ToolTrace buffer and /traces endpoint serve all three if the UI does not invent a parallel narrative channel.

2C.2 What would falsify the design

- Calibration “success” without a fidelity history that crossed the threshold.
- UI READY state disagreeing with /device/state.is_ready for the same poll.
- Traces that do not match tools actually invoked (LLM fiction).
- Interface latency that never samples because applies are skipped.
- A hardware port that requires rewriting page.tsx to show drift.

RULE OF THUMB

If swapping the adapter requires redesigning the ledger, the waist was too wide. If swapping the planner requires redesigning metrics, the metrics were tied to NL rather than to tools.

THREE CONCERNS · ONE DIRECTION

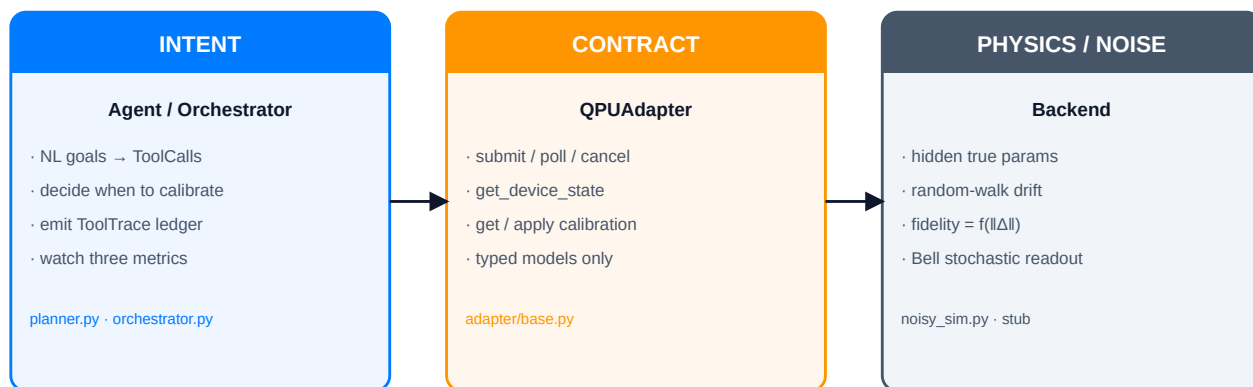


FIG. 2C.1 Intent / contract / physics — falsification targets live at the seams.

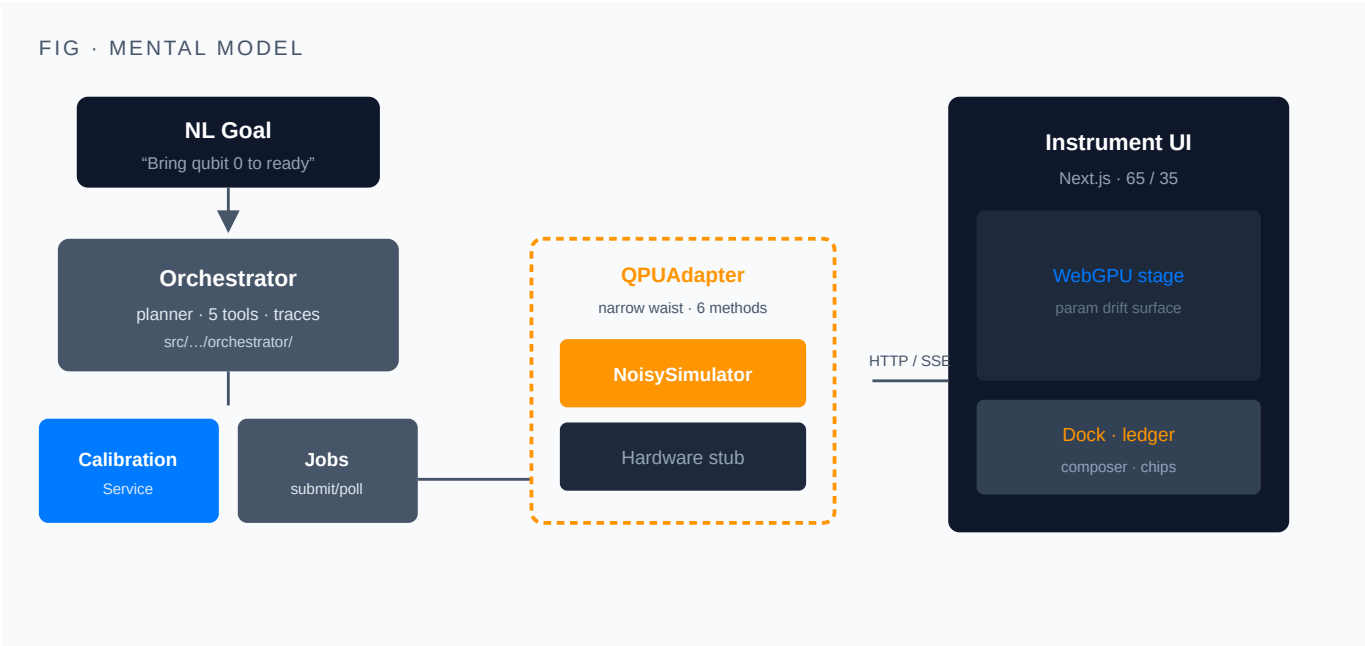


FIG. 2C.2 Stack map for trust-protocol discussion.

Metrics algebra

Formal-ish definitions matching CalibrationMetrics and MetricsAggregator.

```
calibration_success_rate = successes / attempts          # 0 if attempts=0
avg_time_to_calibrated   = total_time_to_cal_s / successes # 0 if successes=0
avg_interface_latency    = total_interface_latency_s / latency_samples

aggregator.p50_ms = percentile_50(latencies_ms)
aggregator.p95_ms = percentile_95(latencies_ms)
aggregator.avg_fidelity = mean(fidelity_samples)
```

Note: orchestrator records tool latencies separately from calibration interface latency. /metrics returns both. UI may emphasise calibration block for the founder narrative.

THE THREE METRICS

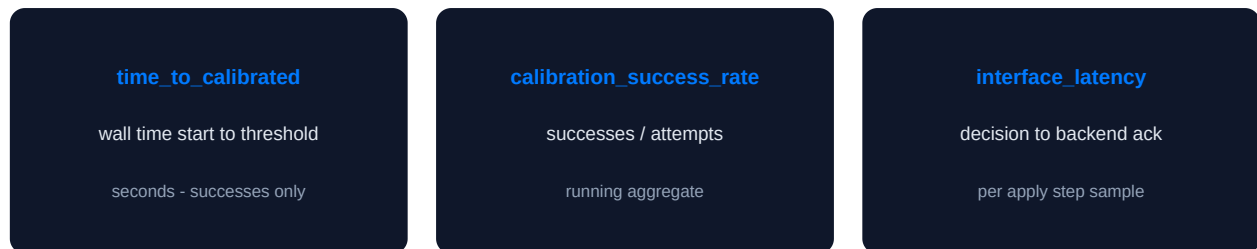


FIG. 6C.1 The three primary dials.

Licence and third-party

This system is MIT-licensed (LICENSE in repo root). UI depends on Next.js, React, three.js, recharts, cmdk, sonner, motion, and shadcn/Radix components — each under their respective licences. Emil Kowalski skills are vendored under `.cursor/skills` for design guidance; they are not runtime dependencies beyond the libraries chosen.

This manual's diagrams are original to the manual source tree under `docs/manual/diagrams`. Screenshots capture the live three-pane UI (`docs/manual/assets/ui/manual-3pane-*.png`). TinyNeuro's manual was studied for structure only; no TinyNeuro content is reproduced.